

MySQL Interview Preparation Using PulseDesk

A primary study guide for MySQL 8, built from the real PulseDesk backend and schema.

This document uses one connected help-desk domain from beginning to end. Each query question repeats the exact input rows it needs, shows the expected result, and explains why the SQL works. The rows are small, illustrative datasets that obey the PulseDesk schema; they are not a dump of a production database.

The SQL targets **MySQL 8.0**. That matters because the PulseDesk schema uses enforced **CHECK** constraints, **JSON_TABLE**, generated columns, common table expressions, and window functions. MySQL 8.0 syntax is used throughout.

How to study this document

For each question, first read the business goal and inspect the shown rows. Predict the output before reading the answer. Then trace the query in logical execution order:

```
FROM / JOIN -> WHERE -> GROUP BY -> HAVING -> window functions
-> SELECT -> DISTINCT -> ORDER BY -> LIMIT / OFFSET
```

Aliases become available at different stages. MySQL permits a **SELECT** alias in **GROUP BY**, **HAVING**, and **ORDER BY**, but generally not in **WHERE**, because **WHERE** is logically evaluated earlier.

Contents

- 1. PulseDesk Database Understanding
- 2. Basic SQL
- 3. Filtering Data
- 4. String Operations
- 5. Date and Time
- 6. Aggregate Functions
- 7. **GROUP BY**
- 8. **HAVING**
- 9. **GROUP BY** + **HAVING** + **COUNT** + **ORDER BY**
- 10. JOINS
- 11. JOIN + **GROUP BY**
- 12. Subqueries
- 13. Conditional Aggregation
- 14. Advanced Interview Query Patterns
- 15. Classic Employee/Salary Interview Questions
- 16. Important MySQL Conceptual Interview Questions
- 17. Common Interview Traps and Mistakes

Part 1 — PulseDesk Database Understanding

PulseDesk Database Overview

PulseDesk is a help-desk and IT asset application. Employees raise tickets, agents work on them, comments record discussion, notifications alert participants, and assets can be assigned to users and linked to tickets.

The database knowledge in this section comes from:

- the JPA entity classes under **com.pulsedesk.entites**;
- Spring Data repositories and their native SQL/JPQL queries;
- repository projections and API DTOs;
- **pulsedesk-dbschema.sql** and the two live MySQL resource scripts;
- service code that defines valid roles, ticket states, priorities, and application behavior.

There is no Flyway or Liquibase configuration. Hibernate is configured with **spring.jpa.hibernate.ddl-auto=none**, so the SQL schema, rather than Hibernate, owns the physical structure.

Tables and their purpose

role

Stores application roles. The seeded values are **admin**, **employee**, and **agent**.

Item	Details
Primary key	id BIGINT AUTO_INCREMENT

Item	Details
Important columns	<code>name VARCHAR(100) NOT NULL UNIQUE</code>
Referenced by	<code>users.role_id</code>
Relationship	One role can belong to many users

users

Stores every PulseDesk account. Employees request support, agents may receive tickets, and admins manage the system.

Item	Details
Primary key	<code>user_id INT AUTO_INCREMENT</code>
Important columns	<code>name, unique email, mobile_number, password hash pwd, status, create_dt</code>
Foreign key	<code>role_id -> role.id</code>
Referenced by	ticket requester/assignee, asset assignee, comment author, notification recipient, and assignment-history actor columns
Allowed status in service code	<code>ACTIVE, INACTIVE</code>

`create_dt` is physically a `VARCHAR(255)`, and the service writes an ISO-style `LocalDateTime` string. That is a schema weakness: date filtering may require conversion, lexical ordering depends on a consistent format, and a real migration should change it to `DATETIME(6)`.

tickets

Stores support requests. A ticket must have a requester and may have an assigned support agent.

Item	Details
Primary key	<code>id BIGINT AUTO_INCREMENT</code>
Important columns	<code>title, description, category, priority, status, created_at, updated_at, resolved_at, version</code>
Foreign keys	<code>requester_id -> users.user_id; nullable assignee_id -> users.user_id</code>
Allowed categories	<code>NETWORK, HARDWARE, SOFTWARE, ACCESS, OTHER</code>
Allowed priorities	<code>LOW, MEDIUM, HIGH</code>
Allowed states	<code>OPEN, IN_PROGRESS, RESOLVED, CLOSED</code>
Relationships	Many tickets can be requested by one user; many can be assigned to one user; one ticket can have many comments, notifications, and linked assets

`version` is incremented by update queries and supports optimistic-concurrency intent. `resolved_at` is set when status becomes `RESOLVED` or `CLOSED`, and cleared if a ticket is reopened.

assets

Stores managed IT equipment and software licences.

Item	Details
Primary key	<code>id BIGINT AUTO_INCREMENT</code>
Important columns	<code>unique tag, type, model, status, purchased_at, nullable coverage_until, timestamps</code>
Foreign key	<code>nullable assigned_to_id -> users.user_id</code>
Allowed types	<code>LAPTOP, MONITOR, PHONE, KEYBOARD, DOCK, LICENSE</code>
Allowed states	<code>IN_USE, IN_STOCK, IN_REPAIR, RETIRED</code>
Relationships	Many assets may currently belong to one user; an asset can appear on many tickets through <code>ticket_assets</code>

ticket_assets

Bridge table connecting tickets to affected assets. Its composite primary key prevents the same asset from being linked to the same ticket twice.

Item	Details
Primary key	<code>(ticket_id, asset_id)</code>
Foreign keys	<code>ticket_id -> tickets.id; asset_id -> assets.id</code>
Important column	<code>created_at</code>
Relationship	Implements the many-to-many relationship between tickets and assets

Deleting a ticket cascades to its bridge rows; deleting a linked asset is restricted.

asset_assignment_history

Preserves every period during which an asset was assigned to a user, including who performed assignment and unassignment.

Item	Details
Primary key	id BIGINT AUTO_INCREMENT
Foreign keys	asset_id, assigned_to_id, nullable assigned_by_id, nullable unassigned_by_id
Important columns	assigned_at, nullable unassigned_at
Generated column	active_asset_id equals asset_id only while unassigned_at IS NULL
Business constraint	Unique active_asset_id permits at most one open assignment per asset

The generated-column technique is useful because MySQL unique indexes allow several **NULL** values. Closed rows produce **NULL**, while an open row produces the asset ID and must be unique.

comments

Stores the conversation on a ticket.

Item	Details
Primary key	id BIGINT AUTO_INCREMENT
Foreign keys	ticket_id -> tickets.id; author_id -> users.user_id
Important columns	nonblank body, created_at
Relationships	Many comments belong to one ticket; many comments may be authored by one user

Deleting a ticket deletes its comments. Deleting a referenced author is restricted.

notifications

Stores comment notifications for ticket participants.

Item	Details
Primary key	id BIGINT AUTO_INCREMENT
Foreign keys	recipient_user_id -> users.user_id; ticket_id -> tickets.id; comment_id -> comments.id
Important columns	type, message, nullable read_at, created_at
Unique rule	(recipient_user_id, comment_id, type) prevents duplicate delivery of the same notification kind
Current type	TICKET_COMMENT_ADDED

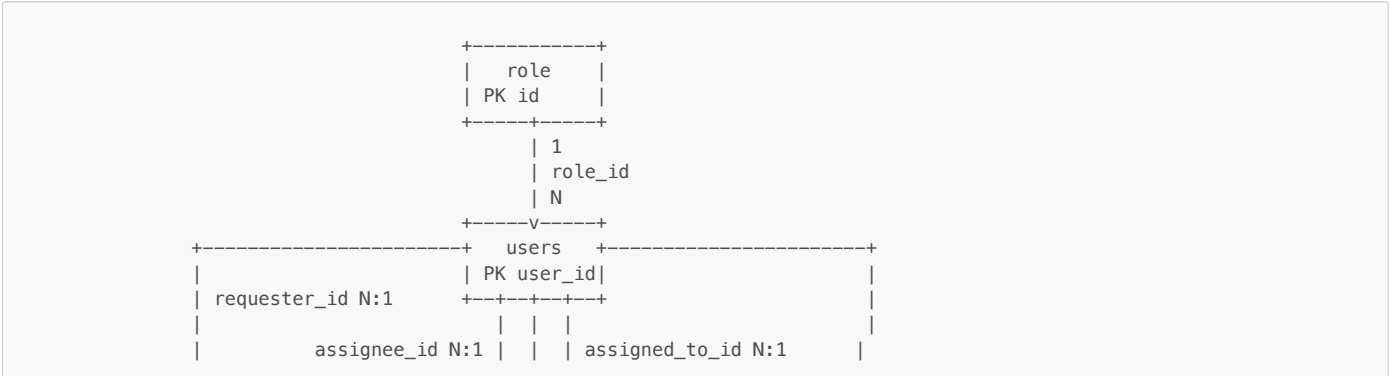
read_at IS NULL means unread. All three foreign keys cascade on parent deletion.

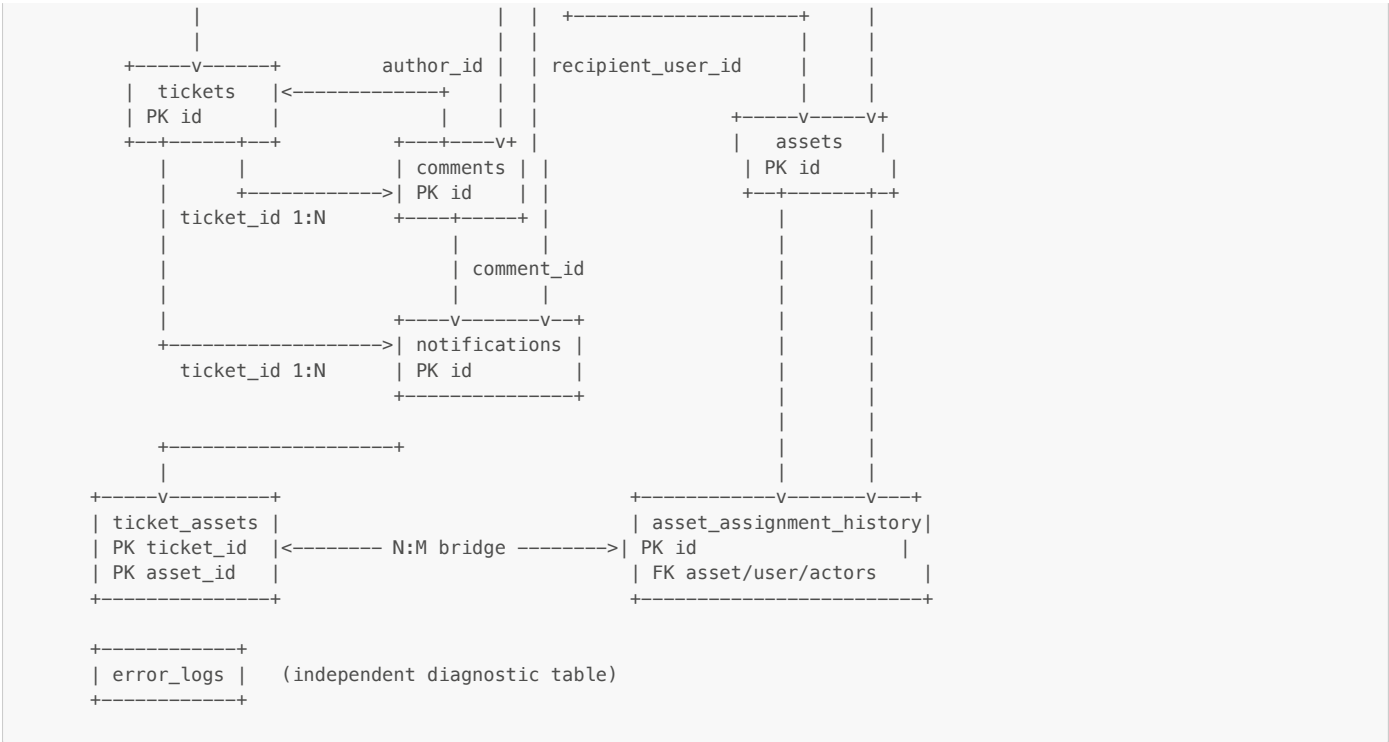
error_logs

Stores unexpected backend errors for diagnosis.

Item	Details
Primary key	application-generated UUID string id VARCHAR(36)
Important columns	exception_class, message, stack_trace, request_uri, request_method, occurred_at
Foreign keys	None

Relationship diagram





The `ticket_assets` box links `tickets` and `assets`; the diagram spreads the bridge horizontally only to keep the text readable.

What the JPA model actually maps

The physical database has rich foreign keys, but the entity classes do **not** declare `@OneToOne`, `@OneToMany`, `@ManyToOne`, `@ManyToMany`, or `@JoinColumn`. They hold scalar IDs such as `requesterId`, `assigneeId`, and `roleId`. Repository native SQL performs the joins.

That design means:

- loading a `TicketEntity` does not automatically load requester or assignee objects;
- SQL controls joins explicitly, which avoids accidental lazy loading but gives up ORM navigation and cascade semantics;
- `AllUsersEntity`, `AssetsUserEntity`, and `UserTicketEntity` behave as native-query result carriers even though they are annotated `@Entity`; they do not correspond to physical tables of those names;
- repository projections such as `TicketDetailsProjection` and `TicketCommentProjection` are a cleaner example of read models assembled by aliases;
- `AssetsEntity` also has no `@Table(name = "assets")`; its repository's custom operations explicitly use native SQL against `assets`, while inherited JPA operations would depend on Hibernate's implicit table naming and deserve verification;
- `TicketEntity` maps `tickets` but omits `@GeneratedValue` even though the database key is auto-incremented; ticket creation currently goes through the stored procedure;
- `AssetsEntity` exposes only `id`, `tag`, `type`, and `model`, while native read models expose assignment/status/date fields; `TicketEntity` omits physical `updated_at` and `version` fields;
- `TicketEntity.id`, `AssetsEntity.id`, notification/comment ticket IDs, and several projections use Java `Integer` while the DDL uses `BIGINT`; aligning them to `Long` would avoid overflow/type mismatch risk.

Real repository query patterns worth recognizing

The application already contains interview-grade patterns:

- optional filters: `(:status IS NULL OR t.status = :status)`;
- two joins to `users`, once as requester and once as assignee;
- `LEFT JOIN` for nullable assignees;
- least-loaded-agent selection with `LEFT JOIN`, `GROUP BY`, `COUNT`, `ORDER BY`, and `LIMIT 1`;
- dynamic sorting through `CASE` expressions;
- page queries with a separate `countQuery`;
- unread notification checks with `read_at IS NULL`;
- atomic auto-assignment using `UPDATE ... WHERE assignee_id IS NULL`;
- a stored procedure using `LAST_INSERT_ID()` and `JSON_TABLE()` to create a ticket plus bridge rows.

Reusable domain vocabulary

Concept	PulseDesk meaning
Parent	A row referenced by another row, such as a ticket referenced by comments
Child	A referencing row, such as a comment belonging to a ticket
Requester	User who raised a ticket

Concept	PulseDesk meaning
Assignee	Agent currently responsible; may be NULL
Active workload	Tickets whose status is OPEN or IN_PROGRESS
Resolution time	Difference between created_at and resolved_at
Unread notification	Notification whose read_at is NULL
Current asset assignment	History row whose unassigned_at is NULL

Part 2 — Basic SQL

SELECT chooses expressions or columns from rows produced by **FROM**. Avoid **SELECT *** in stable application APIs because schema changes can unexpectedly change the result, transfer unused data, and prevent some covering-index optimizations. It remains useful for exploration.

ORDER BY is required whenever row order matters. Without it, MySQL may return rows in any order. **LIMIT** restricts returned rows, and **OFFSET** skips rows, but pagination must use a deterministic order with a unique tiebreaker.

Question 01 — Return every ticket column

Concepts: SELECT, projection, aliases, DISTINCT, ORDER BY, LIMIT, OFFSET

Difficulty: Basic

Relevant Table — tickets

id	title	description	category	priority	status	requester_id	assignee_id	resolved_at	created_at	updated_at	version
101	VPN timeout	VPN disconnects every hour	NETWORK	HIGH	OPEN	2	3	NULL	2026-09-14 09:00	2026-09-14 09:00	1
102	Laptop battery	Battery drains within an hour	HARDWARE	MEDIUM	IN_PROGRESS	5	4	NULL	2026-09-14 10:00	2026-09-14 10:30	1

Think About It

- Which columns must the result expose?
- Does the requirement need deterministic ordering or row limiting?

Question

During exploration, return all stored columns for all tickets.

Answer

```
SELECT *
FROM tickets;
```

Expected Output

id	title	description	category	priority	status	requester_id	assignee_id	resolved_at	created_at	updated_at	version
101	VPN timeout	VPN disconnects every hour	NETWORK	HIGH	OPEN	2	3	NULL	2026-09-14 09:00	2026-09-14 09:00	1
102	Laptop battery	Battery drains within an hour	HARDWARE	MEDIUM	IN_PROGRESS	5	4	NULL	2026-09-14 10:00	2026-09-14 10:30	1

How It Works: **FROM** supplies every ticket row and ***** expands to every column. Production code should name the required columns explicitly.

Why This Is Asked in Interviews

Tests whether you can control result shape and ordering precisely.

Common Mistake

Relying on implicit row order or selecting more columns than the consumer needs.

Interview Follow-up

How would you make the result deterministic when two rows share the same sort value?

Question 02 — Project only the ticket summary columns

Concepts: SELECT, projection, aliases, DISTINCT, ORDER BY, LIMIT, OFFSET

Difficulty: Basic

Relevant Table — tickets

id	title	status	priority	description
101	VPN timeout	OPEN	HIGH	VPN disconnects every hour
102	Laptop battery	IN_PROGRESS	MEDIUM	Battery drains within an hour

Think About It

- Which columns must the result expose?
- Does the requirement need deterministic ordering or row limiting?

Question

Return only `id`, `title`, and `status` for a list screen.

Answer

```
SELECT id, title, status
FROM tickets;
```

Expected Output

id	title	status
101	VPN timeout	OPEN
102	Laptop battery	IN_PROGRESS

How It Works: Projection controls the shape of the result independently of the table's full shape.

Why This Is Asked in Interviews

Tests whether you can control result shape and ordering precisely.

Common Mistake

Relying on implicit row order or selecting more columns than the consumer needs.

Interview Follow-up

How would you make the result deterministic when two rows share the same sort value?

Question 03 — Give output columns API-friendly aliases

Concepts: SELECT, projection, aliases, DISTINCT, ORDER BY, LIMIT, OFFSET

Difficulty: Basic

Relevant Table — users

user_id	name	email
2	Ravi Shah	ravi@pulse.test
3	Meera Nair	meera@pulse.test

Think About It

- Which columns must the result expose?
- Does the requirement need deterministic ordering or row limiting?

Question

Return user IDs and names as `id` and `display_name`.

Answer

```
SELECT user_id AS id,
       name AS display_name
```

```
FROM users;
```

Expected Output

id	display_name
2	Ravi Shah
3	Meera Nair

How It Works: **AS** renames result columns, not stored columns. Aliases are especially important when a projection interface expects exact names.

Why This Is Asked in Interviews

Tests whether you can control result shape and ordering precisely.

Common Mistake

Relying on implicit row order or selecting more columns than the consumer needs.

Interview Follow-up

How would you make the result deterministic when two rows share the same sort value?

Question 04 — Find the distinct ticket categories in use

Concepts: SELECT, projection, aliases, DISTINCT, ORDER BY, LIMIT, OFFSET

Difficulty: Basic

Relevant Table — **tickets**

id	category
101	NETWORK
102	HARDWARE
103	SOFTWARE
104	NETWORK

Think About It

- Which columns must the result expose?
- Does the requirement need deterministic ordering or row limiting?

Question

List each category once.

Answer

```
SELECT DISTINCT category
FROM tickets
ORDER BY category;
```

Expected Output

category
HARDWARE
NETWORK
SOFTWARE

How It Works: **DISTINCT** removes duplicate result rows after projection. It applies to the complete selected tuple when several columns are selected.

Why This Is Asked in Interviews

Tests whether you can control result shape and ordering precisely.

Common Mistake

Relying on implicit row order or selecting more columns than the consumer needs.

Interview Follow-up

How would you make the result deterministic when two rows share the same sort value?

Question 05 — Sort newest tickets first with a stable tie-breaker

Concepts: SELECT, projection, aliases, DISTINCT, ORDER BY, LIMIT, OFFSET

Difficulty: Basic

Relevant Table — tickets

id	title	created_at
101	VPN timeout	2026-09-14 09:00:00
102	Laptop battery	2026-09-14 10:00:00
103	IDE licence	2026-09-14 10:00:00

Think About It

- Which columns must the result expose?
- Does the requirement need deterministic ordering or row limiting?

Question

Show the newest ticket first. For equal timestamps, show the larger ID first.

Answer

```
SELECT id, title, created_at
FROM tickets
ORDER BY created_at DESC, id DESC;
```

Expected Output

id	title	created_at
103	IDE licence	2026-09-14 10:00:00
102	Laptop battery	2026-09-14 10:00:00
101	VPN timeout	2026-09-14 09:00:00

How It Works: MySQL evaluates sort keys left to right. The unique ID makes ordering deterministic when timestamps tie.

Why This Is Asked in Interviews

Tests whether you can control result shape and ordering precisely.

Common Mistake

Relying on implicit row order or selecting more columns than the consumer needs.

Interview Follow-up

How would you make the result deterministic when two rows share the same sort value?

Question 06 — Fetch the second page of tickets

Concepts: SELECT, projection, aliases, DISTINCT, ORDER BY, LIMIT, OFFSET

Difficulty: Basic

Relevant Table — tickets

id	title	created_at
108	MFA reset	2026-09-16 08:00:00
105	Monitor flicker	2026-09-15 11:00:00
102	Laptop battery	2026-09-14 10:00:00
101	VPN timeout	2026-09-14 09:00:00
106	Email crash	2026-09-10 08:00:00

Think About It

- Which columns must the result expose?
- Does the requirement need deterministic ordering or row limiting?

Question

With a page size of two, return page 2 using zero-based paging.

Answer

```
SELECT id, title, created_at
FROM tickets
ORDER BY created_at DESC, id DESC
LIMIT 2 OFFSET 2;
```

Expected Output

id	title	created_at
102	Laptop battery	2026-09-14 10:00:00
101	VPN timeout	2026-09-14 09:00:00

How It Works: The first two ordered rows are skipped and the next two are returned. Large offsets become slower because MySQL still locates and discards earlier rows; keyset pagination is discussed later.

Why This Is Asked in Interviews

Tests whether you can control result shape and ordering precisely.

Common Mistake

Relying on implicit row order or selecting more columns than the consumer needs.

Interview Follow-up

How would you make the result deterministic when two rows share the same sort value?

Part 3 — Filtering Data

WHERE removes individual rows before grouping. Comparisons with **NULL** produce unknown rather than true, so use **IS NULL** or **IS NOT NULL**. Parenthesize mixed **AND/OR** conditions: **AND** has higher precedence, but explicit parentheses make business intent obvious.

Question 07 — Find open tickets

Concepts: WHERE, Boolean predicates, IN, BETWEEN, NULL, CASE

Difficulty: Basic

Relevant Table — **tickets**

id	title	status
101	VPN timeout	OPEN
102	Laptop battery	IN_PROGRESS
103	IDE licence	RESOLVED

Think About It

- Which conditions are evaluated for each individual row?
- Can NULL make the predicate unknown?

Question

Return tickets whose status is **OPEN**.

Answer

```
SELECT id, title
FROM tickets
WHERE status = 'OPEN';
```

Expected Output

id	title
101	VPN timeout

How It Works: **WHERE** tests each source row and retains only rows for which the predicate is true.

Why This Is Asked in Interviews

Tests row filtering, Boolean logic, and SQL's three-valued NULL behavior.

Common Mistake

Mixing AND/OR without clear parentheses or comparing NULL with =.

Interview Follow-up

How would the answer change if one of the filtered columns could be NULL?

Question 08 — Combine filters with AND

Concepts: WHERE, Boolean predicates, IN, BETWEEN, NULL, CASE

Difficulty: Basic

Relevant Table — tickets

id	category	priority	status
101	NETWORK	HIGH	OPEN
105	HARDWARE	HIGH	OPEN
107	NETWORK	LOW	CLOSED

Think About It

- Which conditions are evaluated for each individual row?
- Can NULL make the predicate unknown?

Question

Find open, high-priority network tickets.

Answer

```
SELECT id, category, priority, status
FROM tickets
WHERE status = 'OPEN'
      AND priority = 'HIGH'
      AND category = 'NETWORK';
```

Expected Output

id	category	priority	status
101	NETWORK	HIGH	OPEN

How It Works: Every AND condition must be true for a row to survive.

Why This Is Asked in Interviews

Tests row filtering, Boolean logic, and SQL's three-valued NULL behavior.

Common Mistake

Mixing AND/OR without clear parentheses or comparing NULL with =.

Interview Follow-up

How would the answer change if one of the filtered columns could be NULL?

Question 09 — Combine AND and OR safely

Concepts: WHERE, Boolean predicates, IN, BETWEEN, NULL, CASE

Difficulty: Basic

Relevant Table — tickets

id	category	priority	status
101	NETWORK	HIGH	OPEN
102	HARDWARE	MEDIUM	IN_PROGRESS
103	SOFTWARE	LOW	RESOLVED

id	category	priority	status
105	HARDWARE	HIGH	OPEN

Think About It

- Which conditions are evaluated for each individual row?
- Can NULL make the predicate unknown?

Question

Find unresolved tickets (**OPEN** or **IN_PROGRESS**) that are high priority.

Answer

```
SELECT id, status, priority
FROM tickets
WHERE status IN ('OPEN', 'IN_PROGRESS')
AND priority = 'HIGH';
```

Expected Output

id	status	priority
101	OPEN	HIGH
105	OPEN	HIGH

How It Works: **IN** expresses the status alternatives as one condition. The equivalent form is **WHERE (status = 'OPEN' OR status = 'IN_PROGRESS')** **AND priority = 'HIGH'**.

Why This Is Asked in Interviews

Tests row filtering, Boolean logic, and SQL's three-valued NULL behavior.

Common Mistake

Mixing AND/OR without clear parentheses or comparing NULL with =.

Interview Follow-up

How would the answer change if one of the filtered columns could be NULL?

Question 10 — Exclude closed tickets with **NOT**

Concepts: WHERE, Boolean predicates, IN, BETWEEN, NULL, CASE

Difficulty: Basic

Relevant Table — **tickets**

id	title	status
101	VPN timeout	OPEN
103	IDE licence	RESOLVED
104	Payroll access	CLOSED

Think About It

- Which conditions are evaluated for each individual row?
- Can NULL make the predicate unknown?

Question

Return tickets that are not closed.

Answer

```
SELECT id, title, status
FROM tickets
WHERE NOT status = 'CLOSED';
```

Expected Output

id	title	status
101	VPN timeout	OPEN
103	IDE licence	RESOLVED

How It Works: **NOT** reverses the Boolean comparison. `status <> 'CLOSED'` is the usual shorter form. Neither form matches **NULL**, although PulseDesk makes ticket status non-null.

Why This Is Asked in Interviews

Tests row filtering, Boolean logic, and SQL's three-valued NULL behavior.

Common Mistake

Mixing AND/OR without clear parentheses or comparing NULL with =.

Interview Follow-up

How would the answer change if one of the filtered columns could be NULL?

Question 11 — Filter a set with **IN**

Concepts: WHERE, Boolean predicates, IN, BETWEEN, NULL, CASE

Difficulty: Basic

Relevant Table — tickets

id	category
101	NETWORK
102	HARDWARE
103	SOFTWARE
104	ACCESS

Think About It

- Which conditions are evaluated for each individual row?
- Can NULL make the predicate unknown?

Question

Return infrastructure-related tickets in **NETWORK** or **HARDWARE**.

Answer

```
SELECT id, category
FROM tickets
WHERE category IN ('NETWORK', 'HARDWARE');
```

Expected Output

id	category
101	NETWORK
102	HARDWARE

How It Works: **IN** is a readable equality test against several constants.

Why This Is Asked in Interviews

Tests row filtering, Boolean logic, and SQL's three-valued NULL behavior.

Common Mistake

Mixing AND/OR without clear parentheses or comparing NULL with =.

Interview Follow-up

How would the answer change if one of the filtered columns could be NULL?

Question 12 — Exclude a set with **NOT IN**

Concepts: WHERE, Boolean predicates, IN, BETWEEN, NULL, CASE

Difficulty: Basic

Relevant Table — **assets**

id	tag	status
201	LT-001	IN_USE
202	MON-001	IN_REPAIR
204	DOCK-001	IN_STOCK
206	PHONE-001	RETIRED

Think About It

- Which conditions are evaluated for each individual row?
- Can NULL make the predicate unknown?

Question

Return assets available for normal inventory views, excluding repair and retired assets.

Answer

```
SELECT id, tag, status
FROM assets
WHERE status NOT IN ('IN_REPAIR', 'RETIRED');
```

Expected Output

id	tag	status
201	LT-001	IN_USE
204	DOCK-001	IN_STOCK

How It Works: The predicate rejects either listed value. Beware **NOT IN (subquery)** if the subquery can return **NULL**; one **NULL** can make every comparison unknown. **NOT EXISTS** is safer for anti-matching.

Why This Is Asked in Interviews

Tests row filtering, Boolean logic, and SQL's three-valued NULL behavior.

Common Mistake

Mixing AND/OR without clear parentheses or comparing NULL with =.

Interview Follow-up

How would the answer change if one of the filtered columns could be NULL?

Question 13 — Use **BETWEEN** for an inclusive range

Concepts: WHERE, Boolean predicates, IN, BETWEEN, NULL, CASE

Difficulty: Basic

Relevant Table — **assets**

id	tag	purchased_at
201	LT-001	2025-01-15
202	MON-001	2025-02-10
203	LT-002	2025-04-12

Think About It

- Which conditions are evaluated for each individual row?
- Can NULL make the predicate unknown?

Question

Find assets purchased from 1 January through 31 March 2025.

Answer

```
SELECT id, tag, purchased_at
FROM assets
```

```
WHERE purchased_at BETWEEN '2025-01-01' AND '2025-03-31';
```

Expected Output

id	tag	purchased_at
201	LT-001	2025-01-15
202	MON-001	2025-02-10

How It Works: **BETWEEN** includes both endpoints. For **DATETIME**, a half-open range (`>= start AND < next_day`) is safer than ending at an assumed last time of day.

Why This Is Asked in Interviews

Tests row filtering, Boolean logic, and SQL's three-valued NULL behavior.

Common Mistake

Mixing AND/OR without clear parentheses or comparing NULL with =.

Interview Follow-up

How would the answer change if one of the filtered columns could be NULL?

Question 14 — Find unassigned tickets with **IS NULL**

Concepts: WHERE, Boolean predicates, IN, BETWEEN, NULL, CASE

Difficulty: Basic

Relevant Table — **tickets**

id	title	assignee_id
101	VPN timeout	3
105	Monitor flicker	NULL
108	MFA reset	6

Think About It

- Which conditions are evaluated for each individual row?
- Can NULL make the predicate unknown?

Question

Return tickets that have no assignee.

Answer

```
SELECT id, title
FROM tickets
WHERE assignee_id IS NULL;
```

Expected Output

id	title
105	Monitor flicker

How It Works: **NULL** means missing/unknown. `assignee_id = NULL` never evaluates to true; SQL requires **IS NULL**.

Why This Is Asked in Interviews

Tests row filtering, Boolean logic, and SQL's three-valued NULL behavior.

Common Mistake

Mixing AND/OR without clear parentheses or comparing NULL with =.

Interview Follow-up

How would the answer change if one of the filtered columns could be NULL?

Question 15 — Find tickets that have a resolution timestamp

Concepts: WHERE, Boolean predicates, IN, BETWEEN, NULL, CASE

Difficulty: Basic

Relevant Table — **tickets**

id	status	resolved_at
101	OPEN	NULL
103	RESOLVED	2026-09-02 12:00:00
104	CLOSED	2026-08-20 14:00:00

Think About It

- Which conditions are evaluated for each individual row?
- Can NULL make the predicate unknown?

Question

Return tickets for which resolution time can be measured.

Answer

```
SELECT id, status, resolved_at
FROM tickets
WHERE resolved_at IS NOT NULL;
```

Expected Output

id	status	resolved_at
103	RESOLVED	2026-09-02 12:00:00
104	CLOSED	2026-08-20 14:00:00

How It Works: **IS NOT NULL** keeps only known timestamps.

Why This Is Asked in Interviews

Tests row filtering, Boolean logic, and SQL's three-valued NULL behavior.

Common Mistake

Mixing AND/OR without clear parentheses or comparing NULL with =.

Interview Follow-up

How would the answer change if one of the filtered columns could be NULL?

Question 16 — Label tickets with **CASE**

Concepts: WHERE, Boolean predicates, IN, BETWEEN, NULL, CASE

Difficulty: Basic

Relevant Table — **tickets**

id	status	priority
101	OPEN	HIGH
102	IN_PROGRESS	MEDIUM
103	RESOLVED	LOW

Think About It

- Which conditions are evaluated for each individual row?
- Can NULL make the predicate unknown?

Question

Label unresolved high-priority tickets **URGENT**, other unresolved tickets **ACTIVE**, and completed tickets **DONE**.

Answer

```
SELECT id,
       CASE
         WHEN status IN ('OPEN', 'IN_PROGRESS') AND priority = 'HIGH' THEN 'URGENT'
         WHEN status IN ('OPEN', 'IN_PROGRESS') THEN 'ACTIVE'
```

```
        ELSE 'DONE'
      END AS work_bucket
FROM tickets
ORDER BY id;
```

Expected Output

id	work_bucket
101	URGENT
102	ACTIVE
103	DONE

How It Works: A searched **CASE** evaluates **WHEN** clauses top to bottom and returns the first match, so the more specific urgent rule must come first.

Why This Is Asked in Interviews

Tests row filtering, Boolean logic, and SQL's three-valued NULL behavior.

Common Mistake

Mixing AND/OR without clear parentheses or comparing NULL with =.

Interview Follow-up

How would the answer change if one of the filtered columns could be NULL?

Part 4 — String Operations

MySQL string functions clean, search, combine, and derive text. **LIKE** uses **%** for any number of characters and **_** for exactly one character. Case sensitivity depends on collation; PulseDesk uses **utf8mb4_0900_ai_ci**, where **ai_ci** means accent-insensitive and case-insensitive. Apply functions carefully in indexed predicates: **LOWER(column)** often prevents a normal index from being used unless a matching functional index exists.

Question 17 — Search ticket titles by prefix

Concepts: MySQL string functions, LIKE, wildcards, text normalization

Difficulty: Basic

Relevant Table — tickets

id	title
101	VPN timeout
102	Laptop battery
109	VPN account locked

Think About It

- Is the match a prefix, suffix, fixed character, or substring?
- Are bytes and characters the same for this function?

Question

Find ticket titles that begin with **VPN**.

Answer

```
SELECT id, title
FROM tickets
WHERE title LIKE 'VPN%';
```

Expected Output

id	title
101	VPN timeout
109	VPN account locked

How It Works: **%** matches zero or more trailing characters. A fixed prefix can often use an index; a leading wildcard usually cannot.

Why This Is Asked in Interviews

Tests practical text search, parsing, and MySQL string-function behavior.

Common Mistake

Using a leading wildcard or a function on an indexed column without considering scan cost.

Interview Follow-up

How would collation or multibyte text change the result?

Question 18 — Search descriptions with wildcards

Concepts: MySQL string functions, LIKE, wildcards, text normalization

Difficulty: Basic

Relevant Table — tickets

id	description
101	VPN disconnects every hour
102	Battery drains within an hour
103	IntelliJ licence has expired

Think About It

- Is the match a prefix, suffix, fixed character, or substring?
- Are bytes and characters the same for this function?

Question

Find descriptions containing the word hour anywhere.

Answer

```
SELECT id, description
FROM tickets
WHERE description LIKE '%hour%';
```

Expected Output

id	description
101	VPN disconnects every hour
102	Battery drains within an hour

How It Works: A leading and trailing % allow any text on both sides. On large text datasets, use PulseDesk's full-text index rather than a leading-wildcard scan.

Why This Is Asked in Interviews

Tests practical text search, parsing, and MySQL string-function behavior.

Common Mistake

Using a leading wildcard or a function on an indexed column without considering scan cost.

Interview Follow-up

How would collation or multibyte text change the result?

Question 19 — Match exactly one wildcard character

Concepts: MySQL string functions, LIKE, wildcards, text normalization

Difficulty: Basic

Relevant Table — assets

id	tag
201	LT-001
202	LT-011
203	LT-1001
204	MON-001

Think About It

- Is the match a prefix, suffix, fixed character, or substring?
- Are bytes and characters the same for this function?

Question

Match **LT-0**, followed by exactly two characters.

Answer

```
SELECT id, tag
FROM assets
WHERE tag LIKE 'LT-0__';
```

Expected Output

id	tag
201	LT-001
202	LT-011

How It Works: Each underscore matches exactly one character, whereas **%** could match any length.

Why This Is Asked in Interviews

Tests practical text search, parsing, and MySQL string-function behavior.

Common Mistake

Using a leading wildcard or a function on an indexed column without considering scan cost.

Interview Follow-up

How would collation or multibyte text change the result?

Question 20 — Build a readable ticket label with **CONCAT**

Concepts: MySQL string functions, LIKE, wildcards, text normalization

Difficulty: Basic

Relevant Table — **tickets**

id	title	status
101	VPN timeout	OPEN
103	IDE licence	RESOLVED

Think About It

- Is the match a prefix, suffix, fixed character, or substring?
- Are bytes and characters the same for this function?

Question

Produce labels such as **#101 - VPN timeout [OPEN]**.

Answer

```
SELECT CONCAT('#', id, ' - ', title, ' [', status, ']') AS ticket_label
FROM tickets
ORDER BY id;
```

Expected Output

ticket_label
#101 - VPN timeout [OPEN]
#103 - IDE licence [RESOLVED]

How It Works: **CONCAT** converts values to strings and joins them. If any argument is **NULL**, **CONCAT** returns **NULL**; use **CONCAT_WS** or **COALESCE** when nullable parts are expected.

Why This Is Asked in Interviews

Tests practical text search, parsing, and MySQL string-function behavior.

Common Mistake

Using a leading wildcard or a function on an indexed column without considering scan cost.

Interview Follow-up

How would collation or multibyte text change the result?

Question 21 — Extract email domains

Concepts: MySQL string functions, LIKE, wildcards, text normalization

Difficulty: Basic

Relevant Table — users

user_id	email
2	ravi@pulse.test
3	meera@support.test
4	kabir@pulse.test

Think About It

- Is the match a prefix, suffix, fixed character, or substring?
- Are bytes and characters the same for this function?

Question

Return the portion after @ as the email domain.

Answer

```
SELECT user_id,
       SUBSTRING_INDEX(email, '@', -1) AS email_domain
FROM users
ORDER BY user_id;
```

Expected Output

user_id	email_domain
2	pulse.test
3	support.test
4	pulse.test

How It Works: SUBSTRING_INDEX(text, delimiter, -1) returns the part after the final delimiter.

Why This Is Asked in Interviews

Tests practical text search, parsing, and MySQL string-function behavior.

Common Mistake

Using a leading wildcard or a function on an indexed column without considering scan cost.

Interview Follow-up

How would collation or multibyte text change the result?

Question 22 — Normalize and validate comment text

Concepts: MySQL string functions, LIKE, wildcards, text normalization

Difficulty: Basic

Relevant Table — comments

id	body
1001	VPN drops every hour
1002	OK

Think About It

- Is the match a prefix, suffix, fixed character, or substring?
- Are bytes and characters the same for this function?

Question

Trim surrounding spaces, uppercase the text, and return its trimmed character length.

Answer

```
SELECT id,
       UPPER(TRIM(body)) AS normalized_body,
       CHAR_LENGTH(TRIM(body)) AS character_count
FROM comments
ORDER BY id;
```

Expected Output

id	normalized_body	character_count
1001	VPN DROPS EVERY HOUR	20
1002	OK	2

How It Works: `TRIM` removes edge spaces, `UPPER` changes case, and `CHAR_LENGTH` counts characters. `LENGTH` counts bytes, which can differ for `utf8mb4` text.

Why This Is Asked in Interviews

Tests practical text search, parsing, and MySQL string-function behavior.

Common Mistake

Using a leading wildcard or a function on an indexed column without considering scan cost.

Interview Follow-up

How would collation or multibyte text change the result?

Question 23 — Find user names ending with specific text

Concepts: MySQL string functions, LIKE, wildcards, text normalization

Difficulty: Basic

Relevant Table — `users`

user_id	name
2	Ravi Shah
3	Meera Nair
4	Kabir Rao
5	Neha Joshi

Think About It

- Is the match a prefix, suffix, fixed character, or substring?
- Are bytes and characters the same for this function?

Question

Find users whose names end with `Rao`.

Answer

```
SELECT user_id, name
FROM users
WHERE name LIKE '%Rao';
```

Expected Output

user_id	name
4	Kabir Rao

How It Works: The leading `%` accepts any prefix, while the fixed final text requires the value to end in `Rao`.

Why This Is Asked in Interviews

Tests practical text search, parsing, and MySQL string-function behavior.

Common Mistake

Using a leading wildcard or a function on an indexed column without considering scan cost.

Interview Follow-up

How would collation or multibyte text change the result?

Question 24 — Read fixed parts of an asset tag with LEFT and RIGHT

Concepts: MySQL string functions, LIKE, wildcards, text normalization

Difficulty: Basic

Relevant Table — assets

id	tag
201	LT-001
202	MON-042

Think About It

- Is the match a prefix, suffix, fixed character, or substring?
- Are bytes and characters the same for this function?

Question

Return the first two and last three characters of each tag.

Answer

```
SELECT id,
       tag,
       LEFT(tag, 2) AS tag_prefix,
       RIGHT(tag, 3) AS tag_number
FROM assets
ORDER BY id;
```

Expected Output

id	tag	tag_prefix	tag_number
201	LT-001	LT	001
202	MON-042	MO	042

How It Works: LEFT and RIGHT return a requested number of characters from an edge. This example also shows why a variable-length prefix is better extracted with a delimiter function than with LEFT(tag, 2).

Why This Is Asked in Interviews

Tests practical text search, parsing, and MySQL string-function behavior.

Common Mistake

Using a leading wildcard or a function on an indexed column without considering scan cost.

Interview Follow-up

How would collation or multibyte text change the result?

Question 25 — Normalize an email username with SUBSTRING, LOWER, and REPLACE

Concepts: MySQL string functions, LIKE, wildcards, text normalization

Difficulty: Basic

Relevant Table — users

user_id	email
2	Ravi.Shah@pulse.test
3	MEERA.NAIR@pulse.test

Think About It

- Is the match a prefix, suffix, fixed character, or substring?
- Are bytes and characters the same for this function?

Question

Extract the text before @, lowercase it, and replace periods with spaces.

Answer

```
SELECT user_id,
       REPLACE(
         LOWER(SUBSTRING(email, 1, LOCATE('@', email) - 1)),
         '.', ' '),
       ) AS email_user_label
FROM users
ORDER BY user_id;
```

Expected Output

user_id	email_user_label
2	ravi shah
3	meera nair

How It Works: LOCATE finds the delimiter position, SUBSTRING takes everything before it, LOWER normalizes case, and REPLACE changes the separator.

Why This Is Asked in Interviews

Tests practical text search, parsing, and MySQL string-function behavior.

Common Mistake

Using a leading wildcard or a function on an indexed column without considering scan cost.

Interview Follow-up

How would collation or multibyte text change the result?

Question 26 — Compare byte length and character length

Concepts: MySQL string functions, LIKE, wildcards, text normalization

Difficulty: Basic

Relevant Table — comments

id	body
1001	VPN cafe
1002	VPN café

Think About It

- Is the match a prefix, suffix, fixed character, or substring?
- Are bytes and characters the same for this function?

Question

Compare LENGTH and CHAR_LENGTH for ASCII and multibyte text in utf8mb4.

Answer

```
SELECT id,
       body,
       LENGTH(body) AS byte_length,
       CHAR_LENGTH(body) AS character_length
FROM comments
ORDER BY id;
```

Expected Output

id	body	byte_length	character_length
1001	VPN cafe	8	8
1002	VPN café	9	8

How It Works: UTF-8 stores é with two bytes but it is one character. `LENGTH` counts bytes; `CHAR_LENGTH` counts characters.

Why This Is Asked in Interviews

Tests practical text search, parsing, and MySQL string-function behavior.

Common Mistake

Using a leading wildcard or a function on an indexed column without considering scan cost.

Interview Follow-up

How would collation or multibyte text change the result?

Part 5 — Date and Time

PulseDesk correctly stores operational timestamps as `DATETIME(6)`. Prefer half-open ranges for filtering: `column >= start AND column < end`. They include every fractional timestamp and remain index-friendly. Applying `DATE(column)` in a `WHERE` predicate is readable but can prevent efficient range use.

`TIMESTAMPDIFF(unit, start, end)` returns whole unit boundaries. Use seconds or minutes and divide if fractional precision matters.

Question 27 — Find tickets created today

Concepts: MySQL date/time functions, half-open ranges, intervals, duration

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	title	created_at
105	Monitor flicker	2026-09-15 11:00:00
108	MFA reset	2026-09-16 08:00:00
109	VPN account locked	2026-09-16 18:30:00

Think About It

- Is this a calendar period or a rolling duration?
- Can a half-open range keep the timestamp predicate index-friendly?

Question

Assuming `CURRENT_DATE` is `2026-09-16`, find tickets created today.

Answer

```
SELECT id, title, created_at
FROM tickets
WHERE created_at >= CURRENT_DATE
      AND created_at < DATE_ADD(CURRENT_DATE, INTERVAL 1 DAY);
```

Expected Output

id	title	created_at
108	MFA reset	2026-09-16 08:00:00
109	VPN account locked	2026-09-16 18:30:00

How It Works: The half-open interval includes the entire current day, including microseconds, without wrapping the indexed column in a function. `CURRENT_DATE` is midnight today; `DATE_ADD` produces midnight tomorrow.

Why This Is Asked in Interviews

Tests boundary handling, MySQL date functions, and precise time-window interpretation.

Common Mistake

Using `23:59:59` as an inclusive end or applying `DATE()` to an indexed filter column.

Interview Follow-up

How would you make the query safe for time zones and fractional seconds?

Question 28 — Find tickets from the previous seven days

Concepts: MySQL date/time functions, half-open ranges, intervals, duration

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	created_at
104	2026-08-20 10:00:00
106	2026-09-10 08:00:00
101	2026-09-14 09:00:00
108	2026-09-16 08:00:00

Think About It

- Is this a calendar period or a rolling duration?
- Can a half-open range keep the timestamp predicate index-friendly?

Question

As of `2026-09-16 23:59:59`, return tickets created since midnight seven calendar days earlier.

Answer

```
SELECT id, created_at
FROM tickets
WHERE created_at >= '2026-09-10 00:00:00'
      AND created_at < '2026-09-17 00:00:00'
ORDER BY created_at;
```

Expected Output

id	created_at
106	2026-09-10 08:00:00
101	2026-09-14 09:00:00
108	2026-09-16 08:00:00

How It Works: Explicit bounds make the sample reproducible. A rolling 168-hour window and seven calendar dates are different requirements; clarify which the interviewer wants.

Why This Is Asked in Interviews

Tests boundary handling, MySQL date functions, and precise time-window interpretation.

Common Mistake

Using `23:59:59` as an inclusive end or applying `DATE()` to an indexed filter column.

Interview Follow-up

How would you make the query safe for time zones and fractional seconds?

Question 29 — Calculate ticket resolution time

Concepts: MySQL date/time functions, half-open ranges, intervals, duration

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	created_at	resolved_at
103	2026-09-01 09:00:00	2026-09-02 12:00:00
104	2026-08-20 10:00:00	2026-08-20 14:00:00
101	2026-09-14 09:00:00	NULL

Think About It

- Is this a calendar period or a rolling duration?
- Can a half-open range keep the timestamp predicate index-friendly?

Question

For completed tickets, return whole hours to resolution.

Answer

```
SELECT id,
       TIMESTAMPDIFF(HOUR, created_at, resolved_at) AS resolution_hours
FROM tickets
WHERE resolved_at IS NOT NULL
ORDER BY id;
```

Expected Output

id	resolution_hours
103	27
104	4

How It Works: The function subtracts the start from the end in whole hours. Filtering out **NULL** prevents unresolved rows from producing **NULL** durations.

Why This Is Asked in Interviews

Tests boundary handling, MySQL date functions, and precise time-window interpretation.

Common Mistake

Using 23:59:59 as an inclusive end or applying DATE() to an indexed filter column.

Interview Follow-up

How would you make the query safe for time zones and fractional seconds?

Question 30 — Label ticket age with date arithmetic and CASE

Concepts: MySQL date/time functions, half-open ranges, intervals, duration

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	status	created_at
101	OPEN	2026-09-14 09:00:00
105	OPEN	2026-09-15 11:00:00
108	OPEN	2026-09-16 08:00:00

Think About It

- Is this a calendar period or a rolling duration?
- Can a half-open range keep the timestamp predicate index-friendly?

Question

At **2026-09-16 12:00:00**, label unresolved tickets older than 24 hours as **STALE**; otherwise label them **FRESH**.

Answer

```
SELECT id,
       CASE
         WHEN created_at < '2026-09-15 12:00:00' THEN 'STALE'
         ELSE 'FRESH'
       END AS age_bucket
FROM tickets
WHERE status IN ('OPEN', 'IN_PROGRESS')
ORDER BY id;
```

Expected Output

id	age_bucket
101	STALE

id	age_bucket
105	FRESH
108	FRESH

How It Works: Comparing the timestamp to a computed cutoff is simpler and more index-friendly than calculating the age for every row. Live SQL could use `NOW() - INTERVAL 24 HOUR`.

Why This Is Asked in Interviews

Tests boundary handling, MySQL date functions, and precise time-window interpretation.

Common Mistake

Using 23:59:59 as an inclusive end or applying `DATE()` to an indexed filter column.

Interview Follow-up

How would you make the query safe for time zones and fractional seconds?

Question 31 — Extract calendar parts for reporting

Concepts: MySQL date/time functions, half-open ranges, intervals, duration

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	created_at
104	2026-08-20 10:00:00
103	2026-09-01 09:00:00

Think About It

- Is this a calendar period or a rolling duration?
- Can a half-open range keep the timestamp predicate index-friendly?

Question

Show each ticket's year, month number, and month label.

Answer

```
SELECT id,
       YEAR(created_at) AS created_year,
       MONTH(created_at) AS created_month,
       DATE_FORMAT(created_at, '%Y-%m') AS month_label
FROM tickets
ORDER BY id;
```

Expected Output

id	created_year	created_month	month_label
103	2026	9	2026-09
104	2026	8	2026-08

How It Works: Extraction produces numeric parts; `DATE_FORMAT` produces a presentation label. Group chronologically by year/month or an actual month date, not a month name alone.

Why This Is Asked in Interviews

Tests boundary handling, MySQL date functions, and precise time-window interpretation.

Common Mistake

Using 23:59:59 as an inclusive end or applying `DATE()` to an indexed filter column.

Interview Follow-up

How would you make the query safe for time zones and fractional seconds?

Question 32 — Find assets whose coverage has expired

Concepts: MySQL date/time functions, half-open ranges, intervals, duration

Difficulty: Basic / Intermediate

Relevant Table — **assets**

id	tag	coverage_until
201	LT-001	2028-01-15
205	LIC-001	2026-08-31
206	PHONE-001	NULL

Think About It

- Is this a calendar period or a rolling duration?
- Can a half-open range keep the timestamp predicate index-friendly?

Question

As of 16 September 2026, return assets with a known coverage date that has passed.

Answer

```
SELECT id, tag, coverage_until
FROM assets
WHERE coverage_until < '2026-09-16';
```

Expected Output

id	tag	coverage_until
205	LIC-001	2026-08-31

How It Works: A simple date comparison excludes future dates and also excludes **NULL**, because an unknown comparison is not true.

Why This Is Asked in Interviews

Tests boundary handling, MySQL date functions, and precise time-window interpretation.

Common Mistake

Using 23:59:59 as an inclusive end or applying DATE() to an indexed filter column.

Interview Follow-up

How would you make the query safe for time zones and fractional seconds?

Question 33 — Query the string-based user creation date safely

Concepts: MySQL date/time functions, half-open ranges, intervals, duration

Difficulty: Basic / Intermediate

Relevant Table — **users**

user_id	name	create_dt
2	Ravi Shah	2026-01-10T09:30:00
3	Meera Nair	2026-09-01T11:15:00
4	Kabir Rao	2025-12-20T08:00:00

Think About It

- Is this a calendar period or a rolling duration?
- Can a half-open range keep the timestamp predicate index-friendly?

Question

Find users created during 2026 despite **create_dt** being a **VARCHAR**.

Answer

```
SELECT user_id, name, create_dt
FROM users
WHERE STR_TO_DATE(create_dt, '%Y-%m-%dT%H:%i:%s') >= '2026-01-01'
      AND STR_TO_DATE(create_dt, '%Y-%m-%dT%H:%i:%s') < '2027-01-01';
```

Expected Output

user_id	name	create_dt
2	Ravi Shah	2026-01-10T09:30:00
3	Meera Nair	2026-09-01T11:15:00

How It Works: `STR_TO_DATE` parses the service's ISO-like text. This conversion must run per row and is difficult to index; changing the column to `DATETIME(6)` is the real fix.

Why This Is Asked in Interviews

Tests boundary handling, MySQL date functions, and precise time-window interpretation.

Common Mistake

Using 23:59:59 as an inclusive end or applying `DATE()` to an indexed filter column.

Interview Follow-up

How would you make the query safe for time zones and fractional seconds?

Question 34 — Find tickets created yesterday

Concepts: MySQL date/time functions, half-open ranges, intervals, duration

Difficulty: Basic / Intermediate

Relevant Table — `tickets`

id	title	created_at
105	Monitor flicker	2026-09-15 11:00:00
108	MFA reset	2026-09-16 08:00:00
109	Guest access	2026-09-14 18:00:00

Think About It

- Is this a calendar period or a rolling duration?
- Can a half-open range keep the timestamp predicate index-friendly?

Question

Assuming `CURRENT_DATE` is `2026-09-16`, find tickets created yesterday.

Answer

```
SELECT id, title, created_at
FROM tickets
WHERE created_at >= CURRENT_DATE - INTERVAL 1 DAY
AND created_at < CURRENT_DATE;
```

Expected Output

id	title	created_at
105	Monitor flicker	2026-09-15 11:00:00

How It Works: The lower bound is yesterday at midnight and the exclusive upper bound is today at midnight. It covers the whole previous calendar day.

Why This Is Asked in Interviews

Tests boundary handling, MySQL date functions, and precise time-window interpretation.

Common Mistake

Using 23:59:59 as an inclusive end or applying `DATE()` to an indexed filter column.

Interview Follow-up

How would you make the query safe for time zones and fractional seconds?

Question 35 — Find tickets from the last 30 days

Concepts: MySQL date/time functions, half-open ranges, intervals, duration

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	created_at
104	2026-08-20 10:00:00
107	2026-08-15 09:00:00
101	2026-09-14 09:00:00

Think About It

- Is this a calendar period or a rolling duration?
- Can a half-open range keep the timestamp predicate index-friendly?

Question

Assuming `NOW()` is `2026-09-16 12:00:00`, find tickets from the rolling previous 30 days.

Answer

```
SELECT id, created_at
FROM tickets
WHERE created_at >= DATE_SUB(NOW(), INTERVAL 30 DAY)
AND created_at <= NOW()
ORDER BY created_at;
```

Expected Output

id	created_at
104	2026-08-20 10:00:00
101	2026-09-14 09:00:00

How It Works: `DATE_SUB` computes the rolling cutoff `2026-08-17 12:00:00`. This differs from “current and previous 29 calendar dates,” so clarify the intended window.

Why This Is Asked in Interviews

Tests boundary handling, MySQL date functions, and precise time-window interpretation.

Common Mistake

Using `23:59:59` as an inclusive end or applying `DATE()` to an indexed filter column.

Interview Follow-up

How would you make the query safe for time zones and fractional seconds?

Question 36 — Find tickets from the current month and current year

Concepts: MySQL date/time functions, half-open ranges, intervals, duration

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	created_at
90	2025-12-20 10:00:00
104	2026-08-20 10:00:00
101	2026-09-14 09:00:00
108	2026-09-16 08:00:00

Think About It

- Is this a calendar period or a rolling duration?
- Can a half-open range keep the timestamp predicate index-friendly?

Question

Assuming `CURRENT_DATE` is `2026-09-16`, label whether each ticket belongs to the current month and current year.

Answer

```
SELECT id,
       CASE
         WHEN created_at >= DATE_FORMAT(CURRENT_DATE, '%Y-%m-01')
          AND created_at < DATE_ADD(
                                DATE_FORMAT(CURRENT_DATE, '%Y-%m-01'),
                                INTERVAL 1 MONTH
                              )
         THEN 'YES' ELSE 'NO'
       END AS in_current_month,
       CASE
         WHEN YEAR(created_at) = YEAR(CURRENT_DATE) THEN 'YES'
         ELSE 'NO'
       END AS in_current_year
FROM tickets
ORDER BY id;
```

Expected Output

id	in_current_month	in_current_year
90	NO	NO
101	YES	YES
104	NO	YES
108	YES	YES

How It Works: The month test uses a half-open range from 1 September to 1 October. The year comparison labels every 2026 row. For filtering a large indexed table, year boundaries are preferable to `YEAR(created_at)`.

Why This Is Asked in Interviews

Tests boundary handling, MySQL date functions, and precise time-window interpretation.

Common Mistake

Using 23:59:59 as an inclusive end or applying `DATE()` to an indexed filter column.

Interview Follow-up

How would you make the query safe for time zones and fractional seconds?

Question 37 — Find unresolved tickets older than N days

Concepts: MySQL date/time functions, half-open ranges, intervals, duration

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	status	created_at
101	OPEN	2026-09-10 09:00:00
102	IN_PROGRESS	2026-09-14 10:00:00
103	RESOLVED	2026-09-01 09:00:00

Think About It

- Is this a calendar period or a rolling duration?
- Can a half-open range keep the timestamp predicate index-friendly?

Question

Assuming `CURRENT_DATE` is `2026-09-16`, return unresolved tickets more than five days old and show their calendar-day age.

Answer

```
SELECT id,
       status,
       DATEDIFF(CURRENT_DATE, DATE(created_at)) AS age_days
FROM tickets
WHERE status IN ('OPEN', 'IN_PROGRESS')
      AND created_at < CURRENT_DATE - INTERVAL 5 DAY
ORDER BY age_days DESC;
```

Expected Output

id	status	age_days
101	OPEN	6

How It Works: The sargable timestamp cutoff filters first; `DATEDIFF` then calculates a display age. The strict `<` means a ticket exactly at the five-day cutoff is not included. With the shown 09:00 ticket, the result is unambiguous.

Why This Is Asked in Interviews

Tests boundary handling, MySQL date functions, and precise time-window interpretation.

Common Mistake

Using 23:59:59 as an inclusive end or applying `DATE()` to an indexed filter column.

Interview Follow-up

How would you make the query safe for time zones and fractional seconds?

Part 6 — Aggregate Functions

Aggregates turn many rows into one value. `COUNT(*)` counts rows, `COUNT(column)` counts non-null values, and `COUNT(DISTINCT column)` counts distinct non-null values. `SUM`, `AVG`, `MIN`, and `MAX` ignore `NULL`; an empty input normally returns `NULL` except `COUNT`, which returns zero.

Without `GROUP BY`, the entire filtered result is one group.

Question 38 — Count all tickets

Concepts: COUNT, SUM, AVG, MIN, MAX, NULL behavior

Difficulty: Basic

Relevant Table — tickets

id	status
101	OPEN
102	IN_PROGRESS
103	RESOLVED
104	CLOSED

Think About It

- What exact population is being aggregated?
- Should NULL rows contribute to the metric?

Question

Count the total number of tickets.

Answer

```
SELECT COUNT(*) AS ticket_count
FROM tickets;
```

Expected Output

ticket_count
4

How It Works: `COUNT(*)` counts rows regardless of nullable column values.

Why This Is Asked in Interviews

Tests aggregate semantics and whether you can define the measured population correctly.

Common Mistake

Confusing `COUNT(*)` with `COUNT(nullable_column)` or averaging the wrong row set.

Interview Follow-up

How would the result change if the table were empty or the measured column were NULL?

Question 39 — Compare `COUNT(*)`, `COUNT(column)`, and `COUNT(DISTINCT column)`

Concepts: COUNT, SUM, AVG, MIN, MAX, NULL behavior

Difficulty: Basic

Relevant Table — tickets

id	assignee_id
101	3
102	4
103	3
105	NULL

Think About It

- What exact population is being aggregated?
- Should NULL rows contribute to the metric?

Question

Count tickets, assigned tickets, and different assignees.

Answer

```
SELECT COUNT(*) AS all_tickets,
       COUNT(assignee_id) AS assigned_tickets,
       COUNT(DISTINCT assignee_id) AS distinct_assignees
FROM tickets;
```

Expected Output

all_tickets	assigned_tickets	distinct_assignees
4	3	2

How It Works: The nullable unassigned row contributes only to `COUNT(*)`; duplicate agent 3 contributes once to the distinct count.

Why This Is Asked in Interviews

Tests aggregate semantics and whether you can define the measured population correctly.

Common Mistake

Confusing `COUNT(*)` with `COUNT(nullable_column)` or averaging the wrong row set.

Interview Follow-up

How would the result change if the table were empty or the measured column were NULL?

Question 40 — Find earliest and latest ticket timestamps

Concepts: COUNT, SUM, AVG, MIN, MAX, NULL behavior

Difficulty: Basic

Relevant Table — tickets

id	created_at
101	2026-09-14 09:00:00
104	2026-08-20 10:00:00
108	2026-09-16 08:00:00

Think About It

- What exact population is being aggregated?
- Should NULL rows contribute to the metric?

Question

Find the earliest and latest creation timestamps.

Answer

```
SELECT MIN(created_at) AS first_created_at,  
       MAX(created_at) AS latest_created_at  
FROM tickets;
```

Expected Output

first_created_at	latest_created_at
2026-08-20 10:00:00	2026-09-16 08:00:00

How It Works: **MIN** and **MAX** compare the values in the single aggregate group.

Why This Is Asked in Interviews

Tests aggregate semantics and whether you can define the measured population correctly.

Common Mistake

Confusing COUNT(*) with COUNT(nullable_column) or averaging the wrong row set.

Interview Follow-up

How would the result change if the table were empty or the measured column were NULL?

Question 41 — Calculate average resolution time

Concepts: COUNT, SUM, AVG, MIN, MAX, NULL behavior

Difficulty: Basic

Relevant Table — *tickets*

id	created_at	resolved_at
103	2026-09-01 09:00:00	2026-09-02 12:00:00
104	2026-08-20 10:00:00	2026-08-20 14:00:00
106	2026-09-10 08:00:00	2026-09-10 16:00:00
101	2026-09-14 09:00:00	NULL

Think About It

- What exact population is being aggregated?
- Should NULL rows contribute to the metric?

Question

Calculate the average whole-hour resolution duration for completed tickets.

Answer

```
SELECT ROUND(  
    AVG(TIMESTAMPDIFF(HOUR, created_at, resolved_at)),  
    2  
) AS avg_resolution_hours  
FROM tickets  
WHERE resolved_at IS NOT NULL;
```

Expected Output

avg_resolution_hours
13.00

How It Works: The three durations are 27, 4, and 8 hours; their average is 13. Filtering makes the intended population explicit.

Why This Is Asked in Interviews

Tests aggregate semantics and whether you can define the measured population correctly.

Common Mistake

Confusing COUNT(*) with COUNT(nullable_column) or averaging the wrong row set.

Interview Follow-up

How would the result change if the table were empty or the measured column were NULL?

Question 42 — Sum active-work estimates with CASE

Concepts: COUNT, SUM, AVG, MIN, MAX, NULL behavior

Difficulty: Basic

Relevant Table — tickets

id	status	priority
101	OPEN	HIGH
102	IN_PROGRESS	MEDIUM
103	RESOLVED	LOW
105	OPEN	HIGH

Think About It

- What exact population is being aggregated?
- Should NULL rows contribute to the metric?

Question

Estimate active workload points: HIGH = 3, MEDIUM = 2, LOW = 1; completed tickets add zero.

Answer

```
SELECT SUM(
    CASE
        WHEN status NOT IN ('OPEN', 'IN_PROGRESS') THEN 0
        WHEN priority = 'HIGH' THEN 3
        WHEN priority = 'MEDIUM' THEN 2
        ELSE 1
    END
) AS active_workload_points
FROM tickets;
```

Expected Output

active_workload_points
8

How It Works: CASE converts each ticket to a numeric contribution, and SUM adds those contributions: 3 + 2 + 0 + 3.

Why This Is Asked in Interviews

Tests aggregate semantics and whether you can define the measured population correctly.

Common Mistake

Confusing COUNT(*) with COUNT(nullable_column) or averaging the wrong row set.

Interview Follow-up

How would the result change if the table were empty or the measured column were NULL?

Part 7 — GROUP BY

GROUP BY partitions the filtered rows into groups that share the same grouping key. An aggregate then summarizes each group, producing one result row per group.

Suppose statuses are OPEN, OPEN, CLOSED. GROUP BY status forms an OPEN group containing two rows and a CLOSED group containing one. COUNT(*) is then evaluated once for each group.

Important rules:

- In a grouped query, each selected expression should be an aggregate, a grouping expression, or functionally dependent on the grouping columns.
- MySQL's default ONLY_FULL_GROUP_BY mode rejects ambiguous queries such as SELECT status, title, COUNT(*) FROM tickets GROUP BY status; a status can contain many titles, so MySQL cannot choose one honestly.
- All NULL grouping keys form one group.

- **WHERE** filters rows before grouping. **HAVING** filters groups after aggregates are calculated.
- **DISTINCT category** and **GROUP BY category** can both produce unique categories, but **GROUP BY** is intended for per-group calculations. Use **DISTINCT** when deduplication is the only goal.
- Group by the full time grain. Grouping only by **MONTH(created_at)** combines September from every year.
- Joining one-to-many tables before aggregation can multiply rows. Count the correct key, sometimes with **DISTINCT**, and inspect the joined rowset first.

Real applications use grouping for dashboards, workload balancing, SLA reports, inventory summaries, unread counts, and monthly trends.

Question chain: Questions 43–53 begin with one status bucket, change the grouping key, add **SUM/AVG/MIN/MAX**, move to multiple grouping columns and calendar grains, handle **NULL**, and finish with strict-grouping correctness. Each question changes one major idea while staying inside PulseDesk tickets.

Question 43 — Count tickets by status

Concepts: GROUP BY, aggregates, output grain, ONLY_FULL_GROUP_BY

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	status
101	OPEN
102	IN_PROGRESS
103	RESOLVED
104	CLOSED
105	OPEN
106	RESOLVED
107	CLOSED
108	OPEN

Think About It

- What does one output row represent?
- Which rows belong inside each group?

Question

Count tickets in each status.

Answer

```
SELECT status,
       COUNT(*) AS ticket_count
FROM tickets
GROUP BY status
ORDER BY status;
```

Expected Output

status	ticket_count
CLOSED	2
IN_PROGRESS	1
OPEN	3
RESOLVED	2

How It Works: Rows with the same status form a group, and **COUNT(*)** counts rows inside each group.

Visual data flow

```
Original ticket rows
101 OPEN
102 IN_PROGRESS
103 RESOLVED
104 CLOSED
105 OPEN
106 RESOLVED
107 CLOSED
108 OPEN
```

```
GROUP BY status forms four buckets
OPEN
├ 101
├ 105
└ 108          COUNT(*) = 3

IN_PROGRESS
└ 102          COUNT(*) = 1

RESOLVED
├ 103
└ 106          COUNT(*) = 2

CLOSED
├ 104
└ 107          COUNT(*) = 2

ORDER BY status sorts the four summary rows; it does not sort rows inside a group.
```

Why This Is Asked in Interviews

Tests whether you understand how rows become groups rather than merely remembering syntax.

Common Mistake

Selecting a non-grouped, nonaggregated column or grouping at the wrong grain.

Interview Follow-up

Now filter or rank the groups by the aggregate value.

Question 44 — Count tickets by priority

Concepts: GROUP BY, aggregates, output grain, ONLY_FULL_GROUP_BY

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	priority
101	HIGH
102	MEDIUM
103	LOW
104	HIGH
105	HIGH
106	MEDIUM
107	LOW
108	HIGH

Think About It

- What does one output row represent?
- Which rows belong inside each group?

Question

Return one count for each priority.

Answer

```
SELECT priority,
       COUNT(*) AS ticket_count
FROM tickets
GROUP BY priority
ORDER BY priority;
```

Expected Output

priority	ticket_count
HIGH	4

priority	ticket_count
LOW	2
MEDIUM	2

How It Works: The pattern is identical to status grouping; changing the grouping key changes the buckets.

Why This Is Asked in Interviews

Tests whether you understand how rows become groups rather than merely remembering syntax.

Common Mistake

Selecting a non-grouped, nonaggregated column or grouping at the wrong grain.

Interview Follow-up

Now filter or rank the groups by the aggregate value.

Question 45 — Sum weighted workload by priority

Concepts: GROUP BY, aggregates, output grain, ONLY_FULL_GROUP_BY

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	status	priority
101	OPEN	HIGH
102	IN_PROGRESS	MEDIUM
103	RESOLVED	LOW
105	OPEN	HIGH
108	OPEN	HIGH

Think About It

- What does one output row represent?
- Which rows belong inside each group?

Question

For active tickets only, show the ticket count and workload points per priority (**HIGH=3**, **MEDIUM=2**, **LOW=1**).

Answer

```
SELECT priority,
       COUNT(*) AS active_ticket_count,
       SUM(CASE priority
            WHEN 'HIGH' THEN 3
            WHEN 'MEDIUM' THEN 2
            ELSE 1
           END) AS workload_points
FROM tickets
WHERE status IN ('OPEN', 'IN_PROGRESS')
GROUP BY priority
ORDER BY workload_points DESC;
```

Expected Output

priority	active_ticket_count	workload_points
HIGH	3	9
MEDIUM	1	2

How It Works: **WHERE** first removes the resolved row. **SUM** then totals a numeric expression separately inside each remaining priority group.

Why This Is Asked in Interviews

Tests whether you understand how rows become groups rather than merely remembering syntax.

Common Mistake

Selecting a non-grouped, nonaggregated column or grouping at the wrong grain.

Interview Follow-up

Now filter or rank the groups by the aggregate value.

Question 46 — Calculate average resolution time by category

Concepts: GROUP BY, aggregates, output grain, ONLY_FULL_GROUP_BY

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	category	created_at	resolved_at
103	SOFTWARE	2026-09-01 09:00	2026-09-02 12:00
106	SOFTWARE	2026-09-10 08:00	2026-09-10 16:00
104	ACCESS	2026-08-20 10:00	2026-08-20 14:00
107	NETWORK	2026-08-31 09:00	2026-09-01 09:00
101	NETWORK	2026-09-14 09:00	NULL

Think About It

- What does one output row represent?
- Which rows belong inside each group?

Question

For completed tickets, calculate average resolution hours per category.

Answer

```
SELECT category,
       ROUND(AVG(TIMESTAMPDIFF(HOUR, created_at, resolved_at)), 2)
         AS avg_resolution_hours
FROM tickets
WHERE resolved_at IS NOT NULL
GROUP BY category
ORDER BY category;
```

Expected Output

category	avg_resolution_hours
ACCESS	4.00
NETWORK	24.00
SOFTWARE	17.50

How It Works: The unresolved network ticket is filtered out. Each category's remaining durations are averaged independently.

Why This Is Asked in Interviews

Tests whether you understand how rows become groups rather than merely remembering syntax.

Common Mistake

Selecting a non-grouped, nonaggregated column or grouping at the wrong grain.

Interview Follow-up

Now filter or rank the groups by the aggregate value.

Question 47 — Find first and latest ticket date per requester

Concepts: GROUP BY, aggregates, output grain, ONLY_FULL_GROUP_BY

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	requester_id	created_at
101	2	2026-09-14 09:00
103	2	2026-09-01 09:00

id	requester_id	created_at
104	2	2026-08-20 10:00
102	5	2026-09-14 10:00
105	5	2026-09-15 11:00

Think About It

- What does one output row represent?
- Which rows belong inside each group?

Question

Show each requester's first and latest ticket timestamp.

Answer

```
SELECT requester_id,
       MIN(created_at) AS first_ticket_at,
       MAX(created_at) AS latest_ticket_at
FROM tickets
GROUP BY requester_id
ORDER BY requester_id;
```

Expected Output

requester_id	first_ticket_at	latest_ticket_at
2	2026-08-20 10:00	2026-09-14 09:00
5	2026-09-14 10:00	2026-09-15 11:00

How It Works: `MIN` and `MAX` are calculated within each requester group. A later join will replace IDs with user names.

Why This Is Asked in Interviews

Tests whether you understand how rows become groups rather than merely remembering syntax.

Common Mistake

Selecting a non-grouped, nonaggregated column or grouping at the wrong grain.

Interview Follow-up

Now filter or rank the groups by the aggregate value.

Question 48 — Count tickets per requester ID

Concepts: GROUP BY, aggregates, output grain, ONLY_FULL_GROUP_BY

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	requester_id
101	2
103	2
104	2
102	5
105	5

Think About It

- What does one output row represent?
- Which rows belong inside each group?

Question

Count how many tickets each requester raised.

Answer

```
SELECT requester_id,  
       COUNT(*) AS ticket_count  
FROM tickets  
GROUP BY requester_id  
ORDER BY ticket_count DESC, requester_id;
```

Expected Output

requester_id	ticket_count
2	3
5	2

How It Works: **ORDER BY** can use the aggregate alias after grouping has produced the counts.

Why This Is Asked in Interviews

Tests whether you understand how rows become groups rather than merely remembering syntax.

Common Mistake

Selecting a non-grouped, nonaggregated column or grouping at the wrong grain.

Interview Follow-up

Now filter or rank the groups by the aggregate value.

Question 49 — Count tickets by category and status

Concepts: GROUP BY, aggregates, output grain, ONLY_FULL_GROUP_BY

Difficulty: Basic / Intermediate

Relevant Table — **tickets**

id	category	status
101	NETWORK	OPEN
107	NETWORK	CLOSED
102	HARDWARE	IN_PROGRESS
105	HARDWARE	OPEN
103	SOFTWARE	RESOLVED
106	SOFTWARE	RESOLVED

Think About It

- What does one output row represent?
- Which rows belong inside each group?

Question

Produce one row for every category/status combination present.

Answer

```
SELECT category,  
       status,  
       COUNT(*) AS ticket_count  
FROM tickets  
GROUP BY category, status  
ORDER BY category, status;
```

Expected Output

category	status	ticket_count
HARDWARE	IN_PROGRESS	1
HARDWARE	OPEN	1
NETWORK	CLOSED	1
NETWORK	OPEN	1

category	status	ticket_count
SOFTWARE	RESOLVED	2

How It Works: The grouping key is the pair (*category*, *status*). Only combinations that exist appear; generating zero-count combinations requires a cross join to a dimension set.

Why This Is Asked in Interviews

Tests whether you understand how rows become groups rather than merely remembering syntax.

Common Mistake

Selecting a non-grouped, nonaggregated column or grouping at the wrong grain.

Interview Follow-up

Now filter or rank the groups by the aggregate value.

Question 50 — Count tickets by calendar month

Concepts: GROUP BY, aggregates, output grain, ONLY_FULL_GROUP_BY

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	created_at
104	2026-08-20 10:00
107	2026-08-31 09:00
103	2026-09-01 09:00
106	2026-09-10 08:00
101	2026-09-14 09:00

Think About It

- What does one output row represent?
- Which rows belong inside each group?

Question

Count tickets for each year and month.

Answer

```
SELECT YEAR(created_at) AS report_year,
       MONTH(created_at) AS report_month,
       COUNT(*) AS ticket_count
FROM tickets
GROUP BY YEAR(created_at), MONTH(created_at)
ORDER BY report_year, report_month;
```

Expected Output

report_year	report_month	ticket_count
2026	8	2
2026	9	3

How It Works: Both year and month define the time bucket, so different years never collapse together.

Why This Is Asked in Interviews

Tests whether you understand how rows become groups rather than merely remembering syntax.

Common Mistake

Selecting a non-grouped, nonaggregated column or grouping at the wrong grain.

Interview Follow-up

Now filter or rank the groups by the aggregate value.

Question 51 — Group completed tickets by resolution day

Concepts: GROUP BY, aggregates, output grain, ONLY_FULL_GROUP_BY

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	resolved_at
103	2026-09-02 12:00
104	2026-08-20 14:00
106	2026-09-10 16:00
109	2026-09-10 17:30
101	NULL

Think About It

- What does one output row represent?
- Which rows belong inside each group?

Question

Count completions per calendar date.

Answer

```
SELECT DATE(resolved_at) AS resolution_date,
       COUNT(*) AS resolved_count
FROM tickets
WHERE resolved_at IS NOT NULL
GROUP BY DATE(resolved_at)
ORDER BY resolution_date;
```

Expected Output

resolution_date	resolved_count
2026-08-20	1
2026-09-02	1
2026-09-10	2

How It Works: DATE maps timestamps on the same day to one grouping value. The function is acceptable for reporting; date filtering should still use ranges.

Why This Is Asked in Interviews

Tests whether you understand how rows become groups rather than merely remembering syntax.

Common Mistake

Selecting a non-grouped, nonaggregated column or grouping at the wrong grain.

Interview Follow-up

Now filter or rank the groups by the aggregate value.

Question 52 — See how NULL forms its own group

Concepts: GROUP BY, aggregates, output grain, ONLY_FULL_GROUP_BY

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	assignee_id
101	3
103	3
102	4
105	NULL
109	NULL

Think About It

- What does one output row represent?
- Which rows belong inside each group?

Question

Count tickets by assignee, displaying unassigned rows as **UNASSIGNED**.

Answer

```
SELECT COALESCE(CAST(assignee_id AS CHAR), 'UNASSIGNED') AS assignee_bucket,
        COUNT(*) AS ticket_count
FROM tickets
GROUP BY assignee_id
ORDER BY assignee_id;
```

Expected Output

assignee_bucket	ticket_count
UNASSIGNED	2
3	2
4	1

How It Works: SQL groups all **NULL** assignee values together. **COALESCE** changes only the displayed label. The exact **NULL** sort position depends on direction; use an explicit sort expression if presentation order matters.

Why This Is Asked in Interviews

Tests whether you understand how rows become groups rather than merely remembering syntax.

Common Mistake

Selecting a non-grouped, nonaggregated column or grouping at the wrong grain.

Interview Follow-up

Now filter or rank the groups by the aggregate value.

Question 53 — Correct an **ONLY_FULL_GROUP_BY** error

Concepts: GROUP BY, aggregates, output grain, ONLY_FULL_GROUP_BY

Difficulty: Basic / Intermediate

Relevant Table — **tickets**

id	category	title
101	NETWORK	VPN timeout
107	NETWORK	Guest Wi-Fi
103	SOFTWARE	IDE licence

Think About It

- What does one output row represent?
- Which rows belong inside each group?

Question

Return each category and its ticket count. Explain why selecting **title** would be invalid.

Answer

```
SELECT category,
        COUNT(*) AS ticket_count
FROM tickets
GROUP BY category;
```

Expected Output

category	ticket_count
NETWORK	2

category	ticket_count
SOFTWARE	1

How It Works: `category` identifies each group and `COUNT(*)` summarizes it. `title` cannot be selected as a bare column because the `NETWORK` group has two possible titles. Add `title` to the grouping only if the desired grain is category-plus-title, or deliberately aggregate it with a function suited to the business question.

Why This Is Asked in Interviews

Tests whether you understand how rows become groups rather than merely remembering syntax.

Common Mistake

Selecting a non-grouped, nonaggregated column or grouping at the wrong grain.

Interview Follow-up

Now filter or rank the groups by the aggregate value.

Part 8 — HAVING

HAVING filters completed groups. Use it for conditions involving aggregates, such as “more than five tickets.” Use **WHERE** for ordinary row conditions so MySQL discards unwanted rows before it groups them.

Question chain: Questions 54–64 take the groups built in Part 7, filter them by one and then several aggregate conditions, combine row and group filters, rank aggregate results, select top groups, and finish with multi-column group reports.

Conceptually:

```
FROM tickets
WHERE status = 'OPEN'           -- choose source rows
GROUP BY category              -- build category groups
HAVING COUNT(*) >= 2           -- choose finished groups
SELECT category, COUNT(*)
```

Question 54 — Find statuses containing at least two tickets

Concepts: GROUP BY, HAVING, WHERE, aggregate filtering

Difficulty: Intermediate

Relevant Table — tickets

id	status
101	OPEN
105	OPEN
108	OPEN
103	RESOLVED
106	RESOLVED
102	IN_PROGRESS

Think About It

- Which condition filters source rows?
- Which condition can be evaluated only after groups are counted?

Question

Return statuses having at least two tickets.

Answer

```
SELECT status,
       COUNT(*) AS ticket_count
FROM tickets
GROUP BY status
HAVING COUNT(*) >= 2
ORDER BY status;
```

Expected Output

status	ticket_count
OPEN	3
RESOLVED	2

How It Works: **HAVING** sees the count calculated for each status group and removes the one-row **IN_PROGRESS** group.

Visual data flow

```

STEP 1 – FROM supplies six ticket rows
OPEN, OPEN, OPEN, RESOLVED, RESOLVED, IN_PROGRESS

STEP 2 – GROUP BY status
OPEN      -> [101, 105, 108]
RESOLVED  -> [103, 106]
IN_PROGRESS -> [102]

STEP 3 – COUNT(*)
OPEN      -> 3
RESOLVED  -> 2
IN_PROGRESS -> 1

STEP 4 – HAVING COUNT(*) >= 2
keep OPEN and RESOLVED
remove IN_PROGRESS

STEP 5 – ORDER BY status sorts the two surviving groups

```

WHERE **COUNT(*) >= 2** would be invalid: **WHERE** runs before groups and their counts exist.

Why This Is Asked in Interviews

Tests the crucial distinction between row filtering and aggregate-group filtering.

Common Mistake

Putting an aggregate in **WHERE** or delaying a row-level condition until **HAVING**.

Interview Follow-up

Add a row-level status filter before applying the group threshold.

Question 55 — Find frequent requesters

Concepts: GROUP BY, HAVING, WHERE, aggregate filtering

Difficulty: Intermediate

Relevant Table — **tickets**

id	requester_id
101	2
103	2
104	2
108	2
102	5
105	5

Think About It

- Which condition filters source rows?
- Which condition can be evaluated only after groups are counted?

Question

Return requester IDs that raised more than two tickets.

Answer

```

SELECT requester_id,
       COUNT(*) AS ticket_count

```

```
FROM tickets
GROUP BY requester_id
HAVING COUNT(*) > 2;
```

Expected Output

requester_id	ticket_count
2	4

How It Works: The threshold applies to each requester's aggregate, so it belongs in **HAVING**.

Why This Is Asked in Interviews

Tests the crucial distinction between row filtering and aggregate-group filtering.

Common Mistake

Putting an aggregate in WHERE or delaying a row-level condition until HAVING.

Interview Follow-up

Add a row-level status filter before applying the group threshold.

Question 56 — Find categories with slow average resolution

Concepts: GROUP BY, HAVING, WHERE, aggregate filtering

Difficulty: Intermediate

Relevant Table — tickets

id	category	created_at	resolved_at
103	SOFTWARE	2026-09-01 09:00	2026-09-02 12:00
106	SOFTWARE	2026-09-10 08:00	2026-09-10 16:00
104	ACCESS	2026-08-20 10:00	2026-08-20 14:00
107	NETWORK	2026-08-31 09:00	2026-09-01 09:00

Think About It

- Which condition filters source rows?
- Which condition can be evaluated only after groups are counted?

Question

Return categories whose average resolution time exceeds 10 hours.

Answer

```
SELECT category,
       ROUND(AVG(TIMESTAMPDIFF(HOUR, created_at, resolved_at)), 2)
         AS avg_resolution_hours
FROM tickets
WHERE resolved_at IS NOT NULL
GROUP BY category
HAVING AVG(TIMESTAMPDIFF(HOUR, created_at, resolved_at)) > 10
ORDER BY avg_resolution_hours DESC;
```

Expected Output

category	avg_resolution_hours
NETWORK	24.00
SOFTWARE	17.50

How It Works: **WHERE** defines the completed-ticket population; **HAVING** applies the SLA threshold to category averages.

Why This Is Asked in Interviews

Tests the crucial distinction between row filtering and aggregate-group filtering.

Common Mistake

Putting an aggregate in WHERE or delaying a row-level condition until HAVING.

Interview Follow-up

Add a row-level status filter before applying the group threshold.

Question 57 — Find months with multiple tickets

Concepts: GROUP BY, HAVING, WHERE, aggregate filtering

Difficulty: Intermediate

Relevant Table — tickets

id	created_at
104	2026-08-20 10:00
107	2026-08-31 09:00
103	2026-09-01 09:00
106	2026-09-10 08:00
101	2026-09-14 09:00

Think About It

- Which condition filters source rows?
- Which condition can be evaluated only after groups are counted?

Question

Return calendar months having at least three tickets.

Answer

```
SELECT DATE_FORMAT(created_at, '%Y-%m') AS ticket_month,
       COUNT(*) AS ticket_count
FROM tickets
GROUP BY YEAR(created_at), MONTH(created_at),
         DATE_FORMAT(created_at, '%Y-%m')
HAVING COUNT(*) >= 3;
```

Expected Output

ticket_month	ticket_count
2026-09	3

How It Works: The group includes a year-aware month key. The label is included in GROUP BY to remain explicit under strict grouping rules.

Why This Is Asked in Interviews

Tests the crucial distinction between row filtering and aggregate-group filtering.

Common Mistake

Putting an aggregate in WHERE or delaying a row-level condition until HAVING.

Interview Follow-up

Add a row-level status filter before applying the group threshold.

Question 58 — Combine row filtering and group filtering

Concepts: GROUP BY, HAVING, WHERE, aggregate filtering

Difficulty: Intermediate

Relevant Table — tickets

id	category	status
101	NETWORK	OPEN
107	NETWORK	CLOSED
102	HARDWARE	IN_PROGRESS
105	HARDWARE	OPEN
109	HARDWARE	OPEN

id	category	status
103	SOFTWARE	RESOLVED

Think About It

- Which condition filters source rows?
- Which condition can be evaluated only after groups are counted?

Question

Among active tickets only, return categories having at least two active tickets.

Answer

```
SELECT category,
       COUNT(*) AS active_ticket_count
FROM tickets
WHERE status IN ('OPEN', 'IN_PROGRESS')
GROUP BY category
HAVING COUNT(*) >= 2
ORDER BY category;
```

Expected Output

category	active_ticket_count
HARDWARE	3

How It Works: **WHERE** removes closed/resolved rows first. The remaining rows are grouped, and **HAVING** keeps only sufficiently large groups.

Why This Is Asked in Interviews

Tests the crucial distinction between row filtering and aggregate-group filtering.

Common Mistake

Putting an aggregate in **WHERE** or delaying a row-level condition until **HAVING**.

Interview Follow-up

Add a row-level status filter before applying the group threshold.

Question 59 — Filter groups by two aggregate conditions

Concepts: GROUP BY, HAVING, WHERE, aggregate filtering

Difficulty: Intermediate

Relevant Table — tickets

id	assignee_id	status	priority
101	3	OPEN	HIGH
103	3	RESOLVED	LOW
107	3	CLOSED	LOW
102	4	IN_PROGRESS	MEDIUM
104	4	CLOSED	HIGH
106	4	RESOLVED	MEDIUM
108	6	OPEN	HIGH

Think About It

- Which condition filters source rows?
- Which condition can be evaluated only after groups are counted?

Question

Find assignees with at least three total tickets and at least one high-priority ticket.

Answer

```
SELECT assignee_id,
       COUNT(*) AS total_tickets,
```

```
SUM(priority = 'HIGH') AS high_priority_tickets
FROM tickets
WHERE assignee_id IS NOT NULL
GROUP BY assignee_id
HAVING COUNT(*) >= 3
      AND SUM(priority = 'HIGH') >= 1
ORDER BY assignee_id;
```

Expected Output

assignee_id	total_tickets	high_priority_tickets
3	3	1
4	3	1

How It Works: In MySQL a true comparison acts as 1 and false as 0, so `SUM(priority = 'HIGH')` counts matching rows. `SUM(CASE WHEN ... THEN 1 ELSE 0 END)` is more portable SQL.

Why This Is Asked in Interviews

Tests the crucial distinction between row filtering and aggregate-group filtering.

Common Mistake

Putting an aggregate in WHERE or delaying a row-level condition until HAVING.

Interview Follow-up

Add a row-level status filter before applying the group threshold.

Part 9 — GROUP BY + HAVING + COUNT + ORDER BY

Interview questions often combine the full reporting pipeline. State the desired grain first ("one row per category"), filter source rows, group at that grain, calculate metrics, filter groups, and finally sort or limit the report.

Question 60 — Rank statuses by ticket count

Concepts: GROUP BY, COUNT, HAVING, ORDER BY, LIMIT

Difficulty: Intermediate

Relevant Table — tickets

id	status
101	OPEN
105	OPEN
108	OPEN
103	RESOLVED
106	RESOLVED
104	CLOSED

Think About It

- What is the report grain?
- In what order do filtering, grouping, group filtering, sorting, and limiting occur?

Question

Show statuses with at least two tickets, highest count first.

Answer

```
SELECT status,
      COUNT(*) AS ticket_count
FROM tickets
GROUP BY status
HAVING COUNT(*) >= 2
ORDER BY ticket_count DESC, status ASC;
```

Expected Output

status	ticket_count
OPEN	3
RESOLVED	2

How It Works: **HAVING** removes small groups, and **ORDER BY** ranks the surviving aggregates. The status tiebreaker stabilizes equal counts.

Why This Is Asked in Interviews

Tests whether you can assemble the complete reporting pipeline in the correct logical order.

Common Mistake

Applying LIMIT before ranking conceptually or omitting a stable tie-breaker.

Interview Follow-up

Return the top N groups while defining exactly how ties should behave.

Question 61 — Return the top two categories by ticket volume

Concepts: GROUP BY, COUNT, HAVING, ORDER BY, LIMIT

Difficulty: Intermediate

Relevant Table — tickets

id	category
101	NETWORK
107	NETWORK
102	HARDWARE
105	HARDWARE
109	HARDWARE
103	SOFTWARE

Think About It

- What is the report grain?
- In what order do filtering, grouping, group filtering, sorting, and limiting occur?

Question

Find the two categories with the most tickets.

Answer

```
SELECT category,
COUNT(*) AS ticket_count
FROM tickets
GROUP BY category
ORDER BY ticket_count DESC, category ASC
LIMIT 2;
```

Expected Output

category	ticket_count
HARDWARE	3
NETWORK	2

How It Works: Grouping calculates volume, sorting ranks it, and **LIMIT** retains the first two. Decide how ties should be handled; **DENSE_RANK** can include every tied group.

Why This Is Asked in Interviews

Tests whether you can assemble the complete reporting pipeline in the correct logical order.

Common Mistake

Applying LIMIT before ranking conceptually or omitting a stable tie-breaker.

Interview Follow-up

Return the top N groups while defining exactly how ties should behave.

Question 62 — Rank categories with multiple open tickets

Concepts: GROUP BY, COUNT, HAVING, ORDER BY, LIMIT

Difficulty: Intermediate

Relevant Table — tickets

id	category	status
101	NETWORK	OPEN
110	NETWORK	OPEN
107	NETWORK	CLOSED
105	HARDWARE	OPEN
109	HARDWARE	OPEN
111	HARDWARE	OPEN
103	SOFTWARE	RESOLVED

Think About It

- What is the report grain?
- In what order do filtering, grouping, group filtering, sorting, and limiting occur?

Question

Return categories having more than one open ticket, sorted by open count.

Answer

```
SELECT category,
       COUNT(*) AS open_ticket_count
FROM tickets
WHERE status = 'OPEN'
GROUP BY category
HAVING COUNT(*) > 1
ORDER BY open_ticket_count DESC, category;
```

Expected Output

category	open_ticket_count
HARDWARE	3
NETWORK	2

How It Works: This is the canonical sequence: row filter, grouping, group filter, aggregate ordering.

Row filter versus group filter

WHERE status = 'OPEN'

removes CLOSED and RESOLVED ticket rows first

↓

GROUP BY category

builds groups only from the remaining OPEN rows

↓

HAVING COUNT(*) > 1

removes category groups whose finished count is 0 or 1

Putting `status = 'OPEN'` in `HAVING` would blur row-level intent. Putting `COUNT(*) > 1` in `WHERE` would be invalid because the count does not exist yet.

Full logical execution walkthrough

1. `FROM tickets` supplies all seven shown rows.
2. `WHERE status = 'OPEN'` keeps five rows: network 101/110 and hardware 105/109/111.
3. `GROUP BY category` forms a network group of two and a hardware group of three.
4. `HAVING COUNT(*) > 1` keeps both groups. A one-row group would disappear here.
5. `SELECT category, COUNT(*)` shapes each surviving group into one output row.
6. `ORDER BY open_ticket_count DESC, category` puts hardware before network.

There is no `DISTINCT` or `LIMIT` in this query. If present, they would act after projection and ordering according to the document's logical-order model.

Why This Is Asked in Interviews

Tests whether you can assemble the complete reporting pipeline in the correct logical order.

Common Mistake

Applying LIMIT before ranking conceptually or omitting a stable tie-breaker.

Interview Follow-up

Return the top N groups while defining exactly how ties should behave.

Question 63 — Report month-and-status counts

Concepts: GROUP BY, COUNT, HAVING, ORDER BY, LIMIT

Difficulty: Intermediate

Relevant Table — tickets

id	status	created_at
104	CLOSED	2026-08-20 10:00
107	CLOSED	2026-08-31 09:00
103	RESOLVED	2026-09-01 09:00
101	OPEN	2026-09-14 09:00
105	OPEN	2026-09-15 11:00

Think About It

- What is the report grain?
- In what order do filtering, grouping, group filtering, sorting, and limiting occur?

Question

Count each status per month and sort chronologically, then by largest status count.

Answer

```
SELECT YEAR(created_at) AS report_year,
       MONTH(created_at) AS report_month,
       status,
       COUNT(*) AS ticket_count
FROM tickets
GROUP BY YEAR(created_at), MONTH(created_at), status
ORDER BY report_year, report_month, ticket_count DESC, status;
```

Expected Output

report_year	report_month	status	ticket_count
2026	8	CLOSED	2
2026	9	OPEN	2
2026	9	RESOLVED	1

How It Works: Three values define each group. Sorting by numeric year and month preserves time order.

Why This Is Asked in Interviews

Tests whether you can assemble the complete reporting pipeline in the correct logical order.

Common Mistake

Applying LIMIT before ranking conceptually or omitting a stable tie-breaker.

Interview Follow-up

Return the top N groups while defining exactly how ties should behave.

Question 64 — Find significant category-priority combinations

Concepts: GROUP BY, COUNT, HAVING, ORDER BY, LIMIT

Difficulty: Intermediate

Relevant Table — tickets

id	category	priority
101	NETWORK	HIGH
107	NETWORK	LOW
110	NETWORK	HIGH
102	HARDWARE	MEDIUM
105	HARDWARE	HIGH
109	HARDWARE	HIGH
103	SOFTWARE	LOW

Think About It

- What is the report grain?
- In what order do filtering, grouping, group filtering, sorting, and limiting occur?

Question

Return category/priority pairs occurring at least twice, largest group first.

Answer

```
SELECT category,
       priority,
       COUNT(*) AS ticket_count
FROM tickets
GROUP BY category, priority
HAVING COUNT(*) >= 2
ORDER BY ticket_count DESC, category, priority;
```

Expected Output

category	priority	ticket_count
HARDWARE	HIGH	2
NETWORK	HIGH	2

How It Works: The pair determines the reporting grain; **HAVING** and **ORDER BY** act on the count at that grain.

Why This Is Asked in Interviews

Tests whether you can assemble the complete reporting pipeline in the correct logical order.

Common Mistake

Applying **LIMIT** before ranking conceptually or omitting a stable tie-breaker.

Interview Follow-up

Return the top N groups while defining exactly how ties should behave.

Part 10 — JOINS

A join combines related rows. In PulseDesk, foreign keys tell you the intended paths:

- `tickets.requester_id = users.user_id`;
- `tickets.assignee_id = users.user_id`;
- `comments.ticket_id = tickets.id`;
- `ticket_assets` bridges `tickets` and `assets`.

An **INNER JOIN** keeps matches. A **LEFT JOIN** keeps every left row and supplies **NULL** for missing right data. A **RIGHT JOIN** is the mirrored form and can usually be rewritten as a clearer left join by swapping table order. A self join uses two aliases of the same table. A cross join produces every combination and therefore multiplies row counts.

Always ask about cardinality. Joining a ticket to three comments creates three joined ticket rows. That is correct row-level data, but it can inflate later counts. Also keep optional-side filters in the **ON** clause when unmatched left rows must survive; placing them in **WHERE** can turn a left join into an inner join.

Question chain: Questions 65–73 start with the mandatory requester join, preserve a nullable assignee, reuse `users` with two aliases, traverse comments and the asset bridge, and then contrast right, self, and cross joins. Questions 74–82 add grouping, zero-count parents, workload balancing, **HAVING**, and protection from join multiplication.

Question 65 — Join tickets to their requesters

Concepts: JOINS, foreign keys, cardinality, unmatched-row preservation

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	title	requester_id
101	VPN timeout	2
102	Laptop battery	5

Relevant Table — users

user_id	name
2	Ravi Shah
5	Neha Joshi
6	Ishan Bose

Relationship

users.user_id (1) —< (N) tickets.requester_id

Think About It

- Which keys match the rows?
- Which table's unmatched rows must remain in the result?

Question

Show each ticket with its requester's name.

Answer

```
SELECT t.id,  
       t.title,  
       u.name AS requester_name  
FROM tickets AS t  
INNER JOIN users AS u  
      ON u.user_id = t.requester_id  
ORDER BY t.id;
```

Expected Output

id	title	requester_name
101	VPN timeout	Ravi Shah
102	Laptop battery	Neha Joshi

How It Works: The foreign-key equality matches each ticket to exactly one requester. User 6 has no matching ticket and is absent from an inner join.

```
Joined matches  
Ravi (user 2) <-> ticket 101  
Neha (user 5) <-> ticket 102  
Ishan (user 6) -> no ticket, so INNER JOIN excludes him
```

Why This Is Asked in Interviews

Tests relationship reading, join selection, aliases, and row multiplication.

Common Mistake

Choosing an inner join when unmatched rows matter, or overlooking one-to-many duplication.

Interview Follow-up

Change the join type and predict which unmatched rows appear or disappear.

Question 66 — Preserve unassigned tickets with LEFT JOIN

Concepts: JOINS, foreign keys, cardinality, unmatched-row preservation

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	title	assignee_id
101	VPN timeout	3
105	Monitor flicker	NULL

Relevant Table — users

user_id	name
3	Meera Nair
4	Kabir Rao

Relationship



Think About It

- Which keys match the rows?
- Which table's unmatched rows must remain in the result?

Question

Show all tickets and the assignee name, including unassigned tickets.

Answer

```
SELECT t.id,
       t.title,
       u.name AS assignee_name
FROM tickets AS t
LEFT JOIN users AS u
      ON u.user_id = t.assignee_id
ORDER BY t.id;
```

Expected Output

id	title	assignee_name
101	VPN timeout	Meera Nair
105	Monitor flicker	NULL

How It Works: Every left-side ticket survives. The nullable foreign key cannot match for ticket 105, so right-side columns become **NULL**.

LEFT JOIN starts from tickets
ticket 101 -> assignee 3 -> Meera
ticket 105 -> assignee NULL -> no user match -> keep ticket, show NULL name

Why This Is Asked in Interviews

Tests relationship reading, join selection, aliases, and row multiplication.

Common Mistake

Choosing an inner join when unmatched rows matter, or overlooking one-to-many duplication.

Interview Follow-up

Change the join type and predict which unmatched rows appear or disappear.

Question 67 — Join users to roles

Concepts: JOINS, foreign keys, cardinality, unmatched-row preservation

Difficulty: Basic / Intermediate

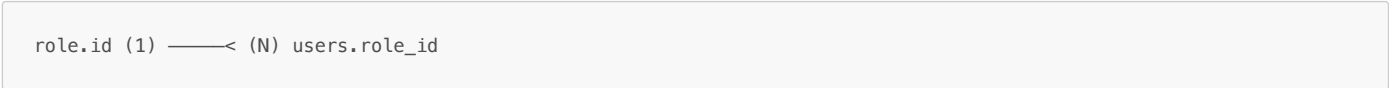
Relevant Table — users

user_id	name	role_id
1	Asha Admin	1
2	Ravi Shah	2
3	Meera Nair	3

Relevant Table — **role**

id	name
1	admin
2	employee
3	agent

Relationship



Think About It

- Which keys match the rows?
- Which table's unmatched rows must remain in the result?

Question

Return each user's role name.

Answer

```
SELECT u.user_id,
       u.name,
       r.name AS role_name
FROM users AS u
JOIN role AS r
  ON r.id = u.role_id
ORDER BY u.user_id;
```

Expected Output

user_id	name	role_name
1	Asha Admin	admin
2	Ravi Shah	employee
3	Meera Nair	agent

How It Works: **role_id** stores the key, while the join retrieves the human-readable role.

Why This Is Asked in Interviews

Tests relationship reading, join selection, aliases, and row multiplication.

Common Mistake

Choosing an inner join when unmatched rows matter, or overlooking one-to-many duplication.

Interview Follow-up

Change the join type and predict which unmatched rows appear or disappear.

Question 68 — Join **users** twice for requester and assignee

Concepts: JOINS, foreign keys, cardinality, unmatched-row preservation

Difficulty: Basic / Intermediate

Relevant Table — **tickets**

id	title	requester_id	assignee_id
101	VPN timeout	2	3
105	Monitor flicker	5	NULL

Relevant Table — users

user_id	name
2	Ravi Shah
3	Meera Nair
5	Neha Joshi

Relationships

```
users.user_id (1) ——< tickets.requester_id (N, required)
users.user_id (1) ——< tickets.assignee_id (N, nullable)
```

Think About It

- Which keys match the rows?
- Which table's unmatched rows must remain in the result?

Question

Show requester and assignee names for every ticket.

Answer

```
SELECT t.id,
       requester.name AS requester_name,
       assignee.name AS assignee_name
FROM tickets AS t
JOIN users AS requester
  ON requester.user_id = t.requester_id
LEFT JOIN users AS assignee
  ON assignee.user_id = t.assignee_id
ORDER BY t.id;
```

Expected Output

id	requester_name	assignee_name
101	Ravi Shah	Meera Nair
105	Neha Joshi	NULL

How It Works: Aliases give the same physical table two business roles. Requester is mandatory, so it uses an inner join; assignee is optional, so it uses a left join. This is the pattern used in [TicketRepository](#).

Why This Is Asked in Interviews

Tests relationship reading, join selection, aliases, and row multiplication.

Common Mistake

Choosing an inner join when unmatched rows matter, or overlooking one-to-many duplication.

Interview Follow-up

Change the join type and predict which unmatched rows appear or disappear.

Question 69 — Show ticket comments with author names

Concepts: JOINS, foreign keys, cardinality, unmatched-row preservation

Difficulty: Basic / Intermediate

Relevant Table — comments

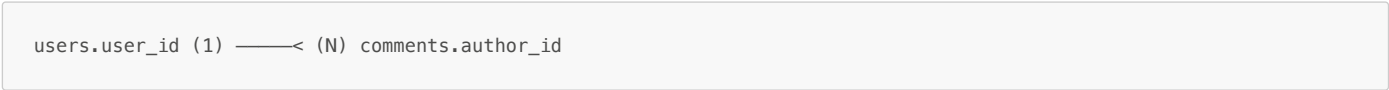
id	ticket_id	author_id	body
1001	101	2	VPN drops every hour
1002	101	3	Checking gateway logs

Relevant Table — users

user_id	name
2	Ravi Shah

user_id	name
3	Meera Nair

Relationship



Think About It

- Which keys match the rows?
- Which table's unmatched rows must remain in the result?

Question

Return the comment stream with author names.

Answer

```
SELECT c.id,
       c.ticket_id,
       u.name AS author_name,
       c.body
FROM comments AS c
JOIN users AS u
  ON u.user_id = c.author_id
ORDER BY c.id;
```

Expected Output

id	ticket_id	author_name	body
1001	101	Ravi Shah	VPN drops every hour
1002	101	Meera Nair	Checking gateway logs

How It Works: The author foreign key identifies one user for each comment. The live repository adds a timestamp ordering tiebreaker.

Why This Is Asked in Interviews

Tests relationship reading, join selection, aliases, and row multiplication.

Common Mistake

Choosing an inner join when unmatched rows matter, or overlooking one-to-many duplication.

Interview Follow-up

Change the join type and predict which unmatched rows appear or disappear.

Question 70 — Traverse the many-to-many ticket/asset relationship

Concepts: JOINS, foreign keys, cardinality, unmatched-row preservation

Difficulty: Basic / Intermediate

Relevant Table — tickets

id	title
101	VPN timeout
105	Monitor flicker

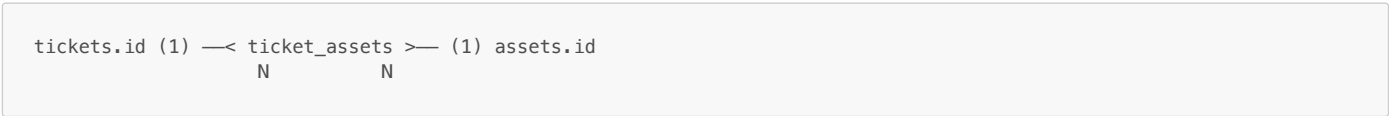
Relevant Table — ticket_assets

ticket_id	asset_id
101	201
101	204
105	202

Relevant Table — assets

id	tag	type
201	LT-001	LAPTOP
202	MON-001	MONITOR
204	DOCK-001	DOCK

Relationship



Think About It

- Which keys match the rows?
- Which table's unmatched rows must remain in the result?

Question

List every ticket/asset pair with readable values.

Answer

```
SELECT t.id AS ticket_id,
       t.title,
       a.tag AS asset_tag,
       a.type AS asset_type
FROM tickets AS t
JOIN ticket_assets AS ta
  ON ta.ticket_id = t.id
JOIN assets AS a
  ON a.id = ta.asset_id
ORDER BY t.id, a.tag;
```

Expected Output

ticket_id	title	asset_tag	asset_type
101	VPN timeout	DOCK-001	DOCK
101	VPN timeout	LT-001	LAPTOP
105	Monitor flicker	MON-001	MONITOR

How It Works: The bridge converts a many-to-many relationship into two one-to-many joins. Ticket 101 correctly appears twice because two assets are linked.

Why This Is Asked in Interviews

Tests relationship reading, join selection, aliases, and row multiplication.

Common Mistake

Choosing an inner join when unmatched rows matter, or overlooking one-to-many duplication.

Interview Follow-up

Change the join type and predict which unmatched rows appear or disappear.

Question 71 — Express an all-users query with **RIGHT JOIN**

Concepts: JOINS, foreign keys, cardinality, unmatched-row preservation

Difficulty: Basic / Intermediate

Relevant Table — **tickets**

id	requester_id
101	2
102	5

Relevant Table — **users**

user_id	name
2	Ravi Shah

user_id	name
5	Neha Joshi
6	Ishan Bose

Relationship

tickets.requester_id (N) >—— (1) users.user_id
RIGHT JOIN preserves the users side.

Think About It

- Which keys match the rows?
- Which table's unmatched rows must remain in the result?

Question

Use **RIGHT JOIN** to show all users, even those who have not requested a ticket.

Answer

```
SELECT u.user_id,
       u.name,
       t.id AS ticket_id
FROM tickets AS t
RIGHT JOIN users AS u
      ON u.user_id = t.requester_id
ORDER BY u.user_id, t.id;
```

Expected Output

user_id	name	ticket_id
2	Ravi Shah	101
5	Neha Joshi	102
6	Ishan Bose	NULL

How It Works: Every right-side user survives. Most teams rewrite this as `users u LEFT JOIN tickets t` because reading from the preserved table first is easier.

Why This Is Asked in Interviews

Tests relationship reading, join selection, aliases, and row multiplication.

Common Mistake

Choosing an inner join when unmatched rows matter, or overlooking one-to-many duplication.

Interview Follow-up

Change the join type and predict which unmatched rows appear or disappear.

Question 72 — Self-join users who share a role

Concepts: JOINS, foreign keys, cardinality, unmatched-row preservation

Difficulty: Basic / Intermediate

Relevant Table — **users**

user_id	name	role_id
2	Ravi Shah	2
3	Meera Nair	3
4	Kabir Rao	3
6	Ishan Bose	3

Self-join relationship

users u1 — same role_id — users u2
u2.user_id > u1.user_id keeps one direction only.

Think About It

- Which keys match the rows?
- Which table's unmatched rows must remain in the result?

Question

Return each unique pair of different users who share the same role.

Answer

```
SELECT u1.name AS first_user,
       u2.name AS second_user,
       u1.role_id
FROM users AS u1
JOIN users AS u2
  ON u2.role_id = u1.role_id
 AND u2.user_id > u1.user_id
ORDER BY first_user, second_user;
```

Expected Output

first_user	second_user	role_id
Kabir Rao	Ishan Bose	3
Meera Nair	Ishan Bose	3
Meera Nair	Kabir Rao	3

How It Works: Two aliases represent two user instances. The > condition prevents pairing a user with themselves and prevents reversed duplicates. Display order follows names, not IDs.

Why This Is Asked in Interviews

Tests relationship reading, join selection, aliases, and row multiplication.

Common Mistake

Choosing an inner join when unmatched rows matter, or overlooking one-to-many duplication.

Interview Follow-up

Change the join type and predict which unmatched rows appear or disappear.

Question 73 — Build every role/status combination with CROSS JOIN

Concepts: JOINS, foreign keys, cardinality, unmatched-row preservation

Difficulty: Basic / Intermediate

Relevant Table — role

id	name
1	admin
2	employee
3	agent

Relevant Table — tickets

id	status
101	OPEN
102	IN_PROGRESS
105	OPEN

Cross-join relationship

3 role rows × 2 distinct status rows = 6 result rows
There is deliberately no matching key.

Think About It

- Which keys match the rows?
- Which table's unmatched rows must remain in the result?

Question

Produce every role crossed with every distinct status currently present.

Answer

```
SELECT r.name AS role_name,  
       s.status  
FROM role AS r  
CROSS JOIN (  
    SELECT DISTINCT status  
    FROM tickets  
) AS s  
ORDER BY r.id, s.status;
```

Expected Output

role_name	status
admin	IN_PROGRESS
admin	OPEN
employee	IN_PROGRESS
employee	OPEN
agent	IN_PROGRESS
agent	OPEN

How It Works: Three role rows multiplied by two status rows produce six combinations. Cross joins are useful for report grids and missing-zero combinations, but accidental cross joins can explode result size.

Why This Is Asked in Interviews

Tests relationship reading, join selection, aliases, and row multiplication.

Common Mistake

Choosing an inner join when unmatched rows matter, or overlooking one-to-many duplication.

Interview Follow-up

Change the join type and predict which unmatched rows appear or disappear.

Part 11 — JOIN + GROUP BY

Joining supplies descriptive columns and missing-parent rows; grouping summarizes the joined result. For a left join, `COUNT(child.id)` returns zero for a parent without children, while `COUNT(*)` returns one null-extended row and is therefore usually wrong for child counts.

Question 74 — Count requested tickets for every user, including zero

Concepts: JOIN, GROUP BY, COUNT, HAVING, cardinality

Difficulty: Intermediate

Relevant Table — `users`

user_id	name
2	Ravi Shah
5	Neha Joshi
6	Ishan Bose

Relevant Table — `tickets`

id	requester_id
101	2
103	2
102	5

Relationship

users.user_id (1) ———< (N) tickets.requester_id
LEFT JOIN preserves users with zero requested tickets.

Think About It

- What rowset does the join create before grouping?
- Should child counts use COUNT(*) or COUNT(child.id)?

Question

Show every user and how many tickets they requested.

Answer

```
SELECT u.user_id,  
       u.name,  
       COUNT(t.id) AS requested_ticket_count  
FROM users AS u  
LEFT JOIN tickets AS t  
      ON t.requester_id = u.user_id  
GROUP BY u.user_id, u.name  
ORDER BY requested_ticket_count DESC, u.user_id;
```

Expected Output

user_id	name	requested_ticket_count
2	Ravi Shah	2
5	Neha Joshi	1
6	Ishan Bose	0

How It Works: The left join preserves Ishan. `COUNT(t.id)` ignores his null-extended ticket value and correctly returns zero.

Why This Is Asked in Interviews

Tests whether you can aggregate across relationships without losing zero-count parents or inflating counts.

Common Mistake

Putting an optional-child filter in WHERE or counting a null-extended row with COUNT(*).

Interview Follow-up

Filter the grouped parents with HAVING or add a second child relationship safely.

Question 75 — Count active workload for every agent

Concepts: JOIN, GROUP BY, COUNT, HAVING, cardinality

Difficulty: Intermediate

Relevant Table — users

user_id	name	role_id	status
3	Meera Nair	3	ACTIVE
4	Kabir Rao	3	ACTIVE
6	Ishan Bose	3	ACTIVE
7	Tara Sen	3	INACTIVE

Relevant Table — role

id	name
3	agent

Relevant Table — tickets

id	assignee_id	status
101	3	OPEN
103	3	RESOLVED
102	4	IN_PROGRESS
104	4	CLOSED

Relationships

```
role.id (1) —< users.role_id
users.user_id (1) —< tickets.assignee_id
```

Think About It

- What rowset does the join create before grouping?
- Should child counts use COUNT(*) or COUNT(child.id)?

Question

Count **OPEN** and **IN_PROGRESS** tickets for every active agent, including agents with zero.

Answer

```
SELECT u.user_id,
       u.name,
       COUNT(t.id) AS active_ticket_count
FROM users AS u
JOIN role AS r
  ON r.id = u.role_id
LEFT JOIN tickets AS t
  ON t.assignee_id = u.user_id
  AND t.status IN ('OPEN', 'IN_PROGRESS')
WHERE u.status = 'ACTIVE'
  AND r.name = 'agent'
GROUP BY u.user_id, u.name
ORDER BY active_ticket_count, u.user_id;
```

Expected Output

user_id	name	active_ticket_count
6	Ishan Bose	0
3	Meera Nair	1
4	Kabir Rao	1

How It Works: The ticket-status condition stays in **ON**, so an agent with no active match survives. Putting it in **WHERE** would remove Ishan. This is the core of PulseDesk auto-assignment.

Why This Is Asked in Interviews

Tests whether you can aggregate across relationships without losing zero-count parents or inflating counts.

Common Mistake

Putting an optional-child filter in WHERE or counting a null-extended row with COUNT(*).

Interview Follow-up

Filter the grouped parents with HAVING or add a second child relationship safely.

Question 76 — Select the least-loaded active agent

Concepts: JOIN, GROUP BY, COUNT, HAVING, cardinality

Difficulty: Intermediate

Relevant Table — users

user_id	name	role_id	status
3	Meera Nair	3	ACTIVE
4	Kabir Rao	3	ACTIVE
6	Ishan Bose	3	ACTIVE

Relevant Table — **role**

id	name
3	agent

Relevant Table — **tickets**

id	assignee_id	status
101	3	OPEN
110	3	IN_PROGRESS
102	4	IN_PROGRESS
104	4	CLOSED

Relationships

```
role.id (1) —< users.role_id
users.user_id (1) —< tickets.assignee_id
group agent rows -> count active child rows -> sort ascending -> LIMIT 1
```

Think About It

- What rowset does the join create before grouping?
- Should child counts use COUNT(*) or COUNT(child.id)?

Question

Return one active agent with the smallest active workload; break ties by user ID.

Answer

```
SELECT u.user_id,
       u.name,
       COUNT(t.id) AS active_ticket_count
FROM users AS u
JOIN role AS r
  ON r.id = u.role_id
LEFT JOIN tickets AS t
  ON t.assignee_id = u.user_id
  AND t.status IN ('OPEN', 'IN_PROGRESS')
WHERE u.status = 'ACTIVE'
  AND r.name = 'agent'
GROUP BY u.user_id, u.name
ORDER BY active_ticket_count ASC, u.user_id ASC
LIMIT 1;
```

Expected Output

user_id	name	active_ticket_count
6	Ishan Bose	0

How It Works: The groups include zero-work agents, ordering brings the smallest count first, and **LIMIT 1** chooses a deterministic winner. PulseDesk's repository uses this exact pattern, selecting only the ID.

Why This Is Asked in Interviews

Tests whether you can aggregate across relationships without losing zero-count parents or inflating counts.

Common Mistake

Putting an optional-child filter in WHERE or counting a null-extended row with COUNT(*)

Interview Follow-up

Filter the grouped parents with HAVING or add a second child relationship safely.

Question 77 — Count comments for every ticket, including zero

Concepts: JOIN, GROUP BY, COUNT, HAVING, cardinality

Difficulty: Intermediate

Relevant Table — **tickets**

id	title
101	VPN timeout
102	Laptop battery
103	IDE licence

Relevant Table — **comments**

id	ticket_id
1001	101
1002	101
1003	101
1004	103

Relationship

tickets.id (1) —< (N) comments.ticket_id
LEFT JOIN preserves a zero-comment ticket.

Think About It

- What rowset does the join create before grouping?
- Should child counts use COUNT(*) or COUNT(child.id)?

Question

Return every ticket with its comment count.

Answer

```
SELECT t.id,
       t.title,
       COUNT(c.id) AS comment_count
FROM tickets AS t
LEFT JOIN comments AS c
      ON c.ticket_id = t.id
GROUP BY t.id, t.title
ORDER BY t.id;
```

Expected Output

id	title	comment_count
101	VPN timeout	3
102	Laptop battery	0
103	IDE licence	1

How It Works: Each comment becomes a joined child row; **COUNT(c.id)** counts real child IDs and returns zero for the unmatched ticket.

Why This Is Asked in Interviews

Tests whether you can aggregate across relationships without losing zero-count parents or inflating counts.

Common Mistake

Putting an optional-child filter in WHERE or counting a null-extended row with COUNT(*).

Interview Follow-up

Filter the grouped parents with HAVING or add a second child relationship safely.

Question 78 — Find tickets with multiple comments

Concepts: JOIN, GROUP BY, COUNT, HAVING, cardinality

Difficulty: Intermediate

Relevant Table — tickets

id	title
101	VPN timeout
102	Laptop battery
103	IDE licence

Relevant Table — comments

id	ticket_id
1001	101
1002	101
1003	101
1004	103

Relationship

tickets.id (1) —< (N) comments.ticket_id
JOIN creates child rows; GROUP BY returns to one row per ticket.

Think About It

- What rowset does the join create before grouping?
- Should child counts use COUNT(*) or COUNT(child.id)?

Question

Return tickets having at least two comments, most discussed first.

Answer

```
SELECT t.id,
       t.title,
       COUNT(c.id) AS comment_count
FROM tickets AS t
JOIN comments AS c
  ON c.ticket_id = t.id
GROUP BY t.id, t.title
HAVING COUNT(c.id) >= 2
ORDER BY comment_count DESC, t.id;
```

Expected Output

id	title	comment_count
101	VPN timeout	3

How It Works: An inner join is sufficient because zero-comment tickets cannot pass the **HAVING** threshold.

Why This Is Asked in Interviews

Tests whether you can aggregate across relationships without losing zero-count parents or inflating counts.

Common Mistake

Putting an optional-child filter in WHERE or counting a null-extended row with COUNT(*).

Interview Follow-up

Filter the grouped parents with HAVING or add a second child relationship safely.

Question 79 — Count affected assets per ticket

Concepts: JOIN, GROUP BY, COUNT, HAVING, cardinality

Difficulty: Intermediate

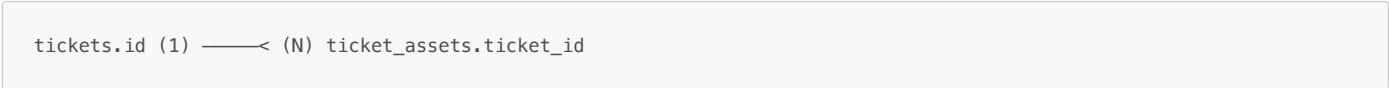
Relevant Table — tickets

id	title
101	VPN timeout
102	Laptop battery
105	Monitor flicker

Relevant Table — ticket_assets

ticket_id	asset_id
101	201
101	204
105	202

Relationship



Think About It

- What rowset does the join create before grouping?
- Should child counts use COUNT(*) or COUNT(child.id)?

Question

Show the number of linked assets for every ticket.

Answer

```
SELECT t.id,
       t.title,
       COUNT(ta.asset_id) AS affected_asset_count
FROM tickets AS t
LEFT JOIN ticket_assets AS ta
      ON ta.ticket_id = t.id
GROUP BY t.id, t.title
ORDER BY t.id;
```

Expected Output

id	title	affected_asset_count
101	VPN timeout	2
102	Laptop battery	0
105	Monitor flicker	1

How It Works: The bridge rows are the items being counted. Its composite key guarantees each asset is counted at most once per ticket.

Why This Is Asked in Interviews

Tests whether you can aggregate across relationships without losing zero-count parents or inflating counts.

Common Mistake

Putting an optional-child filter in WHERE or counting a null-extended row with COUNT(*)

Interview Follow-up

Filter the grouped parents with HAVING or add a second child relationship safely.

Question 80 — Count requested tickets by user role

Concepts: JOIN, GROUP BY, COUNT, HAVING, cardinality

Difficulty: Intermediate

Relevant Table — role

id	name
2	employee
3	agent

Relevant Table — **users**

user_id	name	role_id
2	Ravi Shah	2
5	Neha Joshi	2
3	Meera Nair	3

Relevant Table — **tickets**

id	requester_id
101	2
103	2
102	5
109	3

Relationships

```
role.id (1) —< users.role_id (N)
users.user_id (1) —< tickets.requester_id (N)
```

Think About It

- What rowset does the join create before grouping?
- Should child counts use COUNT(*) or COUNT(child.id)?

Question

Count tickets according to the requester's role.

Answer

```
SELECT r.name AS requester_role,
       COUNT(t.id) AS ticket_count
FROM role AS r
JOIN users AS u
  ON u.role_id = r.id
LEFT JOIN tickets AS t
  ON t.requester_id = u.user_id
GROUP BY r.id, r.name
ORDER BY ticket_count DESC, r.name;
```

Expected Output

requester_role	ticket_count
employee	3
agent	1

How It Works: The join path follows **role** -> **users** -> **tickets**. Grouping at role grain combines tickets requested by several users with the same role.

Why This Is Asked in Interviews

Tests whether you can aggregate across relationships without losing zero-count parents or inflating counts.

Common Mistake

Putting an optional-child filter in WHERE or counting a null-extended row with COUNT(*).

Interview Follow-up

Filter the grouped parents with HAVING or add a second child relationship safely.

Question 81 — Calculate resolution performance by agent

Concepts: JOIN, GROUP BY, COUNT, HAVING, cardinality

Difficulty: Intermediate

Relevant Table — **users**

user_id	name
3	Meera Nair
4	Kabir Rao

Relevant Table — tickets

id	assignee_id	created_at	resolved_at
103	3	2026-09-01 09:00	2026-09-02 12:00
107	3	2026-08-31 09:00	2026-09-01 09:00
104	4	2026-08-20 10:00	2026-08-20 14:00
106	4	2026-09-10 08:00	2026-09-10 16:00
102	4	2026-09-14 10:00	NULL

Relationship

users.user_id (1) —< (N) tickets.assignee_id
completed ticket rows are grouped back to one row per agent.

Think About It

- What rowset does the join create before grouping?
- Should child counts use COUNT(*) or COUNT(child.id)?

Question

For agents with at least two completed tickets, show their average resolution hours.

Answer

```
SELECT u.user_id,
       u.name,
       COUNT(t.id) AS completed_ticket_count,
       ROUND(AVG(TIMESTAMPDIFF(HOUR, t.created_at, t.resolved_at)), 2)
         AS avg_resolution_hours
FROM users AS u
JOIN tickets AS t
  ON t.assignee_id = u.user_id
WHERE t.resolved_at IS NOT NULL
GROUP BY u.user_id, u.name
HAVING COUNT(t.id) >= 2
ORDER BY avg_resolution_hours, u.user_id;
```

Expected Output

user_id	name	completed_ticket_count	avg_resolution_hours
4	Kabir Rao	2	6.00
3	Meera Nair	2	25.50

How It Works: Completed rows are selected before grouping. The join supplies names, **HAVING** enforces the sample size, and the average compares agent performance.

Why This Is Asked in Interviews

Tests whether you can aggregate across relationships without losing zero-count parents or inflating counts.

Common Mistake

Putting an optional-child filter in WHERE or counting a null-extended row with COUNT(*)

Interview Follow-up

Filter the grouped parents with HAVING or add a second child relationship safely.

Question 82 — Avoid double-counting after two one-to-many joins

Concepts: JOIN, GROUP BY, COUNT, HAVING, cardinality

Difficulty: Intermediate

Relevant Table — tickets

id	category
101	NETWORK
105	HARDWARE

Relevant Table — comments

id	ticket_id
1001	101
1002	101

Relevant Table — ticket_assets

ticket_id	asset_id
101	201
101	204
105	202

Relationships

tickets (one row) ———> {< comments
 < ticket_assets

Two independent 1:N joins can multiply: 2 comments × 2 assets = 4 raw rows.

Think About It

- What rowset does the join create before grouping?
- Should child counts use COUNT(*) or COUNT(child.id)?

Question

For each category, count distinct tickets, comments, and linked assets without join multiplication.

Answer

```
SELECT t.category,
       COUNT(DISTINCT t.id) AS ticket_count,
       COUNT(DISTINCT c.id) AS comment_count,
       COUNT(DISTINCT ta.asset_id) AS asset_count
FROM tickets AS t
LEFT JOIN comments AS c
      ON c.ticket_id = t.id
LEFT JOIN ticket_assets AS ta
      ON ta.ticket_id = t.id
GROUP BY t.category
ORDER BY t.category;
```

Expected Output

category	ticket_count	comment_count	asset_count
HARDWARE	1	0	1
NETWORK	1	2	2

How It Works: Ticket 101 creates four raw joined rows (two comments × two assets). **DISTINCT** counts each business key once. For large reports, pre-aggregating each child table in separate CTEs is often more efficient and easier to reason about.

Why This Is Asked in Interviews

Tests whether you can aggregate across relationships without losing zero-count parents or inflating counts.

Common Mistake

Putting an optional-child filter in WHERE or counting a null-extended row with COUNT(*).

Interview Follow-up

Filter the grouped parents with HAVING or add a second child relationship safely.

Part 12 — Subqueries

A subquery is a query nested inside another statement.

- A **scalar subquery** must return one value and can be used in an expression or comparison.
- A **multi-row subquery** feeds operators such as **IN**.
- A **derived table** is a subquery in **FROM** and must have an alias.
- A **correlated subquery** refers to a row from the outer query and is conceptually evaluated for each outer row. MySQL may optimize it, but it can still be expensive.
- **EXISTS** asks whether at least one matching row exists; its select list is irrelevant. **NOT EXISTS** is reliable for anti-matching even when nullable values are involved.

Prefer the clearest expression of the business question. A join is not automatically faster than a subquery; inspect the MySQL execution plan and indexes.

Question chain: Questions 83–90 move from an **IN** set to **EXISTS**, invert it with **NOT EXISTS**, use scalar aggregate results, introduce correlation, compare grouped counts with an average, and finish with latest-child selection.

Non-correlated subquery

Inner query runs conceptually

↓

returns one value or a set

↓

outer query compares/uses that result

Correlated subquery

outer query chooses a current row

↓

inner query refers to a value from that row

↓

returns a row-specific result

↓

outer query keeps or projects the current row

Question 83 — Use **IN** with a subquery to find tickets assigned to active agents

Concepts: scalar and set subqueries, IN, EXISTS, NOT EXISTS, correlation

Difficulty: Intermediate / Advanced

Relevant Table — **users**

user_id	name	role_id	status
3	Meera Nair	3	ACTIVE
4	Kabir Rao	3	INACTIVE
6	Ishan Bose	3	ACTIVE

Relevant Table — **role**

id	name
3	agent

Relevant Table — **tickets**

id	title	assignee_id
101	VPN timeout	3
102	Laptop battery	4
108	MFA reset	6
105	Monitor flicker	NULL

Think About It

- What shape does the inner query return: one value, a set, or existence?
- Does the inner query depend on the current outer row?

Question

Relationships: **role.id (1) —< users.role_id** and **users.user_id (1) —< tickets.assignee_id**.

Return tickets whose assignee is currently an active agent.

Answer

```
SELECT id, title, assignee_id
FROM tickets
WHERE assignee_id IN (
    SELECT u.user_id
    FROM users AS u
    JOIN role AS r
      ON r.id = u.role_id
   WHERE u.status = 'ACTIVE'
     AND r.name = 'agent'
)
ORDER BY id;
```

Expected Output

id	title	assignee_id
101	VPN timeout	3
108	MFA reset	6

How It Works: The inner query produces {3, 6}. The outer query retains tickets whose assignee belongs to that set.

Why This Is Asked in Interviews

Tests query decomposition and whether you can reason about subquery result shape and scope.

Common Mistake

Using = with a multi-row subquery or NOT IN when NULL may be returned.

Interview Follow-up

Rewrite the solution with a join or window function and compare clarity and plan.

Question 84 — Use EXISTS to find tickets with comments

Concepts: scalar and set subqueries, IN, EXISTS, NOT EXISTS, correlation

Difficulty: Intermediate / Advanced

Relevant Table — tickets

id	title
101	VPN timeout
102	Laptop battery
103	IDE licence

Relevant Table — comments

id	ticket_id
1001	101
1002	101
1004	103

Think About It

- What shape does the inner query return: one value, a set, or existence?
- Does the inner query depend on the current outer row?

Question

Return tickets that have at least one comment.

Answer

```
SELECT t.id, t.title
FROM tickets AS t
WHERE EXISTS (
    SELECT 1
    FROM comments AS c
      WHERE c.ticket_id = t.id
)
```

```
WHERE c.ticket_id = t.id
)
ORDER BY t.id;
```

Expected Output

id	title
101	VPN timeout
103	IDE licence

How It Works: The correlated condition connects each outer ticket to its comments. **EXISTS** becomes true as soon as one match is found and does not duplicate ticket 101.

Why This Is Asked in Interviews

Tests query decomposition and whether you can reason about subquery result shape and scope.

Common Mistake

Using = with a multi-row subquery or NOT IN when NULL may be returned.

Interview Follow-up

Rewrite the solution with a join or window function and compare clarity and plan.

Question 85 — Use **NOT EXISTS** to find tickets without comments

Concepts: scalar and set subqueries, IN, EXISTS, NOT EXISTS, correlation

Difficulty: Intermediate / Advanced

Relevant Table — **tickets**

id	title
101	VPN timeout
102	Laptop battery
103	IDE licence

Relevant Table — **comments**

id	ticket_id
1001	101
1004	103

Think About It

- What shape does the inner query return: one value, a set, or existence?
- Does the inner query depend on the current outer row?

Question

Return tickets with no matching comment.

Answer

```
SELECT t.id, t.title
FROM tickets AS t
WHERE NOT EXISTS (
  SELECT 1
  FROM comments AS c
  WHERE c.ticket_id = t.id
);
```

Expected Output

id	title
102	Laptop battery

How It Works: **NOT EXISTS** is true only when the correlated inner query finds no child. It is usually the safest anti-join pattern.

Why This Is Asked in Interviews

Tests query decomposition and whether you can reason about subquery result shape and scope.

Common Mistake

Using = with a multi-row subquery or NOT IN when NULL may be returned.

Interview Follow-up

Rewrite the solution with a join or window function and compare clarity and plan.

Question 86 — Find tickets resolved slower than the overall average

Concepts: scalar and set subqueries, IN, EXISTS, NOT EXISTS, correlation

Difficulty: Intermediate / Advanced

Relevant Table — tickets

id	created_at	resolved_at
103	2026-09-01 09:00	2026-09-02 12:00
104	2026-08-20 10:00	2026-08-20 14:00
106	2026-09-10 08:00	2026-09-10 16:00
107	2026-08-31 09:00	2026-09-01 09:00

Think About It

- What shape does the inner query return: one value, a set, or existence?
- Does the inner query depend on the current outer row?

Question

Return completed tickets whose resolution hours exceed the completed-ticket average.

Answer

```
SELECT id,
       TIMESTAMPDIFF(HOUR, created_at, resolved_at) AS resolution_hours
FROM tickets
WHERE resolved_at IS NOT NULL
      AND TIMESTAMPDIFF(HOUR, created_at, resolved_at) > (
    SELECT AVG(TIMESTAMPDIFF(HOUR, created_at, resolved_at))
    FROM tickets
    WHERE resolved_at IS NOT NULL
  )
ORDER BY resolution_hours DESC;
```

Expected Output

id	resolution_hours
103	27
107	24

How It Works: The scalar subquery returns the overall average $(27+4+8+24)/4 = 15.75$; the outer query compares each duration to it.

Subquery data flow

Inner aggregate query
durations = [27, 4, 8, 24]
AVG(...) = 15.75
↓ scalar result
Outer query compares each completed ticket
27 > 15.75 -> keep ticket 103
4 > 15.75 -> remove ticket 104
8 > 15.75 -> remove ticket 106
24 > 15.75 -> keep ticket 107
↓
ORDER BY resolution_hours DESC -> 103, 107

Why This Is Asked in Interviews

Tests query decomposition and whether you can reason about subquery result shape and scope.

Common Mistake

Using = with a multi-row subquery or NOT IN when NULL may be returned.

Interview Follow-up

Rewrite the solution with a join or window function and compare clarity and plan.

Question 87 — Show each user with a correlated ticket count

Concepts: scalar and set subqueries, IN, EXISTS, NOT EXISTS, correlation

Difficulty: Intermediate / Advanced

Relevant Table — **users**

user_id	name
2	Ravi Shah
5	Neha Joshi
6	Ishan Bose

Relevant Table — **tickets**

id	requester_id
101	2
103	2
102	5

Think About It

- What shape does the inner query return: one value, a set, or existence?
- Does the inner query depend on the current outer row?

Question

Use a scalar correlated subquery to calculate each user's requested-ticket count.

Answer

```
SELECT u.user_id,
       u.name,
       (
         SELECT COUNT(*)
         FROM tickets AS t
         WHERE t.requester_id = u.user_id
       ) AS requested_ticket_count
FROM users AS u
ORDER BY u.user_id;
```

Expected Output

user_id	name	requested_ticket_count
2	Ravi Shah	2
5	Neha Joshi	1
6	Ishan Bose	0

How It Works: The inner count refers to the current outer user. A left join plus grouping is often better for bulk reports, but this version clearly demonstrates correlation.

Why This Is Asked in Interviews

Tests query decomposition and whether you can reason about subquery result shape and scope.

Common Mistake

Using = with a multi-row subquery or NOT IN when NULL may be returned.

Interview Follow-up

Rewrite the solution with a join or window function and compare clarity and plan.

Question 88 — Find users whose request count is above the average requester count

Concepts: scalar and set subqueries, IN, EXISTS, NOT EXISTS, correlation

Difficulty: Intermediate / Advanced

Relevant Table — `users`

user_id	name
2	Ravi Shah
5	Neha Joshi
6	Ishan Bose

Relevant Table — `tickets`

id	requester_id
101	2
103	2
104	2
102	5

Think About It

- What shape does the inner query return: one value, a set, or existence?
- Does the inner query depend on the current outer row?

Question

Relationship: `users.user_id (1) ———< (N) tickets.requester_id`; the left join keeps zero-ticket users in the derived counts.

Include zero-ticket users, then return users whose count exceeds the average count across all users.

Answer

```
WITH user_counts AS (
  SELECT u.user_id,
         u.name,
         COUNT(t.id) AS ticket_count
  FROM users AS u
  LEFT JOIN tickets AS t
    ON t.requester_id = u.user_id
  GROUP BY u.user_id, u.name
)
SELECT user_id, name, ticket_count
FROM user_counts
WHERE ticket_count > (
  SELECT AVG(ticket_count)
  FROM user_counts
)
ORDER BY ticket_count DESC, user_id;
```

Expected Output

user_id	name	ticket_count
2	Ravi Shah	3

How It Works: The CTE produces counts `{3,1,0}` and the scalar subquery averages them to `1.333...`. Defining whether zero-ticket users belong in the average is a business requirement, not a syntax detail.

Why This Is Asked in Interviews

Tests query decomposition and whether you can reason about subquery result shape and scope.

Common Mistake

Using `=` with a multi-row subquery or `NOT IN` when `NULL` may be returned.

Interview Follow-up

Rewrite the solution with a join or window function and compare clarity and plan.

Question 89 — Find categories whose volume is above the average category volume

Concepts: scalar and set subqueries, IN, EXISTS, NOT EXISTS, correlation

Difficulty: Intermediate / Advanced

Relevant Table — tickets

id	category
101	NETWORK
107	NETWORK
102	HARDWARE
105	HARDWARE
109	HARDWARE
103	SOFTWARE

Think About It

- What shape does the inner query return: one value, a set, or existence?
- Does the inner query depend on the current outer row?

Question

Return categories whose ticket count is above the average count of represented categories.

Answer

```
WITH category_counts AS (  
    SELECT category, COUNT(*) AS ticket_count  
    FROM tickets  
    GROUP BY category  
)  
SELECT category, ticket_count  
FROM category_counts  
WHERE ticket_count > (  
    SELECT AVG(ticket_count)  
    FROM category_counts  
);
```

Expected Output

category	ticket_count
HARDWARE	3

How It Works: Category counts are {3,2,1}, whose average is 2. Only hardware is strictly above it. The CTE avoids repeating the grouped subquery.

Why This Is Asked in Interviews

Tests query decomposition and whether you can reason about subquery result shape and scope.

Common Mistake

Using = with a multi-row subquery or NOT IN when NULL may be returned.

Interview Follow-up

Rewrite the solution with a join or window function and compare clarity and plan.

Question 90 — Find each ticket's latest comment with a correlated subquery

Concepts: scalar and set subqueries, IN, EXISTS, NOT EXISTS, correlation

Difficulty: Intermediate / Advanced

Relevant Table — tickets

id	title
101	VPN timeout
103	IDE licence

Relevant Table — comments

id	ticket_id	body	created_at
1001	101	VPN drops every hour	2026-09-14 09:15

id	ticket_id	body	created_at
1002	101	Checking gateway logs	2026-09-14 09:30
1003	101	Issue persists	2026-09-14 11:00
1004	103	Licence refreshed	2026-09-01 10:00

Think About It

- What shape does the inner query return: one value, a set, or existence?
- Does the inner query depend on the current outer row?

Question

Relationship: `tickets.id (1) —>< (N) comments.ticket_id`.

Return the most recent comment for each ticket. Break timestamp ties with the largest comment ID.

Answer

```
SELECT t.id AS ticket_id,
       c.id AS comment_id,
       c.body,
       c.created_at
FROM tickets AS t
JOIN comments AS c
  ON c.ticket_id = t.id
WHERE c.id = (
  SELECT c2.id
  FROM comments AS c2
  WHERE c2.ticket_id = t.id
  ORDER BY c2.created_at DESC, c2.id DESC
  LIMIT 1
)
ORDER BY t.id;
```

Expected Output

ticket_id	comment_id	body	created_at
101	1003	Issue persists	2026-09-14 11:00
103	1004	Licence refreshed	2026-09-01 10:00

How It Works: The correlated subquery chooses one comment ID for the current ticket. A later window-function solution is usually easier to extend.

Why This Is Asked in Interviews

Tests query decomposition and whether you can reason about subquery result shape and scope.

Common Mistake

Using `=` with a multi-row subquery or `NOT IN` when `NULL` may be returned.

Interview Follow-up

Rewrite the solution with a join or window function and compare clarity and plan.

Part 13 — Conditional Aggregation

Conditional aggregation puts conditions inside aggregates. It can pivot several metrics into one row or one row per group without separate queries.

Question chain: Questions 91–95 first turn status rows into four dashboard columns, then add agent grouping, category/priority pivots, null-based read state, and a multi-bucket asset inventory report.

Portable forms are:

```
SUM(CASE WHEN condition THEN 1 ELSE 0 END)
COUNT(CASE WHEN condition THEN 1 END)
```

The `COUNT` form works because unmatched rows return `NULL`, which `COUNT(expression)` ignores. Use an explicit `ELSE 0` for sums so every row contributes a number.

Question 91 — Return all ticket-status counts in one row

Concepts: CASE, SUM, COUNT, conditional aggregation, pivot-style reports

Difficulty: Intermediate

Relevant Table — tickets

id	status
101	OPEN
102	IN_PROGRESS
103	RESOLVED
104	CLOSED
105	OPEN
106	RESOLVED

Think About It

- What number or NULL does CASE emit for each row?
- What will the outer aggregate do with those emitted values?

Question

Produce one dashboard row containing one count per status.

Answer

```
SELECT SUM(CASE WHEN status = 'OPEN' THEN 1 ELSE 0 END) AS open_count,
       SUM(CASE WHEN status = 'IN_PROGRESS' THEN 1 ELSE 0 END) AS in_progress_count,
       SUM(CASE WHEN status = 'RESOLVED' THEN 1 ELSE 0 END) AS resolved_count,
       SUM(CASE WHEN status = 'CLOSED' THEN 1 ELSE 0 END) AS closed_count
FROM tickets;
```

Expected Output

open_count	in_progress_count	resolved_count	closed_count
2	1	2	1

How It Works: Each CASE emits 1 only for its target status, and each SUM totals a separate column over the same scan.

Row-by-row evaluation

ticket	status	OPEN case	IN_PROGRESS case	RESOLVED case	CLOSED case
101	OPEN	1	0	0	0
102	IN_PROGRESS	0	1	0	0
103	RESOLVED	0	0	1	0
104	CLOSED	0	0	0	1
105	OPEN	1	0	0	0
106	RESOLVED	0	0	1	0
SUM		2	1	2	1

The CASE expressions turn categories into numeric flags. SUM does ordinary addition; that addition becomes a count because matches contribute 1 and nonmatches contribute 0.

Why This Is Asked in Interviews

Tests whether you can derive several business metrics in one grouped scan.

Common Mistake

Omitting ELSE 0 in a SUM pattern or creating overlapping conditions unintentionally.

Interview Follow-up

Add a percentage or another mutually exclusive bucket to the same report.

Question 92 — Count active and completed tickets per agent

Concepts: CASE, SUM, COUNT, conditional aggregation, pivot-style reports

Difficulty: Intermediate

Relevant Table — `users`

user_id	name	role_id	status
3	Meera Nair	3	ACTIVE
4	Kabir Rao	3	ACTIVE
6	Ishan Bose	3	ACTIVE

Relevant Table — `role`

id	name
3	agent

Relevant Table — `tickets`

id	assignee_id	status
101	3	OPEN
103	3	RESOLVED
107	3	CLOSED
102	4	IN_PROGRESS
106	4	RESOLVED

Think About It

- What number or NULL does CASE emit for each row?
- What will the outer aggregate do with those emitted values?

Question

Relationships: `role.id (1) —< users.role_id` and `users.user_id (1) —< tickets.assignee_id`.

For every shown agent, report active and completed ticket counts.

Answer

```
SELECT u.user_id,
       u.name,
       COUNT(t.id) AS total_count,
       COUNT(CASE WHEN t.status IN ('OPEN', 'IN_PROGRESS') THEN 1 END)
         AS active_count,
       COUNT(CASE WHEN t.status IN ('RESOLVED', 'CLOSED') THEN 1 END)
         AS completed_count
FROM users AS u
JOIN role AS r
  ON r.id = u.role_id
LEFT JOIN tickets AS t
  ON t.assignee_id = u.user_id
WHERE u.status = 'ACTIVE'
      AND r.name = 'agent'
GROUP BY u.user_id, u.name
ORDER BY u.user_id;
```

Expected Output

user_id	name	total_count	active_count	completed_count
3	Meera Nair	3	1	2
4	Kabir Rao	2	1	1
6	Ishan Bose	0	0	0

How It Works: The left join retains Ishan. `COUNT(t.id)` counts every real ticket, while each conditional `COUNT` ignores rows where its `CASE` returns `NULL`.

Why This Is Asked in Interviews

Tests whether you can derive several business metrics in one grouped scan.

Common Mistake

Omitting `ELSE 0` in a `SUM` pattern or creating overlapping conditions unintentionally.

Interview Follow-up

Add a percentage or another mutually exclusive bucket to the same report.

Question 93 — Build a priority matrix per category

Concepts: CASE, SUM, COUNT, conditional aggregation, pivot-style reports

Difficulty: Intermediate

Relevant Table — tickets

id	category	priority
101	NETWORK	HIGH
107	NETWORK	LOW
102	HARDWARE	MEDIUM
105	HARDWARE	HIGH
109	HARDWARE	HIGH

Think About It

- What number or NULL does CASE emit for each row?
- What will the outer aggregate do with those emitted values?

Question

Show low, medium, and high ticket counts for every category.

Answer

```
SELECT category,
       SUM(CASE WHEN priority = 'LOW' THEN 1 ELSE 0 END) AS low_count,
       SUM(CASE WHEN priority = 'MEDIUM' THEN 1 ELSE 0 END) AS medium_count,
       SUM(CASE WHEN priority = 'HIGH' THEN 1 ELSE 0 END) AS high_count
FROM tickets
GROUP BY category
ORDER BY category;
```

Expected Output

category	low_count	medium_count	high_count
HARDWARE	0	1	2
NETWORK	1	0	1

How It Works: Grouping fixes one row per category; the three conditions split its rows into priority columns.

Why This Is Asked in Interviews

Tests whether you can derive several business metrics in one grouped scan.

Common Mistake

Omitting ELSE 0 in a SUM pattern or creating overlapping conditions unintentionally.

Interview Follow-up

Add a percentage or another mutually exclusive bucket to the same report.

Question 94 — Count read and unread notifications per recipient

Concepts: CASE, SUM, COUNT, conditional aggregation, pivot-style reports

Difficulty: Intermediate

Relevant Table — users

user_id	name
2	Ravi Shah
3	Meera Nair
4	Kabir Rao

Relevant Table — notifications

id	recipient_user_id	read_at
701	3	NULL
702	2	2026-09-14 10:00
703	2	NULL
704	4	NULL

Think About It

- What number or NULL does CASE emit for each row?
- What will the outer aggregate do with those emitted values?

Question

Relationship: users.user_id (1) —< (N) notifications.recipient_user_id.

Report read and unread counts for each recipient who has notifications.

Answer

```
SELECT u.user_id,
       u.name,
       SUM(CASE WHEN n.read_at IS NULL THEN 1 ELSE 0 END) AS unread_count,
       SUM(CASE WHEN n.read_at IS NOT NULL THEN 1 ELSE 0 END) AS read_count
FROM users AS u
JOIN notifications AS n
  ON n.recipient_user_id = u.user_id
GROUP BY u.user_id, u.name
ORDER BY u.user_id;
```

Expected Output

user_id	name	unread_count	read_count
2	Ravi Shah	1	1
3	Meera Nair	1	0
4	Kabir Rao	1	0

How It Works: Nullness represents notification state, so complementary conditions partition every joined notification into exactly one count.

Why This Is Asked in Interviews

Tests whether you can derive several business metrics in one grouped scan.

Common Mistake

Omitting ELSE 0 in a SUM pattern or creating overlapping conditions unintentionally.

Interview Follow-up

Add a percentage or another mutually exclusive bucket to the same report.

Question 95 — Summarize asset states per asset type

Concepts: CASE, SUM, COUNT, conditional aggregation, pivot-style reports

Difficulty: Intermediate

Relevant Table — assets

id	type	status
201	LAPTOP	IN_USE
203	LAPTOP	IN_USE
207	LAPTOP	IN_STOCK
202	MONITOR	IN_REPAIR
208	MONITOR	IN_STOCK

Think About It

- What number or NULL does CASE emit for each row?

- What will the outer aggregate do with those emitted values?

Question

For each asset type, show total, in-use, in-stock, and unavailable counts (**IN_REPAIR** or **RETIRED**).

Answer

```
SELECT type,
       COUNT(*) AS total_count,
       SUM(CASE WHEN status = 'IN_USE' THEN 1 ELSE 0 END) AS in_use_count,
       SUM(CASE WHEN status = 'IN_STOCK' THEN 1 ELSE 0 END) AS in_stock_count,
       SUM(CASE WHEN status IN ('IN_REPAIR', 'RETIRED') THEN 1 ELSE 0 END)
         AS unavailable_count
FROM assets
GROUP BY type
ORDER BY type;
```

Expected Output

type	total_count	in_use_count	in_stock_count	unavailable_count
LAPTOP	3	2	1	0
MONITOR	2	0	1	1

How It Works: Several metrics are computed over each type group in one pass. The category definitions should be mutually exclusive when they are expected to sum to the total.

Why This Is Asked in Interviews

Tests whether you can derive several business metrics in one grouped scan.

Common Mistake

Omitting ELSE 0 in a SUM pattern or creating overlapping conditions unintentionally.

Interview Follow-up

Add a percentage or another mutually exclusive bucket to the same report.

Part 14 — Advanced Interview Query Patterns

MySQL 8 window functions calculate across related rows without collapsing them. **GROUP BY** turns a group into one row; a window function keeps each row and adds a value such as rank, running total, or group average. Common table expressions (CTEs) name intermediate query results so complex logic can be built and tested in stages.

Question chain: Questions 96–104 build CTEs and window functions for pre-aggregation, latest rows, ranks, running totals, percentages, previous-row comparisons, cursor paging, full-text search, and missing dates. Questions 105–114 then apply classic duplicate, second/Nth-highest, top-N, group-wise maximum, above-average, latest-child, and anti-join patterns to PulseDesk.

Question 96 — Pre-aggregate child tables before joining

Concepts: CTEs, window functions, ranking, running totals, keyset pagination

Difficulty: Advanced

Relevant Table — **tickets**

id	title
101	VPN timeout
102	Laptop battery

Relevant Table — **comments**

id	ticket_id
1001	101
1002	101

Relevant Table — **ticket_assets**

ticket_id	asset_id
101	201
101	204
102	203

Think About It

- What grain should each CTE produce?
- How should the window partition, order, and handle ties?

Question

Relationships: `tickets.id (1) —< comments.ticket_id` and `tickets.id (1) —< ticket_assets.ticket_id`; each child is aggregated before the summaries are joined.

Return comment and asset counts per ticket without multiplying the two child sets.

Answer

```
WITH comment_counts AS (  
  SELECT ticket_id, COUNT(*) AS comment_count  
  FROM comments  
  GROUP BY ticket_id  
)  
,  
asset_counts AS (  
  SELECT ticket_id, COUNT(*) AS asset_count  
  FROM ticket_assets  
  GROUP BY ticket_id  
)  
SELECT t.id,  
       t.title,  
       COALESCE(cc.comment_count, 0) AS comment_count,  
       COALESCE(ac.asset_count, 0) AS asset_count  
FROM tickets AS t  
LEFT JOIN comment_counts AS cc  
  ON cc.ticket_id = t.id  
LEFT JOIN asset_counts AS ac  
  ON ac.ticket_id = t.id  
ORDER BY t.id;
```

Expected Output

id	title	comment_count	asset_count
101	VPN timeout	2	2
102	Laptop battery	0	1

How It Works: Each CTE first reduces its child table to one row per ticket. Joining those summaries cannot create a comments-times-assets product.

Why This Is Asked in Interviews

Tests layered query design and MySQL 8 analytical features.

Common Mistake

Filtering a window result in the same WHERE phase or omitting a deterministic window order.

Interview Follow-up

Rewrite the query for tied rows, missing periods, or a larger indexed dataset.

Question 97 — Find the latest comment per ticket with `ROW_NUMBER`

Concepts: CTEs, window functions, ranking, running totals, keyset pagination

Difficulty: Advanced

Relevant Table — `comments`

id	ticket_id	body	created_at
1001	101	VPN drops every hour	2026-09-14 09:15
1002	101	Checking gateway logs	2026-09-14 09:30
1003	101	Issue persists	2026-09-14 11:00

id	ticket_id	body	created_at
1004	103	Licence refreshed	2026-09-01 10:00

Think About It

- What grain should each CTE produce?
- How should the window partition, order, and handle ties?

Question

Return exactly one latest comment per ticket, breaking timestamp ties by ID.

Answer

```
WITH ranked_comments AS (  
    SELECT c.*,  
           ROW_NUMBER() OVER (  
               PARTITION BY ticket_id  
               ORDER BY created_at DESC, id DESC  
           ) AS rn  
    FROM comments AS c  
)  
SELECT ticket_id, id AS comment_id, body, created_at  
FROM ranked_comments  
WHERE rn = 1  
ORDER BY ticket_id;
```

Expected Output

ticket_id	comment_id	body	created_at
101	1003	Issue persists	2026-09-14 11:00
103	1004	Licence refreshed	2026-09-01 10:00

How It Works: `PARTITION BY` restarts numbering for each ticket. Descending order gives the newest row number 1, and the ID tiebreaker guarantees one winner.

Why This Is Asked in Interviews

Tests layered query design and MySQL 8 analytical features.

Common Mistake

Filtering a window result in the same WHERE phase or omitting a deterministic window order.

Interview Follow-up

Rewrite the query for tied rows, missing periods, or a larger indexed dataset.

Question 98 — Rank agents by active workload with ties

Concepts: CTEs, window functions, ranking, running totals, keyset pagination

Difficulty: Advanced

Relevant Table — `users`

user_id	name	role_id	status
3	Meera Nair	3	ACTIVE
4	Kabir Rao	3	ACTIVE
6	Ishan Bose	3	ACTIVE
7	Tara Sen	3	ACTIVE

Relevant Table — `role`

id	name
3	agent

Relevant Table — `tickets`

id	assignee_id	status
101	3	OPEN

id	assignee_id	status
110	3	IN_PROGRESS
102	4	IN_PROGRESS
105	4	OPEN
108	6	OPEN
106	7	RESOLVED

Think About It

- What grain should each CTE produce?
- How should the window partition, order, and handle ties?

Question

Relationships: `role.id (1) —< users.role_id` and `users.user_id (1) —< tickets.assignee_id`.

Rank agents by active tickets, giving tied workloads the same consecutive rank.

Answer

```
WITH workloads AS (
  SELECT u.user_id,
         u.name,
         COUNT(t.id) AS active_ticket_count
  FROM users AS u
  JOIN role AS r
    ON r.id = u.role_id
  LEFT JOIN tickets AS t
    ON t.assignee_id = u.user_id
   AND t.status IN ('OPEN', 'IN_PROGRESS')
 WHERE u.status = 'ACTIVE'
   AND r.name = 'agent'
  GROUP BY u.user_id, u.name
)
SELECT user_id,
       name,
       active_ticket_count,
       DENSE_RANK() OVER (ORDER BY active_ticket_count DESC) AS workload_rank
FROM workloads
ORDER BY workload_rank, user_id;
```

Expected Output

user_id	name	active_ticket_count	workload_rank
3	Meera Nair	2	1
4	Kabir Rao	2	1
6	Ishan Bose	1	2
7	Tara Sen	0	3

How It Works: The CTE derives one workload value per agent. `DENSE_RANK` assigns equal values the same rank and does not leave a gap after ties; `RANK` would produce ranks 1, 1, 3, 4.

Why This Is Asked in Interviews

Tests layered query design and MySQL 8 analytical features.

Common Mistake

Filtering a window result in the same WHERE phase or omitting a deterministic window order.

Interview Follow-up

Rewrite the query for tied rows, missing periods, or a larger indexed dataset.

Question 99 — Calculate a running monthly ticket total

Concepts: CTEs, window functions, ranking, running totals, keyset pagination

Difficulty: Advanced

Relevant Table — `tickets`

id	created_at
100	2026-07-10 10:00
104	2026-08-20 10:00
107	2026-08-31 09:00
103	2026-09-01 09:00
101	2026-09-14 09:00
105	2026-09-15 11:00

Think About It

- What grain should each CTE produce?
- How should the window partition, order, and handle ties?

Question

Show each month's ticket count and the cumulative count through that month.

Answer

```
WITH monthly_counts AS (  
  SELECT DATE_FORMAT(created_at, '%Y-%m-01') AS month_start,  
         COUNT(*) AS ticket_count  
  FROM tickets  
  GROUP BY DATE_FORMAT(created_at, '%Y-%m-01')  
)  
SELECT month_start,  
       ticket_count,  
       SUM(ticket_count) OVER (  
         ORDER BY month_start  
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
       ) AS running_ticket_count  
FROM monthly_counts  
ORDER BY month_start;
```

Expected Output

month_start	ticket_count	running_ticket_count
2026-07-01	1	1
2026-08-01	2	3
2026-09-01	3	6

How It Works: The CTE establishes monthly grain. The windowed SUM adds the current and all preceding monthly rows while retaining one row per month.

Why This Is Asked in Interviews

Tests layered query design and MySQL 8 analytical features.

Common Mistake

Filtering a window result in the same WHERE phase or omitting a deterministic window order.

Interview Follow-up

Rewrite the query for tied rows, missing periods, or a larger indexed dataset.

Question 100 — Calculate each category's percentage of total tickets

Concepts: CTEs, window functions, ranking, running totals, keyset pagination

Difficulty: Advanced

Relevant Table — tickets

id	category
101	NETWORK
107	NETWORK
102	HARDWARE
105	HARDWARE

id	category
109	HARDWARE
103	SOFTWARE

Think About It

- What grain should each CTE produce?
- How should the window partition, order, and handle ties?

Question

Show category counts and their percentage of all tickets.

Answer

```
WITH category_counts AS (  
  SELECT category, COUNT(*) AS ticket_count  
  FROM tickets  
  GROUP BY category  
)  
SELECT category,  
       ticket_count,  
       ROUND(100.0 * ticket_count / SUM(ticket_count) OVER (), 2)  
         AS percentage_of_total  
FROM category_counts  
ORDER BY ticket_count DESC, category;
```

Expected Output

category	ticket_count	percentage_of_total
HARDWARE	3	50.00
NETWORK	2	33.33
SOFTWARE	1	16.67

How It Works: `SUM(...) OVER ()` computes the grand total without collapsing the three category rows. `100.0` forces non-integer arithmetic.

Why This Is Asked in Interviews

Tests layered query design and MySQL 8 analytical features.

Common Mistake

Filtering a window result in the same WHERE phase or omitting a deterministic window order.

Interview Follow-up

Rewrite the query for tied rows, missing periods, or a larger indexed dataset.

Question 101 — Compare each month with the previous month using LAG

Concepts: CTEs, window functions, ranking, running totals, keyset pagination

Difficulty: Advanced

Relevant Table — tickets

id	created_at
100	2026-07-10
104	2026-08-20
107	2026-08-31
103	2026-09-01
101	2026-09-14
105	2026-09-15

Think About It

- What grain should each CTE produce?
- How should the window partition, order, and handle ties?

Question

Show monthly ticket counts and the change from the previous represented month.

Answer

```
WITH monthly_counts AS (  
    SELECT DATE_FORMAT(created_at, '%Y-%m') AS ticket_month,  
           COUNT(*) AS ticket_count  
    FROM tickets  
    GROUP BY DATE_FORMAT(created_at, '%Y-%m')  
,  
with_previous AS (  
    SELECT ticket_month,  
           ticket_count,  
           LAG(ticket_count) OVER (ORDER BY ticket_month) AS previous_count  
    FROM monthly_counts  
)  
SELECT ticket_month,  
       ticket_count,  
       previous_count,  
       ticket_count - previous_count AS month_change  
FROM with_previous  
ORDER BY ticket_month;
```

Expected Output

ticket_month	ticket_count	previous_count	month_change
2026-07	1	NULL	NULL
2026-08	2	1	1
2026-09	3	2	1

How It Works: `LAG` reads the prior ordered row without a self join. Missing calendar months are absent; generate a calendar if “previous month” must mean the immediately prior calendar month.

Why This Is Asked in Interviews

Tests layered query design and MySQL 8 analytical features.

Common Mistake

Filtering a window result in the same `WHERE` phase or omitting a deterministic window order.

Interview Follow-up

Rewrite the query for tied rows, missing periods, or a larger indexed dataset.

Question 102 — Use keyset pagination instead of a large offset

Concepts: CTEs, window functions, ranking, running totals, keyset pagination

Difficulty: Advanced

Relevant Table — `tickets`

id	title	created_at
108	MFA reset	2026-09-16 08:00
105	Monitor flicker	2026-09-15 11:00
102	Laptop battery	2026-09-14 10:00
101	VPN timeout	2026-09-14 09:00
106	Email crash	2026-09-10 08:00

Think About It

- What grain should each CTE produce?
- How should the window partition, order, and handle ties?

Question

The previous page ended at (`created_at='2026-09-15 11:00', id=105`). Fetch the next two rows in descending order.

Answer

```
SELECT id, title, created_at
FROM tickets
WHERE created_at < '2026-09-15 11:00:00'
      OR (created_at = '2026-09-15 11:00:00' AND id < 105)
ORDER BY created_at DESC, id DESC
LIMIT 2;
```

Expected Output

id	title	created_at
102	Laptop battery	2026-09-14 10:00
101	VPN timeout	2026-09-14 09:00

How It Works: The last seen sort key becomes the cursor. With PulseDesk's matching (`created_at`, `id`) index order, MySQL can seek rather than scan and discard an ever-growing offset.

Why This Is Asked in Interviews

Tests layered query design and MySQL 8 analytical features.

Common Mistake

Filtering a window result in the same WHERE phase or omitting a deterministic window order.

Interview Follow-up

Rewrite the query for tied rows, missing periods, or a larger indexed dataset.

Question 103 — Search ticket text with the full-text index

Concepts: CTEs, window functions, ranking, running totals, keyset pagination

Difficulty: Advanced

Relevant Table — tickets

id	title	description
101	VPN timeout	VPN disconnects every hour
102	Laptop battery	Battery drains within an hour
107	Guest Wi-Fi	Guest wireless network is unavailable

Think About It

- What grain should each CTE produce?
- How should the window partition, order, and handle ties?

Question

Use PulseDesk's full-text index to search for rows relevant to `VPN network`.

Answer

```
SELECT id,
       title,
       MATCH(title, description)
         AGAINST ('VPN network' IN NATURAL LANGUAGE MODE) AS relevance
FROM tickets
WHERE MATCH(title, description)
       AGAINST ('VPN network' IN NATURAL LANGUAGE MODE)
ORDER BY relevance DESC, id;
```

Expected Output (ordering, not exact score)

id	title	relevance
101	VPN timeout	positive score
107	Guest Wi-Fi	positive score

How It Works: The `MATCH` column list exactly matches the schema's full-text index. Relevance scores depend on corpus statistics, so tests should assert matching/order intent rather than hard-code a score from this tiny illustration.

Why This Is Asked in Interviews

Tests layered query design and MySQL 8 analytical features.

Common Mistake

Filtering a window result in the same WHERE phase or omitting a deterministic window order.

Interview Follow-up

Rewrite the query for tied rows, missing periods, or a larger indexed dataset.

Question 104 — Generate missing calendar dates with a recursive CTE

Concepts: CTEs, window functions, ranking, running totals, keyset pagination

Difficulty: Advanced

Relevant Table — tickets

id	created_at
101	2026-09-14 09:00
102	2026-09-14 10:00
105	2026-09-16 11:00

Think About It

- What grain should each CTE produce?
- How should the window partition, order, and handle ties?

Question

Generated relationship: each calendar date has zero or one row in the pre-aggregated daily_counts; the left join preserves every generated date.

Report daily ticket counts from 14 through 16 September, including dates with zero tickets.

Answer

```
WITH RECURSIVE calendar AS (  
    SELECT DATE('2026-09-14') AS report_date  
    UNION ALL  
    SELECT report_date + INTERVAL 1 DAY  
    FROM calendar  
    WHERE report_date < '2026-09-16'  
)  
,  
daily_counts AS (  
    SELECT DATE(created_at) AS report_date,  
           COUNT(*) AS ticket_count  
    FROM tickets  
    WHERE created_at >= '2026-09-14'  
          AND created_at < '2026-09-17'  
    GROUP BY DATE(created_at)  
)  
SELECT c.report_date,  
       COALESCE(d.ticket_count, 0) AS ticket_count  
FROM calendar AS c  
LEFT JOIN daily_counts AS d  
    ON d.report_date = c.report_date  
ORDER BY c.report_date;
```

Expected Output

report_date	ticket_count
2026-09-14	2
2026-09-15	0
2026-09-16	1

How It Works: The recursive CTE creates the complete date dimension. A left join preserves its missing date, and COALESCE converts the absent aggregate to zero.

Why This Is Asked in Interviews

Tests layered query design and MySQL 8 analytical features.

Common Mistake

Filtering a window result in the same WHERE phase or omitting a deterministic window order.

Interview Follow-up

Rewrite the query for tied rows, missing periods, or a larger indexed dataset.

Classic Ranking Patterns in PulseDesk

Before the separate salary section, these exercises apply classic ranking and comparison patterns to a real PulseDesk measure: **tickets per user or agent**. They show that the underlying ideas are reusable even when the business metric changes.

Generic metric wording	PulseDesk translation
Employee metric	Agent/requester ticket count or ticket resolution duration
Group/department	Ticket category or user role
Second/Nth highest value	Second/Nth highest workload
Value above overall average	Resolution time above overall average
Value above group average	Resolution time above category average

The essential step is to derive one metric row per subject first, then rank or compare those rows.

Question 105 — Find duplicate comment content

Concepts: duplicates, ranking, group-wise comparison, latest/missing-child patterns

Difficulty: Advanced

Relevant Table — **comments**

id	ticket_id	author_id	body
1001	101	2	Issue persists
1002	101	2	Issue persists
1003	101	3	Checking gateway logs
1004	103	2	Issue persists

Think About It

- What value must be derived before ranking or comparison?
- Should ties produce one row or several?

Question

Find exact duplicate comment submissions on the same ticket by the same author.

Answer

```
SELECT ticket_id,
       author_id,
       body,
       COUNT(*) AS duplicate_count
FROM comments
GROUP BY ticket_id, author_id, body
HAVING COUNT(*) > 1;
```

Expected Output

ticket_id	author_id	body	duplicate_count
101	2	Issue persists	2

How It Works: All columns defining “duplicate” form the grouping key. The same body on ticket 103 is not a duplicate under this business definition. Detecting full-row duplicates requires first deciding which fields constitute identity.

Why This Is Asked in Interviews

Tests classic interview patterns using a real PulseDesk metric and explicit tie semantics.

Common Mistake

Ranking raw ticket rows before first deriving one metric row per subject.

Interview Follow-up

Solve the same pattern with a different tie rule or a correlated subquery.

Question 106 — Find the second-highest distinct agent workload

Concepts: duplicates, ranking, group-wise comparison, latest/missing-child patterns

Difficulty: Advanced

Relevant Table — `users`

user_id	name	role_id	status
3	Meera Nair	3	ACTIVE
4	Kabir Rao	3	ACTIVE
6	Ishan Bose	3	ACTIVE
7	Tara Sen	3	ACTIVE

Relevant Table — `role`

id	name
3	agent

Relevant Table — `tickets`

id	assignee_id	status
101	3	OPEN
110	3	IN_PROGRESS
111	3	OPEN
102	4	IN_PROGRESS
105	4	OPEN
108	6	OPEN
106	7	RESOLVED

Think About It

- What value must be derived before ranking or comparison?
- Should ties produce one row or several?

Question

Relationships: `role.id (1) —< users.role_id` and `users.user_id (1) —< tickets.assignee_id`.

Return every agent tied at the second-highest active-ticket count.

Answer

```
WITH workloads AS (  
  SELECT u.user_id,  
         u.name,  
         COUNT(t.id) AS active_ticket_count  
  FROM users AS u  
  JOIN role AS r  
    ON r.id = u.role_id  
  LEFT JOIN tickets AS t  
    ON t.assignee_id = u.user_id  
   AND t.status IN ('OPEN', 'IN_PROGRESS')  
 WHERE u.status = 'ACTIVE'  
   AND r.name = 'agent'  
  GROUP BY u.user_id, u.name  
,  
ranked AS (  
  SELECT workloads.*,  
         DENSE_RANK() OVER (ORDER BY active_ticket_count DESC) AS workload_rank  
  FROM workloads  
)  
SELECT user_id, name, active_ticket_count  
FROM ranked  
WHERE workload_rank = 2;
```

Expected Output

user_id	name	active_ticket_count
4	Kabir Rao	2

How It Works: Workload is derived before ranking. **DENSE_RANK** ranks distinct counts, matching the usual meaning of “second-highest distinct salary.”

Why This Is Asked in Interviews

Tests classic interview patterns using a real PulseDesk metric and explicit tie semantics.

Common Mistake

Ranking raw ticket rows before first deriving one metric row per subject.

Interview Follow-up

Solve the same pattern with a different tie rule or a correlated subquery.

Question 107 — Find the Nth-highest distinct workload

Concepts: duplicates, ranking, group-wise comparison, latest/missing-child patterns

Difficulty: Advanced

Relevant Table — users

user_id	name	role_id	status
3	Meera Nair	3	ACTIVE
4	Kabir Rao	3	ACTIVE
6	Ishan Bose	3	ACTIVE
7	Tara Sen	3	ACTIVE

Relevant Table — role

id	name
3	agent

Relevant Table — tickets

id	assignee_id	status
101	3	OPEN
110	3	IN_PROGRESS
111	3	OPEN
102	4	IN_PROGRESS
105	4	OPEN
108	6	OPEN
106	7	RESOLVED

Think About It

- What value must be derived before ranking or comparison?
- Should ties produce one row or several?

Question

Relationships: **role.id (1) —< users.role_id** and **users.user_id (1) —< tickets.assignee_id**.

With **N = 3**, return agents at the third-highest distinct active workload.

Answer

```
WITH workloads AS (  
  SELECT u.user_id,  
         u.name,  
         COUNT(t.id) AS active_ticket_count  
  FROM users AS u  
  JOIN role AS r  
    ON r.id = u.role_id  
  LEFT JOIN tickets AS t  
    ON t.assignee_id = u.user_id  
   AND t.status IN ('OPEN', 'IN_PROGRESS')
```

```
WHERE u.status = 'ACTIVE'
AND r.name = 'agent'
GROUP BY u.user_id, u.name
),
ranked AS (
  SELECT workloads.*,
         DENSE_RANK() OVER (ORDER BY active_ticket_count DESC) AS workload_rank
  FROM workloads
)
SELECT user_id, name, active_ticket_count
FROM ranked
WHERE workload_rank = 3;
```

Expected Output

user_id	name	active_ticket_count
6	Ishan Bose	1

How It Works: Replace the final 3 with a bound parameter. If fewer than N distinct workloads exist, the query correctly returns no rows.

Why This Is Asked in Interviews

Tests classic interview patterns using a real PulseDesk metric and explicit tie semantics.

Common Mistake

Ranking raw ticket rows before first deriving one metric row per subject.

Interview Follow-up

Solve the same pattern with a different tie rule or a correlated subquery.

Question 108 — Return the top N agents

Concepts: duplicates, ranking, group-wise comparison, latest/missing-child patterns

Difficulty: Advanced

Relevant Table — users

user_id	name	role_id	status
3	Meera Nair	3	ACTIVE
4	Kabir Rao	3	ACTIVE
6	Ishan Bose	3	ACTIVE

Relevant Table — role

id	name
3	agent

Relevant Table — tickets

id	assignee_id	status
101	3	OPEN
110	3	IN_PROGRESS
102	4	IN_PROGRESS
105	4	OPEN
108	6	OPEN

Think About It

- What value must be derived before ranking or comparison?
- Should ties produce one row or several?

Question

Relationships: role.id (1) —< users.role_id and users.user_id (1) —< tickets.assignee_id.

Return exactly the top two agents by active workload, breaking ties by user ID.

Answer

```
SELECT u.user_id,
       u.name,
       COUNT(t.id) AS active_ticket_count
FROM users AS u
JOIN role AS r
  ON r.id = u.role_id
LEFT JOIN tickets AS t
  ON t.assignee_id = u.user_id
  AND t.status IN ('OPEN', 'IN_PROGRESS')
WHERE u.status = 'ACTIVE'
  AND r.name = 'agent'
GROUP BY u.user_id, u.name
ORDER BY active_ticket_count DESC, u.user_id
LIMIT 2;
```

Expected Output

user_id	name	active_ticket_count
3	Meera Nair	2
4	Kabir Rao	2

How It Works: LIMIT 2 returns exactly two rows. If the requirement is “top two workload levels including ties,” filter DENSE_RANK() <= 2 instead.

Why This Is Asked in Interviews

Tests classic interview patterns using a real PulseDesk metric and explicit tie semantics.

Common Mistake

Ranking raw ticket rows before first deriving one metric row per subject.

Interview Follow-up

Solve the same pattern with a different tie rule or a correlated subquery.

Question 109 — Find the longest-resolving ticket in each category

Concepts: duplicates, ranking, group-wise comparison, latest/missing-child patterns

Difficulty: Advanced

Relevant Table — tickets

id	category	created_at	resolved_at
103	SOFTWARE	2026-09-01 09:00	2026-09-02 12:00
106	SOFTWARE	2026-09-10 08:00	2026-09-10 16:00
101	NETWORK	2026-09-01 09:00	2026-09-02 09:00
107	NETWORK	2026-08-31 09:00	2026-09-01 09:00
104	ACCESS	2026-08-20 10:00	2026-08-20 14:00

Think About It

- What value must be derived before ranking or comparison?
- Should ties produce one row or several?

Question

Return the group-wise maximum resolution duration, including ties.

Answer

```
WITH durations AS (
  SELECT id,
         category,
         TIMESTAMPDIFF(HOUR, created_at, resolved_at) AS resolution_hours
  FROM tickets
  WHERE resolved_at IS NOT NULL
),
ranked AS (
  SELECT durations.*,
         DENSE_RANK() OVER (
           PARTITION BY category
           ORDER BY resolution_hours DESC
         ) AS rank
  FROM durations
)
```

```
        ) AS duration_rank
    FROM durations
)
SELECT id, category, resolution_hours
FROM ranked
WHERE duration_rank = 1
ORDER BY category, id;
```

Expected Output

id	category	resolution_hours
104	ACCESS	4
101	NETWORK	24
107	NETWORK	24
103	SOFTWARE	27

How It Works: Ranking restarts within each category. `DENSE_RANK = 1` preserves both network tickets tied at the maximum; `ROW_NUMBER = 1` would force one winner.

Why This Is Asked in Interviews

Tests classic interview patterns using a real PulseDesk metric and explicit tie semantics.

Common Mistake

Ranking raw ticket rows before first deriving one metric row per subject.

Interview Follow-up

Solve the same pattern with a different tie rule or a correlated subquery.

Question 110 — Find records above the overall average

Concepts: duplicates, ranking, group-wise comparison, latest/missing-child patterns

Difficulty: Advanced

Relevant Table — tickets

id	title	created_at	resolved_at
103	IDE licence	2026-09-01 09:00	2026-09-02 12:00
104	Payroll access	2026-08-20 10:00	2026-08-20 14:00
106	Email crash	2026-09-10 08:00	2026-09-10 16:00
107	Guest Wi-Fi	2026-08-31 09:00	2026-09-01 09:00

Think About It

- What value must be derived before ranking or comparison?
- Should ties produce one row or several?

Question

Use a window average to return resolution durations above the overall average.

Answer

```
WITH measured AS (
    SELECT id,
           title,
           TIMESTAMPDIFF(HOUR, created_at, resolved_at) AS resolution_hours
    FROM tickets
    WHERE resolved_at IS NOT NULL
),
compared AS (
    SELECT measured.*,
           AVG(resolution_hours) OVER () AS overall_avg_hours
    FROM measured
)
SELECT id, title, resolution_hours,
       ROUND(overall_avg_hours, 2) AS overall_avg_hours
FROM compared
```

```
WHERE resolution_hours > overall_avg_hours
ORDER BY resolution_hours DESC;
```

Expected Output

id	title	resolution_hours	overall_avg_hours
103	IDE licence	27	15.75
107	Guest Wi-Fi	24	15.75

How It Works: The window average is attached to every measured row, making the row-to-overall comparison direct. A surrounding CTE is needed because a window result cannot be filtered in the same query's **WHERE** phase.

Why This Is Asked in Interviews

Tests classic interview patterns using a real PulseDesk metric and explicit tie semantics.

Common Mistake

Ranking raw ticket rows before first deriving one metric row per subject.

Interview Follow-up

Solve the same pattern with a different tie rule or a correlated subquery.

Question 111 — Find records above their group average

Concepts: duplicates, ranking, group-wise comparison, latest/missing-child patterns

Difficulty: Advanced

Relevant Table — tickets

id	category	created_at	resolved_at
103	SOFTWARE	2026-09-01 09:00	2026-09-02 12:00
106	SOFTWARE	2026-09-10 08:00	2026-09-10 16:00
101	NETWORK	2026-09-01 09:00	2026-09-02 09:00
109	NETWORK	2026-09-10 08:00	2026-09-10 14:00

Think About It

- What value must be derived before ranking or comparison?
- Should ties produce one row or several?

Question

Return tickets whose resolution duration is above their own category's average.

Answer

```
WITH measured AS (
    SELECT id,
           category,
           TIMESTAMPDIFF(HOUR, created_at, resolved_at) AS resolution_hours
    FROM tickets
    WHERE resolved_at IS NOT NULL
),
compared AS (
    SELECT measured.*,
           AVG(resolution_hours) OVER (PARTITION BY category)
           AS category_avg_hours
    FROM measured
)
SELECT id,
       category,
       resolution_hours,
       category_avg_hours
FROM compared
WHERE resolution_hours > category_avg_hours
ORDER BY category, id;
```

Expected Output

id	category	resolution_hours	category_avg_hours
101	NETWORK	24	15.00
103	SOFTWARE	27	17.50

How It Works: `PARTITION BY category` calculates a different comparison value for each category while retaining ticket-level rows.

Why This Is Asked in Interviews

Tests classic interview patterns using a real PulseDesk metric and explicit tie semantics.

Common Mistake

Ranking raw ticket rows before first deriving one metric row per subject.

Interview Follow-up

Solve the same pattern with a different tie rule or a correlated subquery.

Question 112 — Find the latest ticket per requester

Concepts: duplicates, ranking, group-wise comparison, latest/missing-child patterns

Difficulty: Advanced

Relevant Table — tickets

id	requester_id	title	created_at
101	2	VPN timeout	2026-09-14 09:00
103	2	IDE licence	2026-09-01 09:00
108	2	MFA reset	2026-09-16 08:00
102	5	Laptop battery	2026-09-14 10:00
105	5	Monitor flicker	2026-09-15 11:00

Think About It

- What value must be derived before ranking or comparison?
- Should ties produce one row or several?

Question

Return exactly one most recent ticket for each requester.

Answer

```
WITH ranked AS (  
    SELECT t.*,  
           ROW_NUMBER() OVER (  
               PARTITION BY requester_id  
               ORDER BY created_at DESC, id DESC  
           ) AS rn  
    FROM tickets AS t  
)  
SELECT id, requester_id, title, created_at  
FROM ranked  
WHERE rn = 1  
ORDER BY requester_id;
```

Expected Output

id	requester_id	title	created_at
108	2	MFA reset	2026-09-16 08:00
105	5	Monitor flicker	2026-09-15 11:00

How It Works: This is the “latest record per group” pattern. The unique ID resolves equal timestamps deterministically.

Why This Is Asked in Interviews

Tests classic interview patterns using a real PulseDesk metric and explicit tie semantics.

Common Mistake

Ranking raw ticket rows before first deriving one metric row per subject.

Interview Follow-up

Solve the same pattern with a different tie rule or a correlated subquery.

Question 113 — Find users without matching child tickets

Concepts: duplicates, ranking, group-wise comparison, latest/missing-child patterns

Difficulty: Advanced

Relevant Table — **users**

user_id	name
2	Ravi Shah
5	Neha Joshi
6	Ishan Bose

Relevant Table — **tickets**

id	requester_id
101	2
102	5

Think About It

- What value must be derived before ranking or comparison?
- Should ties produce one row or several?

Question

Return users who have never requested a ticket.

Answer

```
SELECT u.user_id, u.name
FROM users AS u
WHERE NOT EXISTS (
  SELECT 1
  FROM tickets AS t
  WHERE t.requester_id = u.user_id
);
```

Expected Output

user_id	name
6	Ishan Bose

How It Works: This is the parent-without-child anti-semi-join. The equivalent left-join form checks **WHERE t.id IS NULL**.

Why This Is Asked in Interviews

Tests classic interview patterns using a real PulseDesk metric and explicit tie semantics.

Common Mistake

Ranking raw ticket rows before first deriving one metric row per subject.

Interview Follow-up

Solve the same pattern with a different tie rule or a correlated subquery.

Question 114 — Find parent tickets with multiple child comments

Concepts: duplicates, ranking, group-wise comparison, latest/missing-child patterns

Difficulty: Advanced

Relevant Table — **tickets**

id	title
101	VPN timeout
102	Laptop battery

id	title
103	IDE licence

Relevant Table — **comments**

id	ticket_id
1001	101
1002	101
1003	101
1004	103

Think About It

- What value must be derived before ranking or comparison?
- Should ties produce one row or several?

Question

Relationship: `tickets.id (1)` —>< `(N) comments.ticket_id`.

Return parent tickets having more than one comment child.

Answer

```
SELECT t.id,
       t.title,
       COUNT(c.id) AS comment_count
FROM tickets AS t
JOIN comments AS c
  ON c.ticket_id = t.id
GROUP BY t.id, t.title
HAVING COUNT(c.id) > 1
ORDER BY comment_count DESC, t.id;
```

Expected Output

id	title	comment_count
101	VPN timeout	3

How It Works: Joining materializes child rows, grouping restores parent grain, and **HAVING** selects parents with the required multiplicity.

Why This Is Asked in Interviews

Tests classic interview patterns using a real PulseDesk metric and explicit tie semantics.

Common Mistake

Ranking raw ticket rows before first deriving one metric row per subject.

Interview Follow-up

Solve the same pattern with a different tie rule or a correlated subquery.

Part 15 — Classic Employee/Salary Interview Questions

Assumed tables: The following **departments** and **employees** tables do not exist in PulseDesk. They are introduced only because salary problems are common in SQL interviews and salary has no honest equivalent in the PulseDesk schema. Every question reuses this one dataset and repeats the relevant rows so it remains self-contained.

The salary chain progresses from overall ranking to department grouping, group-wise ranking, averages, duplicate values, subqueries, and **HAVING**.

Question chain: Questions 115–129 reuse the exact same eight employees and three departments. The chain moves from highest/second/Nth/top-N values to department aggregation, group-wise maxima, overall and group averages, duplicate salaries, **HAVING**, and conditional aggregation.

Question 115 — Find the highest salary

Concepts: assumed Employee/Department model, salary aggregates, ranking, subqueries

Difficulty: Intermediate / Advanced

Relevant Table — assumed **employees**

employee_id	employee_name	department_id	salary
1	Asha	10	90000
2	Meera	10	70000
3	Kabir	10	70000
4	Ravi	20	120000
5	Neha	20	100000
6	Ishan	20	80000
7	Tara	30	75000
8	Omar	30	60000

Think About It

- Is the question about employee rows, distinct salary levels, or department groups?
- Must tied salaries be preserved?

Question

Return the employee or employees earning the highest salary.

Answer

```
SELECT employee_id, employee_name, salary
FROM employees
WHERE salary = (SELECT MAX(salary) FROM employees);
```

Expected Output

employee_id	employee_name	salary
4	Ravi	120000

How It Works: The scalar aggregate subquery returns **120000**; the outer query returns every employee whose salary equals it, so ties would be preserved.

Why This Is Asked in Interviews

Tests a classic salary pattern that interviewers commonly ask even though PulseDesk has no salary data.

Common Mistake

Confusing top N rows with top N distinct salaries or failing to partition department rankings.

Interview Follow-up

How would the result change if several employees tied at the boundary?

Question 116 — Find the second-highest distinct salary

Concepts: assumed Employee/Department model, salary aggregates, ranking, subqueries

Difficulty: Intermediate / Advanced

Relevant Table — assumed employees

employee_id	employee_name	department_id	salary
1	Asha	10	90000
2	Meera	10	70000
3	Kabir	10	70000
4	Ravi	20	120000
5	Neha	20	100000
6	Ishan	20	80000
7	Tara	30	75000
8	Omar	30	60000

Think About It

- Is the question about employee rows, distinct salary levels, or department groups?
- Must tied salaries be preserved?

Question

Return employees earning the second-highest distinct salary.

Answer

```
SELECT employee_id, employee_name, salary
FROM employees
WHERE salary = (
    SELECT MAX(salary)
    FROM employees
    WHERE salary < (SELECT MAX(salary) FROM employees)
);
```

Expected Output

employee_id	employee_name	salary
5	Neha	100000

How It Works: The innermost query finds **120000**. Values below it remain, and their maximum is **100000**; the outer query returns all employees at that value.

Why This Is Asked in Interviews

Tests a classic salary pattern that interviewers commonly ask even though PulseDesk has no salary data.

Common Mistake

Confusing top N rows with top N distinct salaries or failing to partition department rankings.

Interview Follow-up

How would the result change if several employees tied at the boundary?

Question 117 — Find the Nth-highest distinct salary

Concepts: assumed Employee/Department model, salary aggregates, ranking, subqueries

Difficulty: Intermediate / Advanced

Relevant Table — assumed employees

employee_id	employee_name	department_id	salary
1	Asha	10	90000
2	Meera	10	70000
3	Kabir	10	70000
4	Ravi	20	120000
5	Neha	20	100000
6	Ishan	20	80000
7	Tara	30	75000
8	Omar	30	60000

Think About It

- Is the question about employee rows, distinct salary levels, or department groups?
- Must tied salaries be preserved?

Question

With **N = 3**, return every employee at the third-highest distinct salary.

Answer

```
WITH ranked AS (
  SELECT e.*,
         DENSE_RANK() OVER (ORDER BY salary DESC) AS salary_rank
  FROM employees AS e
)
SELECT employee_id, employee_name, salary
FROM ranked
WHERE salary_rank = 3
ORDER BY employee_id;
```

Expected Output

employee_id	employee_name	salary
1	Asha	90000

How It Works: `DENSE_RANK` ranks distinct values without gaps. Filtering rank 3 returns the third salary level and preserves ties.

Why This Is Asked in Interviews

Tests a classic salary pattern that interviewers commonly ask even though PulseDesk has no salary data.

Common Mistake

Confusing top N rows with top N distinct salaries or failing to partition department rankings.

Interview Follow-up

How would the result change if several employees tied at the boundary?

Question 118 — Return the top three employees by salary

Concepts: assumed Employee/Department model, salary aggregates, ranking, subqueries

Difficulty: Intermediate / Advanced

Relevant Table — assumed employees

employee_id	employee_name	department_id	salary
1	Asha	10	90000
2	Meera	10	70000
3	Kabir	10	70000
4	Ravi	20	120000
5	Neha	20	100000
6	Ishan	20	80000
7	Tara	30	75000
8	Omar	30	60000

Think About It

- Is the question about employee rows, distinct salary levels, or department groups?
- Must tied salaries be preserved?

Question

Return exactly three employees with the highest salaries, using employee ID as the tie-breaker.

Answer

```
SELECT employee_id, employee_name, salary
FROM employees
ORDER BY salary DESC, employee_id
LIMIT 3;
```

Expected Output

employee_id	employee_name	salary
4	Ravi	120000
5	Neha	100000

employee_id	employee_name	salary
1	Asha	90000

How It Works: Sorting ranks rows and **LIMIT 3** returns exactly three. A rank filter would be needed to include every tie at the third salary level.

Why This Is Asked in Interviews

Tests a classic salary pattern that interviewers commonly ask even though PulseDesk has no salary data.

Common Mistake

Confusing top N rows with top N distinct salaries or failing to partition department rankings.

Interview Follow-up

How would the result change if several employees tied at the boundary?

Question 119 — Show every employee with a department name

Concepts: assumed Employee/Department model, salary aggregates, ranking, subqueries

Difficulty: Intermediate / Advanced

Relevant Table — assumed **employees**

employee_id	employee_name	department_id	salary
1	Asha	10	90000
2	Meera	10	70000
3	Kabir	10	70000
4	Ravi	20	120000
5	Neha	20	100000
6	Ishan	20	80000
7	Tara	30	75000
8	Omar	30	60000

Relevant Table — assumed **departments**

department_id	department_name
10	Support
20	Engineering
30	Operations

Relationship: `departments.department_id (1) ———< (N) employees.department_id`

Think About It

- Is the question about employee rows, distinct salary levels, or department groups?
- Must tied salaries be preserved?

Question

Show department-wise employees, highest salary first within each department.

Answer

```
SELECT d.department_name,
       e.employee_name,
       e.salary
FROM departments AS d
JOIN employees AS e
  ON e.department_id = d.department_id
ORDER BY d.department_name, e.salary DESC, e.employee_id;
```

Expected Output

department_name	employee_name	salary
Engineering	Ravi	120000
Engineering	Neha	100000

department_name	employee_name	salary
Engineering	Ishan	80000
Operations	Tara	75000
Operations	Omar	60000
Support	Asha	90000
Support	Meera	70000
Support	Kabir	70000

How It Works: The foreign-key-style equality supplies the department label; ordering first groups presentation by department and then ranks salaries.

Why This Is Asked in Interviews

Tests a classic salary pattern that interviewers commonly ask even though PulseDesk has no salary data.

Common Mistake

Confusing top N rows with top N distinct salaries or failing to partition department rankings.

Interview Follow-up

How would the result change if several employees tied at the boundary?

Question 120 — Calculate total salary per department

Concepts: assumed Employee/Department model, salary aggregates, ranking, subqueries

Difficulty: Intermediate / Advanced

Relevant Table — assumed employees

employee_id	employee_name	department_id	salary
1	Asha	10	90000
2	Meera	10	70000
3	Kabir	10	70000
4	Ravi	20	120000
5	Neha	20	100000
6	Ishan	20	80000
7	Tara	30	75000
8	Omar	30	60000

Relevant Table — assumed departments

department_id	department_name
10	Support
20	Engineering
30	Operations

Relationship: departments.department_id (1) —< (N) employees.department_id

Think About It

- Is the question about employee rows, distinct salary levels, or department groups?
- Must tied salaries be preserved?

Question

Return the salary total for each department, largest total first.

Answer

```
SELECT d.department_name,
       SUM(e.salary) AS total_salary
FROM departments AS d
JOIN employees AS e
  ON e.department_id = d.department_id
```

```
GROUP BY d.department_id, d.department_name
ORDER BY total_salary DESC;
```

Expected Output

department_name	total_salary
Engineering	300000
Support	230000
Operations	135000

How It Works: The join labels rows, grouping forms departments, and **SUM** adds salaries inside each group.

Why This Is Asked in Interviews

Tests a classic salary pattern that interviewers commonly ask even though PulseDesk has no salary data.

Common Mistake

Confusing top N rows with top N distinct salaries or failing to partition department rankings.

Interview Follow-up

How would the result change if several employees tied at the boundary?

Question 121 — Calculate department average, minimum, and maximum salary

Concepts: assumed Employee/Department model, salary aggregates, ranking, subqueries

Difficulty: Intermediate / Advanced

Relevant Table — assumed **employees**

employee_id	employee_name	department_id	salary
1	Asha	10	90000
2	Meera	10	70000
3	Kabir	10	70000
4	Ravi	20	120000
5	Neha	20	100000
6	Ishan	20	80000
7	Tara	30	75000
8	Omar	30	60000

Relevant Table — assumed **departments**

department_id	department_name
10	Support
20	Engineering
30	Operations

Relationship: departments.department_id (1) —< (N) employees.department_id

Think About It

- Is the question about employee rows, distinct salary levels, or department groups?
- Must tied salaries be preserved?

Question

Return average, minimum, and maximum salary for each department.

Answer

```
SELECT d.department_name,
       ROUND(AVG(e.salary), 2) AS avg_salary,
       MIN(e.salary) AS min_salary,
       MAX(e.salary) AS max_salary
FROM departments AS d
JOIN employees AS e
```

```
ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name
ORDER BY d.department_name;
```

Expected Output

department_name	avg_salary	min_salary	max_salary
Engineering	100000.00	80000	120000
Operations	67500.00	60000	75000
Support	76666.67	70000	90000

How It Works: The three aggregates summarize the same department groups in different ways.

Why This Is Asked in Interviews

Tests a classic salary pattern that interviewers commonly ask even though PulseDesk has no salary data.

Common Mistake

Confusing top N rows with top N distinct salaries or failing to partition department rankings.

Interview Follow-up

How would the result change if several employees tied at the boundary?

Question 122 — Find the highest salary in each department

Concepts: assumed Employee/Department model, salary aggregates, ranking, subqueries

Difficulty: Intermediate / Advanced

Relevant Table — assumed employees

employee_id	employee_name	department_id	salary
1	Asha	10	90000
2	Meera	10	70000
3	Kabir	10	70000
4	Ravi	20	120000
5	Neha	20	100000
6	Ishan	20	80000
7	Tara	30	75000
8	Omar	30	60000

Relevant Table — assumed departments

department_id	department_name
10	Support
20	Engineering
30	Operations

Relationship: departments.department_id (1) —< (N) employees.department_id

Think About It

- Is the question about employee rows, distinct salary levels, or department groups?
- Must tied salaries be preserved?

Question

Return the highest-paid employee in each department, preserving ties.

Answer

```
WITH ranked AS (
  SELECT e.*,
         DENSE_RANK() OVER (
           PARTITION BY department_id
           ORDER BY salary DESC

```

```
        ) AS salary_rank
    FROM employees AS e
)
SELECT d.department_name,
       r.employee_name,
       r.salary
FROM ranked AS r
JOIN departments AS d
  ON d.department_id = r.department_id
WHERE r.salary_rank = 1
ORDER BY d.department_name, r.employee_id;
```

Expected Output

department_name	employee_name	salary
Engineering	Ravi	120000
Operations	Tara	75000
Support	Asha	90000

How It Works: The rank restarts for each department, and rank 1 identifies the department-wise maximum while retaining ties.

Why This Is Asked in Interviews

Tests a classic salary pattern that interviewers commonly ask even though PulseDesk has no salary data.

Common Mistake

Confusing top N rows with top N distinct salaries or failing to partition department rankings.

Interview Follow-up

How would the result change if several employees tied at the boundary?

Question 123 — Find the second-highest salary in each department

Concepts: assumed Employee/Department model, salary aggregates, ranking, subqueries

Difficulty: Intermediate / Advanced

Relevant Table — assumed employees

employee_id	employee_name	department_id	salary
1	Asha	10	90000
2	Meera	10	70000
3	Kabir	10	70000
4	Ravi	20	120000
5	Neha	20	100000
6	Ishan	20	80000
7	Tara	30	75000
8	Omar	30	60000

Relevant Table — assumed departments

department_id	department_name
10	Support
20	Engineering
30	Operations

Relationship: departments.department_id (1) —< (N) employees.department_id

Think About It

- Is the question about employee rows, distinct salary levels, or department groups?
- Must tied salaries be preserved?

Question

Return employees at the second-highest distinct salary within each department.

Answer

```
WITH ranked AS (  
    SELECT e.*,  
           DENSE_RANK() OVER (  
               PARTITION BY department_id  
               ORDER BY salary DESC  
           ) AS salary_rank  
    FROM employees AS e  
)  
SELECT d.department_name,  
       r.employee_name,  
       r.salary  
FROM ranked AS r  
JOIN departments AS d  
    ON d.department_id = r.department_id  
WHERE r.salary_rank = 2  
ORDER BY d.department_name, r.employee_id;
```

Expected Output

department_name	employee_name	salary
Engineering	Neha	100000
Operations	Omar	60000
Support	Meera	70000
Support	Kabir	70000

How It Works: **DENSE_RANK** treats Meera and Kabir's equal salaries as the same second salary level and returns both.

Why This Is Asked in Interviews

Tests a classic salary pattern that interviewers commonly ask even though PulseDesk has no salary data.

Common Mistake

Confusing top N rows with top N distinct salaries or failing to partition department rankings.

Interview Follow-up

How would the result change if several employees tied at the boundary?

Question 124 — Find employees earning above the overall average

Concepts: assumed Employee/Department model, salary aggregates, ranking, subqueries

Difficulty: Intermediate / Advanced

Relevant Table — assumed employees

employee_id	employee_name	department_id	salary
1	Asha	10	90000
2	Meera	10	70000
3	Kabir	10	70000
4	Ravi	20	120000
5	Neha	20	100000
6	Ishan	20	80000
7	Tara	30	75000
8	Omar	30	60000

Think About It

- Is the question about employee rows, distinct salary levels, or department groups?
- Must tied salaries be preserved?

Question

Return employees earning more than the overall average salary.

Answer

```
SELECT employee_id, employee_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees)
ORDER BY salary DESC;
```

Expected Output

employee_id	employee_name	salary
4	Ravi	120000
5	Neha	100000
1	Asha	90000

How It Works: The subquery returns **83125**; the outer query compares each salary with that one scalar value.

Why This Is Asked in Interviews

Tests a classic salary pattern that interviewers commonly ask even though PulseDesk has no salary data.

Common Mistake

Confusing top N rows with top N distinct salaries or failing to partition department rankings.

Interview Follow-up

How would the result change if several employees tied at the boundary?

Question 125 — Find employees earning above their department average

Concepts: assumed Employee/Department model, salary aggregates, ranking, subqueries

Difficulty: Intermediate / Advanced

Relevant Table — assumed **employees**

employee_id	employee_name	department_id	salary
1	Asha	10	90000
2	Meera	10	70000
3	Kabir	10	70000
4	Ravi	20	120000
5	Neha	20	100000
6	Ishan	20	80000
7	Tara	30	75000
8	Omar	30	60000

Relevant Table — assumed **departments**

department_id	department_name
10	Support
20	Engineering
30	Operations

Relationship: departments.department_id (1) —< (N) employees.department_id

Think About It

- Is the question about employee rows, distinct salary levels, or department groups?
- Must tied salaries be preserved?

Question

Return employees whose salary is strictly above their own department average.

Answer

```
SELECT d.department_name,
       e.employee_name,
       e.salary
```

```
FROM employees AS e
JOIN departments AS d
  ON d.department_id = e.department_id
WHERE e.salary > (
  SELECT AVG(e2.salary)
  FROM employees AS e2
  WHERE e2.department_id = e.department_id
)
ORDER BY d.department_name;
```

Expected Output

department_name	employee_name	salary
Engineering	Ravi	120000
Operations	Tara	75000
Support	Asha	90000

How It Works: The correlated subquery calculates a different average for the current employee's department. Neha equals Engineering's average, so the strict > excludes her.

Why This Is Asked in Interviews

Tests a classic salary pattern that interviewers commonly ask even though PulseDesk has no salary data.

Common Mistake

Confusing top N rows with top N distinct salaries or failing to partition department rankings.

Interview Follow-up

How would the result change if several employees tied at the boundary?

Question 126 — Find employees who share the same salary

Concepts: assumed Employee/Department model, salary aggregates, ranking, subqueries

Difficulty: Intermediate / Advanced

Relevant Table — assumed employees

employee_id	employee_name	department_id	salary
1	Asha	10	90000
2	Meera	10	70000
3	Kabir	10	70000
4	Ravi	20	120000
5	Neha	20	100000
6	Ishan	20	80000
7	Tara	30	75000
8	Omar	30	60000

Think About It

- Is the question about employee rows, distinct salary levels, or department groups?
- Must tied salaries be preserved?

Question

Return employees whose salary value occurs more than once.

Answer

```
SELECT employee_id, employee_name, salary
FROM employees
WHERE salary IN (
  SELECT salary
  FROM employees
  GROUP BY salary
  HAVING COUNT(*) > 1
)
ORDER BY salary DESC, employee_id;
```

Expected Output

employee_id	employee_name	salary
2	Meera	70000
3	Kabir	70000

How It Works: The inner grouped query returns duplicated salary values, here {70000}; the outer query returns the actual employee rows at those values.

Why This Is Asked in Interviews

Tests a classic salary pattern that interviewers commonly ask even though PulseDesk has no salary data.

Common Mistake

Confusing top N rows with top N distinct salaries or failing to partition department rankings.

Interview Follow-up

How would the result change if several employees tied at the boundary?

Question 127 — Find departments whose total salary exceeds 200000

Concepts: assumed Employee/Department model, salary aggregates, ranking, subqueries

Difficulty: Intermediate / Advanced

Relevant Table — assumed employees

employee_id	employee_name	department_id	salary
1	Asha	10	90000
2	Meera	10	70000
3	Kabir	10	70000
4	Ravi	20	120000
5	Neha	20	100000
6	Ishan	20	80000
7	Tara	30	75000
8	Omar	30	60000

Relevant Table — assumed departments

department_id	department_name
10	Support
20	Engineering
30	Operations

Relationship: departments.department_id (1) —< (N) employees.department_id

Think About It

- Is the question about employee rows, distinct salary levels, or department groups?
- Must tied salaries be preserved?

Question

Return departments whose salary total is greater than 200000.

Answer

```
SELECT d.department_name,  
       SUM(e.salary) AS total_salary  
FROM departments AS d  
JOIN employees AS e  
  ON e.department_id = d.department_id  
GROUP BY d.department_id, d.department_name  
HAVING SUM(e.salary) > 200000  
ORDER BY total_salary DESC;
```

Expected Output

department_name	total_salary
Engineering	300000
Support	230000

How It Works: **HAVING** tests department totals after **SUM**; **WHERE SUM(...)** would be invalid because row filtering happens before grouping.

Why This Is Asked in Interviews

Tests a classic salary pattern that interviewers commonly ask even though PulseDesk has no salary data.

Common Mistake

Confusing top N rows with top N distinct salaries or failing to partition department rankings.

Interview Follow-up

How would the result change if several employees tied at the boundary?

Question 128 — Find departments whose average salary exceeds 75000

Concepts: assumed Employee/Department model, salary aggregates, ranking, subqueries

Difficulty: Intermediate / Advanced

Relevant Table — assumed **employees**

employee_id	employee_name	department_id	salary
1	Asha	10	90000
2	Meera	10	70000
3	Kabir	10	70000
4	Ravi	20	120000
5	Neha	20	100000
6	Ishan	20	80000
7	Tara	30	75000
8	Omar	30	60000

Relevant Table — assumed **departments**

department_id	department_name
10	Support
20	Engineering
30	Operations

Relationship: **departments.department_id (1) —< (N) employees.department_id**

Think About It

- Is the question about employee rows, distinct salary levels, or department groups?
- Must tied salaries be preserved?

Question

Return departments whose average salary is greater than 75000.

Answer

```
SELECT d.department_name,  
       ROUND(AVG(e.salary), 2) AS avg_salary  
FROM departments AS d  
JOIN employees AS e  
  ON e.department_id = d.department_id  
GROUP BY d.department_id, d.department_name  
HAVING AVG(e.salary) > 75000  
ORDER BY avg_salary DESC;
```

Expected Output

department_name	avg_salary
Engineering	100000.00
Support	76666.67

How It Works: The threshold applies to one value per department after aggregation, so it belongs in **HAVING**.

Why This Is Asked in Interviews

Tests a classic salary pattern that interviewers commonly ask even though PulseDesk has no salary data.

Common Mistake

Confusing top N rows with top N distinct salaries or failing to partition department rankings.

Interview Follow-up

How would the result change if several employees tied at the boundary?

Question 129 — Count high earners per department with conditional aggregation

Concepts: assumed Employee/Department model, salary aggregates, ranking, subqueries

Difficulty: Intermediate / Advanced

Relevant Table — assumed employees

employee_id	employee_name	department_id	salary
1	Asha	10	90000
2	Meera	10	70000
3	Kabir	10	70000
4	Ravi	20	120000
5	Neha	20	100000
6	Ishan	20	80000
7	Tara	30	75000
8	Omar	30	60000

Relevant Table — assumed departments

department_id	department_name
10	Support
20	Engineering
30	Operations

Relationship: departments.department_id (1) —< (N) employees.department_id

Think About It

- Is the question about employee rows, distinct salary levels, or department groups?
- Must tied salaries be preserved?

Question

For each department, count all employees and those earning at least 90000; retain departments with at least one high earner.

Answer

```
SELECT d.department_name,
       COUNT(*) AS employee_count,
       SUM(CASE WHEN e.salary >= 90000 THEN 1 ELSE 0 END) AS high_earner_count
FROM departments AS d
JOIN employees AS e
  ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name
HAVING SUM(CASE WHEN e.salary >= 90000 THEN 1 ELSE 0 END) >= 1
ORDER BY high_earner_count DESC, d.department_name;
```

Expected Output

department_name	employee_count	high_earner_count
Engineering	3	2
Support	3	1

How It Works: The `CASE` emits 1 for qualifying employees, `SUM` counts them per department, and `HAVING` removes Operations because its conditional count is zero.

Why This Is Asked in Interviews

Tests a classic salary pattern that interviewers commonly ask even though PulseDesk has no salary data.

Common Mistake

Confusing top N rows with top N distinct salaries or failing to partition department rankings.

Interview Follow-up

How would the result change if several employees tied at the boundary?

Part 16 — Important MySQL Conceptual Interview Questions

These questions are conceptual rather than query exercises, so they do not need sample input tables. A strong interview answer defines the concept, explains why it matters, gives a PulseDesk example, and mentions the main trade-off.

C1. What is the difference between SQL and MySQL?

SQL is the language standard for defining, querying, and changing relational data. MySQL is a database management system that implements SQL plus MySQL-specific behavior and extensions. `SELECT`, joins, and grouping are SQL ideas; `JSON_TABLE`, `MATCH ... AGAINST`, storage engines, and MySQL's exact date functions are product-specific.

C2. What is SQL's logical query execution order?

The interview-level order is:

```
FROM / JOIN
WHERE
GROUP BY
HAVING
window functions
SELECT
DISTINCT
ORDER BY
LIMIT / OFFSET
```

It explains why an aggregate cannot normally appear in `WHERE`, why a left join can be changed by a later filter, and why a `SELECT` alias is unavailable to `WHERE`. Physical execution can differ because the optimizer is free to choose an equivalent plan.

C3. Primary key versus unique key?

A primary key uniquely identifies every row and is implicitly non-null. A table has one primary key, though it can contain several columns. A table can have several unique constraints. In MySQL, a unique index normally permits multiple `NULL` values because `NULL` values are not considered equal.

PulseDesk examples are `users.user_id` as a primary key, `users.email` as a unique alternate key, and `(ticket_id, asset_id)` as a composite primary key in `ticket_assets`.

C4. What is a foreign key?

A foreign key enforces that a child value refers to an existing parent key, subject to nullability. It protects relationships even if a client bypasses the application. `tickets.requester_id` must reference a user; `tickets.assignee_id` may be null but, when present, must reference a user.

An index improves foreign-key joins, but the constraint and the index solve different problems: integrity versus access speed.

C5. Explain PulseDesk's delete actions.

ON DELETE RESTRICT prevents deletion while important children exist; **ON DELETE SET NULL** preserves the child but removes an optional link; **ON DELETE CASCADE** deletes dependent rows.

PulseDesk restricts deleting a ticket requester, sets an assignee to null if that user is deleted, cascades ticket deletion into comments/notifications/bridge rows, and restricts deleting an asset linked by `ticket_assets`. Each action expresses a business lifecycle decision.

C6. What does **NULL** mean?

NULL means missing, unknown, or not applicable; it is not zero or an empty string. Most comparisons with it evaluate to unknown, so use **IS NULL**. **COUNT(column)** ignores it, aggregates generally ignore it, and **COALESCE(value, fallback)** supplies a display/calculation fallback.

In PulseDesk, null `assignee_id` means unassigned, null `resolved_at` means no resolution timestamp, and null `read_at` means unread.

C7. Why can **NOT IN** be dangerous with **NULL**?

If a subquery used by **NOT IN** returns even one **NULL**, comparisons such as `x <> NULL` become unknown and the predicate may return no rows. **NOT EXISTS** expresses anti-matching safely by checking correlated row existence. A non-null schema guarantee can make **NOT IN** safe, but the guarantee should be explicit.

C8. **COUNT(*)** versus **COUNT(column)** versus **COUNT(DISTINCT column)**?

COUNT(*) counts rows. **COUNT(column)** counts non-null values in that column. **COUNT(DISTINCT column)** counts different non-null values. In a **LEFT JOIN** parent/child report, count the child key, such as **COUNT(c.id)**, so a parent with no child gets zero rather than one.

C9. **WHERE** versus **HAVING**?

WHERE filters source rows before grouping; **HAVING** filters groups after aggregate calculation. Put `status='OPEN'` in **WHERE** to define which tickets participate, and `COUNT(*) > 5` in **HAVING** to retain large groups. A nonaggregate condition sometimes works in **HAVING**, but moving it to **WHERE** is usually clearer and allows earlier filtering.

C10. **GROUP BY** versus **DISTINCT**?

Both can remove duplicate projected values, but their intent differs. **DISTINCT** says “return unique result rows.” **GROUP BY** says “form groups so I can calculate per-group metrics.” Use **SELECT DISTINCT category** for a list of category values and **GROUP BY category** with **COUNT(*)** for a category report.

C11. What is **ONLY_FULL_GROUP_BY**?

It is a MySQL SQL mode that rejects selected columns that are neither grouped, aggregated, nor functionally dependent on the grouping key. It prevents arbitrary values from being chosen from a group. Keep it enabled; fix the query's reporting grain rather than disabling the protection.

C12. Inner join versus left join?

An inner join retains only matching pairs. A left join retains every left row plus matches; missing right columns become null. Ticket/requester is naturally inner because the requester is mandatory. Ticket/assignee is naturally left because unassigned tickets must remain visible.

C13. How can a **LEFT JOIN** accidentally become an inner join?

If a right-side condition appears in **WHERE**, null-extended rows fail it. For example, **LEFT JOIN tickets t ... WHERE t.status='OPEN'** removes users without tickets. Move the condition to **ON** when the goal is to preserve users and count only their open tickets.

C14. What is a self join?

A self join gives one table multiple aliases and treats them as separate row sources. It can find users sharing a role or compare rows from the same category. A condition such as `u2.user_id > u1.user_id` eliminates self-pairs and reversed duplicates.

C15. What is a cross join?

A cross join returns the Cartesian product: M rows times N rows. It is useful for building all category/status combinations or a date/report grid. It is dangerous when accidental because row counts can grow dramatically.

C16. **UNION** versus **UNION ALL**?

Both stack compatible result sets. **UNION** removes duplicate rows, which requires extra work; **UNION ALL** keeps duplicates and is normally faster. Use **UNION ALL** unless deduplication is a stated requirement. Column count and corresponding data types must be compatible, and final ordering belongs after the combined query.

C17. **IN** versus **EXISTS**?

IN compares a value with a set and is natural for small or clearly non-null sets. **EXISTS** asks whether a correlated match exists and can stop at the first match. Modern MySQL can transform either into a semi-join, so choose based on meaning and validate with **EXPLAIN**. Prefer **NOT EXISTS** over nullable **NOT IN**.

C18. Join versus subquery?

Neither form is universally faster. MySQL may transform them into similar plans. Joins are natural when columns from both sides are returned; **EXISTS** is natural when only presence matters; scalar subqueries are readable for a single comparison value. Avoid a correlated scalar subquery that repeats expensive work for a large outer result unless the plan shows it is optimized.

C19. What is a CTE?

A common table expression names a query result for the duration of one statement. It helps split workload derivation, ranking, and filtering into understandable phases. A recursive CTE refers to itself and can generate a hierarchy or date series. A CTE is primarily a query-structure tool; materialization and optimization behavior should be verified rather than assumed.

C20. **GROUP BY** versus window functions?

GROUP BY collapses many rows to one row per group. A window function keeps row detail and calculates across a partition. Use grouping for one count per agent; use **AVG(duration) OVER (PARTITION BY category)** when every ticket row must retain its own duration plus the category average.

C21. **ROW_NUMBER**, **RANK**, and **DENSE_RANK**?

- **ROW_NUMBER** gives every row a unique sequence, even when values tie.
- **RANK** gives ties the same rank and leaves gaps afterward: 1, 1, 3.
- **DENSE_RANK** gives ties the same rank without gaps: 1, 1, 2.

Use **ROW_NUMBER** for exactly one latest ticket per requester; use **DENSE_RANK** for Nth-highest distinct workload including ties.

C22. **CHAR**, **VARCHAR**, and **TEXT**?

CHAR(n) is fixed length and suits truly fixed codes. **VARCHAR(n)** stores variable-length strings and is appropriate for names, emails, statuses, and titles. **TEXT** suits larger content such as descriptions and messages but has different default/indexing rules. PulseDesk uses **VARCHAR** for structured short text and **TEXT/LONGTEXT** for descriptions, messages, and stack traces.

C23. **DATETIME** versus **TIMESTAMP**?

DATETIME represents a calendar date/time without automatic session-time-zone conversion and has a wide range. **TIMESTAMP** is stored in UTC and converted between the session time zone and UTC, with a narrower range. Choose based on semantics and define an application time-zone policy. PulseDesk uses **DATETIME(6)** for operational timestamps, retaining microseconds.

C24. Why is **users.create_dt VARCHAR** a problem?

The database cannot validate date values, natural date functions require parsing, index range scans become difficult, and inconsistent formats break lexical sorting. A safe migration would add a **DATETIME(6)** column, backfill validated conversions, update application writes, verify, then replace the old column.

C25. **DECIMAL** versus **FLOAT/DOUBLE**?

DECIMAL stores exact fixed-point values and is the default for money. Floating-point types are approximate and suit scientific measurements. PulseDesk has no money column, but a future asset purchase price should use a suitable **DECIMAL(p,s)**, never binary floating point.

C26. What is an index?

An index is an auxiliary structure that lets MySQL locate ordered key values without scanning every row. It speeds selected reads and can support ordering/grouping, but consumes storage and makes inserts/updates/deletes more expensive. Index columns used by selective filters, join keys, and common orderings; do not index every column blindly.

C27. Explain the leftmost-prefix rule.

For a composite B-tree index (**assignee_id, status, created_at, id**), MySQL can efficiently use leading sequences such as **assignee_id**, or **assignee_id + status**, then possibly a range/order on **created_at**. A query filtering only **created_at** generally cannot seek through that index efficiently because the preceding values are unknown.

C28. Why does column order matter in a composite index?

Order should match important query patterns: equality predicates typically come before range predicates, and useful ordering may follow. PulseDesk has separate indexes because ticket lists filter/order by different leading dimensions: status, priority, assignee, requester, and resolution timestamp. One wide index cannot efficiently lead with every possible filter.

C29. What is a covering index?

An index covers a query when it contains all columns needed for its predicates and output, so InnoDB need not fetch the base row. Coverage can reduce I/O but larger indexes cost memory and write work. **EXPLAIN** may show **Using index**; that phrase means covering in this context, not merely that an index was used.

C30. How does InnoDB organize primary and secondary indexes?

InnoDB stores table rows in the clustered primary-key B-tree. Secondary-index leaf entries contain the indexed values plus the primary key, which InnoDB uses to fetch the full row. Therefore a wide or random primary key increases every secondary index. PulseDesk's numeric auto-increment keys are compact; the UUID string key in **error_logs** has different locality/storage trade-offs.

C31. What is sargability?

A sargable predicate can be used as an index search argument. `created_at >= start AND created_at < end` is sargable. `DATE(created_at) = target_date` often is not because MySQL must compute a function for rows. Other common index blockers include leading-wildcard `LIKE`, implicit type conversions, and functions applied to indexed columns.

C32. What does `EXPLAIN` tell you?

`EXPLAIN` describes table access order, join type, candidate/chosen indexes, estimated rows, filtered percentage, and extra operations such as temporary tables or filesorts. `EXPLAIN ANALYZE` executes the statement and reports actual timing/rows, so use it carefully on expensive queries. Estimates are clues; compare them with actual behavior and current statistics.

C33. What are cardinality and selectivity?

Cardinality is the number of distinct values; selectivity describes how narrowly a predicate filters. A unique email is highly selective, while ticket status has only four possible values. A low-cardinality column can still be useful as the leading part of an index when paired with time/order columns and when a status slice is a common workload.

C34. What is a full-text index?

It is a token-based index for natural-language or Boolean text search, queried in MySQL with `MATCH(...) AGAINST(...)`. PulseDesk defines one over ticket title and description and another over asset tag and model. It provides relevance-based search that differs from substring `LIKE '%text%'`; tokenizer, stopword, minimum-token, and language behavior matter.

C35. What is normalization?

Normalization separates facts to reduce duplication and update anomalies. PulseDesk stores a role name once in `role`, links users by `role_id`, and uses `ticket_assets` for a many-to-many relation rather than repeating asset IDs inside a ticket. Third normal form roughly means non-key attributes depend on the key, the whole key, and nothing but the key.

C36. When would you denormalize?

Denormalize only for a measured read need or immutable historical snapshot, accepting synchronization responsibility. `notifications` stores both `ticket_id` and `comment_id` even though the comment implies its ticket; this makes notification lookup/direct integrity explicit but introduces redundant relationship information that must remain consistent.

C37. What are ACID properties?

- **Atomicity:** all operations in a transaction succeed or roll back together.
- **Consistency:** constraints and application rules move the database between valid states.
- **Isolation:** concurrent transactions behave according to the chosen isolation guarantees.
- **Durability:** committed changes survive failures according to the storage configuration.

Creating a ticket and its `ticket_assets` rows should be atomic so a failure does not leave a partially created business operation.

C38. What is autocommit?

With autocommit enabled, each statement is its own transaction unless an explicit transaction is active. Multi-statement business operations need a transaction boundary, commonly Spring's `@Transactional`. Do not assume a stored procedure is automatically atomic merely because it is a procedure; transaction behavior still depends on the caller and procedure statements.

C39. Name the standard transaction isolation levels.

They are Read Uncommitted, Read Committed, Repeatable Read, and Serializable. InnoDB's default is normally Repeatable Read. Stronger isolation reduces allowed anomalies but can increase locking/conflicts. The correct level follows the invariant, not a general "strongest is best" rule.

C40. Dirty read, non-repeatable read, and phantom read?

A dirty read sees another transaction's uncommitted change. A non-repeatable read gets a different value when reading the same row twice. A phantom is a new/deleted row appearing in a repeated range query. InnoDB uses MVCC and, where necessary, locking reads/gap locks to implement isolation behavior.

C41. What is MVCC?

Multi-version concurrency control lets consistent reads see a transaction snapshot using row versions rather than blocking on every writer. It improves read/write concurrency. Long-running transactions keep old versions needed for their snapshots and can increase undo/history work, so transaction scope should remain short.

C42. What is a deadlock and how should an application handle it?

A deadlock occurs when transactions wait cyclically for locks held by each other. InnoDB detects it and rolls one transaction back. Applications should retry the whole transaction when safe, keep transactions short, access rows in a consistent order, and index predicates so updates lock fewer rows.

C43. Optimistic versus pessimistic concurrency?

Pessimistic concurrency locks data before competing work proceeds, such as `SELECT ... FOR UPDATE`. Optimistic concurrency reads without a long lock and updates only if a version is unchanged. PulseDesk has a `tickets.version` column and increments it, but the shown repository update does not include `AND version = :expectedVersion`; a complete optimistic-lock check would include that predicate and treat zero updated rows as a conflict.

C44. DELETE, TRUNCATE, and DROP?

DELETE removes selected rows and supports **WHERE**; it is DML and fires delete triggers. **TRUNCATE TABLE** removes all rows using DDL-like behavior, has no **WHERE**, resets auto-increment, and is restricted by foreign-key relationships. **DROP TABLE** removes the table definition and data. Transaction/implicit-commit behavior is MySQL-specific, so do not repeat SQL Server assumptions.

C45. What do CHECK constraints do in MySQL 8?

They reject rows whose condition is false; an unknown result normally passes unless a separate **NOT NULL** prevents it. MySQL enforced check constraints starting in 8.0.16. PulseDesk uses checks for allowed ticket category/priority/status, nonblank content, valid coverage dates, and assignment chronology.

C46. What are AUTO_INCREMENT and LAST_INSERT_ID() ?

AUTO_INCREMENT generates a numeric key when a row omits it. **LAST_INSERT_ID()** returns the generated value for the current connection, making it concurrency-safe for that session. PulseDesk's **create_ticket** procedure inserts a ticket, captures its ID, and uses it for **ticket_assets** rows.

C47. Why does PulseDesk use a generated column in assignment history?

active_asset_id becomes the asset ID only for open assignments and **NULL** for closed assignments. A unique index then allows many closed **NULL** rows but only one open row per asset ID. It turns "at most one current assignment" into a database-enforced invariant.

C48. View versus stored procedure?

A view is a named query used like a table and is suited to reusable read abstraction and permission boundaries. A stored procedure is invoked explicitly, accepts parameters, and can perform procedural/multi-statement work. PulseDesk's ticket creation procedure validates a requester, inserts the ticket, expands JSON asset IDs, and returns a result.

C49. What is a trigger and when should it be used?

A trigger runs automatically for row events. It can enforce cross-client auditing but hides work from application call sites, complicates debugging, and can create unexpected cascades. PulseDesk performs assignment-history writes explicitly in repositories/services, which makes the sequence visible; a trigger would trade that visibility for centralized enforcement.

C50. How do prepared statements prevent SQL injection?

They send SQL structure separately from bound data, so input remains a value rather than executable syntax. Spring Data named parameters such as **:ticketId** are bound values. Parameters cannot substitute identifiers or **ASC/DESC**, which is why sort fields/directions must be allowlisted rather than concatenated from unchecked input.

C51. Offset versus keyset pagination?

Offset pagination is simple and supports jumping to a page, but large offsets require locating/discarding earlier rows and concurrent inserts can shift results. Keyset pagination uses the last ordered key as a cursor, providing stable, efficient sequential navigation. Both require a deterministic order such as **(created_at DESC, id DESC)**.

C52. Native SQL versus JPQL in PulseDesk?

JPQL queries entity names and Java fields; native SQL queries physical tables and columns and can use MySQL features. **NotificationRepository** uses JPQL for entity updates such as **n.readAt**; ticket and asset repositories use native SQL for projections, joins, procedures, dynamic ordering, and MySQL syntax. Native SQL offers control but reduces database portability.

C53. What is the ORM N+1 query problem?

One query loads N parents, then lazy relationship access issues one query per parent, producing N+1 total queries. PulseDesk's entities store scalar foreign-key IDs rather than JPA relationships, so this particular lazy-navigation mechanism is absent; services may still perform repeated repository lookups, such as one role lookup per user, unless they join or batch data.

C54. What is unusual about PulseDesk's read entities?

AllUsersEntity, **AssetsUserEntity**, and **UserTicketEntity** are annotated as entities but primarily carry native-query/stored-procedure results. They have no **@Table** and do not represent corresponding physical tables in the supplied schema. With **ddl-auto=none**, Hibernate will not create them, but interface projections or dedicated DTO mappings communicate read-model intent more clearly.

C55. What type mismatches should be corrected?

Several DDL primary keys are **BIGINT**, while **TicketEntity**, **AssetsEntity**, projection getters, and DTOs use Java **Integer**. A MySQL **BIGINT** maps naturally to Java **Long**; keeping **Integer** risks overflow and conversion problems as IDs grow. Relationship IDs and procedure result types should be aligned end to end.

C56. Are optional-filter predicates always efficient?

The repository pattern **(:status IS NULL OR t.status = :status)** is convenient and safe for a small set of optional parameters. The **OR** can make index planning less selective than purpose-built query variants. For critical large queries, compare plans for null and non-null values or construct allowlisted predicates dynamically while keeping values parameterized.

C57. What is the trade-off in dynamic ORDER BY CASE?

PulseDesk maps an allowlisted sort choice to several **CASE** expressions. It avoids injecting an identifier and keeps one static query, but can prevent MySQL from using an index directly for ordering and makes SQL verbose. For high-volume paths, separate known query variants or a safe query builder may produce better plans.

C58. What constraints are present in code but not fully modeled in JPA?

The database contains foreign keys, cascading actions, checks, composite keys, full-text indexes, and a generated-column uniqueness invariant. The entities mostly expose scalar fields and do not declare relationships or all physical columns (for example ticket **updated_at** and **version**). Treat the DDL as the relational source of truth and keep the Java model synchronized deliberately.

C59. What schema changes would most improve PulseDesk?

The highest-value corrections are:

- 1. change **users.create_dt** from text to **DATETIME(6)**;
- 2. align every **BIGINT** ID with Java **Long**;
- 3. formalize migrations with Flyway or Liquibase rather than a monolithic recreate script;
- 4. add an actual optimistic version comparison or map **@Version**;
- 5. consider named constraints for every foreign key and add missing indexes only after workload analysis;
- 6. replace entity-annotated read carriers with explicit projections/DTOs.

These are observations for interview discussion; this study document does not modify the application.

C60. What do **COMMIT** and **ROLLBACK** do?

COMMIT makes the current transaction's changes durable and visible according to isolation rules. **ROLLBACK** abandons changes made since the transaction began, or **ROLLBACK TO SAVEPOINT** abandons a later portion. Neither command can rescue work after it has already been committed. In PulseDesk, ticket creation plus its **ticket_assets** inserts should commit together; an asset-link failure should roll back the ticket insert as part of the same transaction.

SQL Server to MySQL 8 conversion reference

This table addresses common syntax that makes SQL Server interview notes misleading in a MySQL interview.

SQL Server	MySQL 8	Important difference
SELECT TOP 5 ...	SELECT ... LIMIT 5	MySQL limit comes after ordering
OFFSET 10 ROWS FETCH NEXT 5 ROWS ONLY	LIMIT 5 OFFSET 10	Both require deterministic ORDER BY for paging
GETDATE()	NOW() or CURRENT_TIMESTAMP	MySQL also supports microsecond precision
ISNULL(x, y)	IFNULL(x, y) or COALESCE(x, y)	COALESCE is standard and accepts multiple values
LEN(text)	CHAR_LENGTH(text)	MySQL LENGTH counts bytes
DATEADD(day, 7, d)	d + INTERVAL 7 DAY or DATE_ADD	Unit syntax differs
DATEDIFF(day, start, end)	DATEDIFF(end, start)	MySQL DATEDIFF returns days and ignores time; use TIMESTAMPDIFF for other units
DATENAME(month, d)	DATE_FORMAT(d, '%M')	Formatting tokens differ
YEAR(d)	YEAR(d)	Same common form
[table]	`table`	Avoid quoting ordinary safe identifiers when possible
IDENTITY(1,1)	AUTO_INCREMENT	Generated-key retrieval uses LAST_INSERT_ID()
NEWID()	UUID()	MySQL returns a UUID string; storage design still matters
IIF(condition,a,b)	IF(condition,a,b)	CASE is more portable
+ for string concatenation	CONCAT(...) / CONCAT_WS(...)	+ is numeric addition in MySQL
STRING_AGG(x, ',')	GROUP_CONCAT(x SEPARATOR ',')	MySQL result length is controlled by group_concat_max_len
SELECT ... INTO new_table	CREATE TABLE new_table AS SELECT ...	MySQL SELECT ... INTO is used for variables/files
#temp	CREATE TEMPORARY TABLE ...	Temporary-table syntax and scope differ
CROSS APPLY / OUTER APPLY	Often LATERAL derived tables or rewrites	Confirm support/version and rewrite requirements
BIT used as Boolean	commonly BOOLEAN/TINYINT(1)	MySQL BOOLEAN is a synonym, not a separate Boolean storage class
NVARCHAR	VARCHAR with utf8mb4 charset	Unicode handling is charset/collation based
MERGE	INSERT ... ON DUPLICATE KEY UPDATE for common upserts	Semantics are not identical
clustered-index choice	InnoDB clusters by primary key	SQL Server permits a separately chosen clustered index

PulseDesk source-to-database mapping

Java type	Physical source/result	Query style
UserEntity	users	derived repository methods and normal JPA persistence
RoleEntity	role	derived repository methods
TicketEntity	tickets	native SQL plus stored procedure
AssetsEntity	native queries target assets; entity lacks explicit @Table	native SQL and writes
TicketCommentEntity	comments	JPA save plus native projection query
NotificationEntity	notifications	derived reads/counts and JPQL updates
ErrorLogEntity	error_logs	standard JPA repository
AllUsersEntity	joined users + role result	native paged read model; no physical table
AssetsUserEntity	joined assets + users result	native read model; no physical table
UserTicketEntity	result returned by create_ticket	stored-procedure result carrier; no physical table
TicketDetailsProjection	joined ticket/requester/assignee result	interface projection from native SQL
TicketCommentProjection	joined comment/author result	interface projection from native SQL

A reliable interview-solving routine

1. State the output grain: one row per ticket, user, category, agent, or month.

2. Identify the driving table and relationship path from the schema.

3. Decide which missing rows must survive; that determines inner versus outer joins.

4. Apply row filters in WHERE or optional-side filters in ON.

5. Group only when the desired grain is higher than source-row grain.

6. Use HAVING only for group conditions.

7. Check null behavior and one-to-many multiplication.

8. Define tie behavior for "top," "second," "latest," and "maximum."

9. Make ordering deterministic with a unique tiebreaker.

10. Consider indexes and validate important queries with EXPLAIN ANALYZE on safe representative data.

When explaining an answer aloud, say what rows FROM/JOIN creates, what WHERE removes, what each group represents, what the aggregate measures, and how sorting/limiting chooses the final rows. That demonstrates understanding rather than memorization.

Part 17 — Common Interview Traps and Mistakes

Trap	Why it is wrong	Better approach in PulseDesk
assignee_id = NULL	Comparisons with NULL are unknown	assignee_id IS NULL
NOT IN (subquery) when the set may contain NULL	One null can make every comparison unknown	Correlated NOT EXISTS
COUNT(*) after a parent LEFT JOIN	Counts the null-extended parent row as one	COUNT(child.id)
Right-side filter in WHERE after a LEFT JOIN	Removes unmatched parents	Put optional-child filters in ON
Selecting title while grouping only by status	A group can contain several titles	Select grouped columns or real aggregates
WHERE COUNT(*) > 5	Counts do not exist at the WHERE phase	GROUP BY ... HAVING COUNT(*) > 5
Filtering OPEN in HAVING	Delays a row-level condition and obscures intent	WHERE status = 'OPEN' before grouping
Joining comments and assets, then counting normally	Two one-to-many joins multiply rows	Pre-aggregate children or count distinct keys
Grouping by MONTH(created_at) alone	Merges the same month across years	Group by year plus month/month-start date
Ending a timestamp range at 23:59:59	Misses fractional seconds	Use >= start AND < next_boundary
DATE(created_at) = CURRENT_DATE on a large table	Often prevents an index range seek	Compare the raw timestamp with day boundaries
LIKE '%vpn%' for large text search	Leading wildcard normally scans	Use PulseDesk's full-text index when semantics fit
Paging without ORDER BY	Database row order is undefined	Use a stable order such as (created_at, id)
Paging with a nonunique sort key	Equal values can move/repeat between pages	Add the primary key as a tie-breaker
Assuming LIMIT 3 includes all ties	It returns exactly three rows	Use DENSE_RANK() <= 3 for top three levels

Trap	Why it is wrong	Better approach in PulseDesk
Using <code>ROW_NUMBER</code> for tied maxima	Arbitrarily chooses one tied row	Use <code>RANK/DENSE_RANK</code> when ties must survive
Integer division in percentages	Can truncate the fractional result	Multiply by <code>100.0</code> or cast to <code>DECIMAL</code>
Averaging per-user counts but omitting zero users	Changes the population and the average	Start from <code>users</code> and left join ticket counts
Treating <code>resolved_at IS NOT NULL</code> and <code>status='RESOLVED'</code> as identical	PulseDesk also timestamps <code>CLOSED</code> tickets	Define “completed” explicitly as the report requires
Assuming an assignee FK guarantees the <code>agent</code> role	The FK only guarantees a user exists	Join <code>role</code> and filter active agents
Hard-coding role ID 3	Seed IDs are implementation details	Join by key and filter <code>r.name='agent'</code>
Converting <code>users.create_dt</code> as if it were a true date column	It is <code>VARCHAR</code> in the actual schema	Parse cautiously; migrate it to <code>DATETIME(6)</code>
Mapping MySQL <code>BIGINT</code> to Java <code>Integer</code>	Can overflow before the database key does	Use Java <code>Long</code> consistently
Calling <code>LENGTH</code> a character count	It counts bytes	Use <code>CHAR_LENGTH</code> for characters
Copying SQL Server <code>TOP</code> , <code>GETDATE()</code> , or <code>[]</code>	Those are not MySQL query forms	Use <code>LIMIT</code> , <code>NOW()</code> , and normal/backtick identifiers
Disabling <code>ONLY_FULL_GROUP_BY</code> to make a query pass	Hides an ambiguous reporting grain	Correct the selected/grouped expressions
Concatenating a requested sort column into SQL	Allows injection through identifiers	Allowlist fields or use safe fixed query variants
Assuming a CTE always materializes or is always faster	Optimizer behavior varies by query/version	Use it for clarity and inspect the plan
Assuming indexes only help	Every index costs storage and write maintenance	Add indexes for measured, important access paths
Explaining syntax without row flow	Does not demonstrate query understanding	Walk through joins, filters, groups, aggregates, and final ordering

Final self-check before answering an interview query

- What is one output row supposed to represent?
- Which table supplies that grain?
- Which keys connect the tables, and what is each relationship's cardinality?
- Must unmatched parents remain?
- Does each condition filter rows, groups, or windowed results?
- Can `NULL` change the predicate or aggregate?
- Can a join multiply the rows I intend to count?
- What should happen when values tie?
- Is the output order deterministic?
- Does the syntax and function behavior belong to MySQL 8?